

RĪGAS VALSTS TEHNIKUMS

DATORIKAS NODAĻA

Izglītības programma: Programmēšana

KVALIFIKĀCIJAS DARBS

“Fizisko treniņu plānu un uztura izsekošanas sistēma”

Paskaidrojošais raksts 146 lpp.

Audzēknis:

Herberts Pauls Eisulis

Grupa:

DP4-3

Nodaļas vadītājs:

Katrīna Gandzjuka

Nodaļas vadītājs:

Normunds Barbāns

Rīga, 2026

ANOTĀCIJA

JustFitness mobilā lietotne ir izstrādāta, lai lietotāji varētu efektīvi pārvaldīt savus treniņu plānus, reģistrēt izpildītos vingrinājumus, sekot līdzi uzturam, uzņemtajam ūdenim, svara izmaiņām un personīgajiem veselības mērķiem. Sistēmas izstrādes laikā tika izmantotas tādas tehnoloģijas kā JavaScript, React Native, Expo, Node.js, Express.js un MySQL. Papildus tika izmantotas React Native, Expo Router, i18n-js un citas bibliotēkas, lai nodrošinātu mobilās lietotnes saskarni, navigāciju, tulkojumus un datu apmaiņu ar serveri.

Kvalifikācijas darbs ietver ievadu, uzdevumu nostādni, prasību specifikāciju, uzdevuma risināšanas līdzekļu izvēles pamatojumu, programmatūras produkta modelēšanas un projektēšanas aprakstu, datu struktūru aprakstu, lietotāja ceļvedi, programmatūras lietojamības testēšanu un secinājumus. Ievadā ir aprakstīta JustFitness mobilās lietotnes aktualitāte un nepieciešamība. Uzdevumu nostādnē ir norādīti uzdevumi, kurus sistēmai nepieciešams veikt. Prasību specifikācija sastāv no ieejas un izejas informācijas, kā arī no sistēmas funkcionālajām un nefunkcionālajām prasībām. Uzdevuma risināšanas līdzekļu izvēles pamatojumā ir norādīti izstrādē izmantotie rīki un to pielietojums. Programmatūras produkta modelēšanas un projektēšanas apraksts ietver sistēmas struktūras modeli, funkcionālo sistēmas modeli un lietotāja saskarnes skices. Datu struktūru aprakstā tiek parādīta datubāzes relāciju shēma un tabulu struktūra ar datu tipu norādīšanu. Lietotāja ceļvedī ir norādītas sistēmas prasības aparatūrai un programmatūrai, instalācija un palaišana, kā arī programmas lietošanas pamācība. Testa piemēros ir dots detalizēts programmatūras funkcionalitātes pārbaudes apraksts ar sagaidāmajiem rezultātiem.

Kvalifikācijas darbs sastāv no 147 lappusēm, kurās ietilpst 60 attēli, 24 tabulas un 1 pielikums. Pielikums satur programmas pirmkodu fragmentus.

ANNOTATION

The JustFitness mobile application was developed to allow users to effectively manage their workout plans, record completed exercises, track nutrition, water intake, weight changes and personal health goals. During the development of the system, technologies such as JavaScript, React Native, Expo, Node.js, Express.js and MySQL were used. Additionally, React Native Paper, Expo Router, i18n-js and other libraries were used to provide the mobile application interface, navigation, translations and data exchange with the server.

The qualification work includes an introduction, task statement, requirements specification, justification for the choice of solution development tools, software product modelling and design description, data structure description, user guide, software usability testing and conclusions. The introduction describes the relevance and necessity of the JustFitness mobile application. The task statement specifies the tasks that the system must perform. The requirements specification consists of input and output information, as well as the functional and non-functional requirements of the system. The justification for the choice of solution development tools describes the tools used in development and their application. The software product modelling and design description includes the system structure model, the functional system model and user interface sketches. The data structure description presents the database relational schema and the table structure with data types. The user guide specifies the hardware and software requirements of the system, installation and launch, as well as instructions for using the program. The test examples provide a detailed description of software functionality testing with expected results.

The qualification work consists of 147 pages, including 60 images, 24 tables and 1 appendice. The appendice contains fragments of the programs source code.

SATURS

IEVADS	5
1. UZDEVUMA NOSTĀDNE	7
2. PRASĪBU SPECIFIKĀCIJA	9
2.1. Ieejas un izejas informācijas apraksts.....	9
2.1.1. Ieejas informācijas apraksts.....	9
2.1.2. Izejas informācijas apraksts	12
2.2. Funkcionālās prasības.....	14
2.3. Nefunkcionālās prasības	22
3. UZDEVUMA RISINĀŠANAS LĪDZEKĻU IZVĒLES PAMATOJUMS	25
4. PROGRAMMATŪRAS PRODUKTA MODELĒŠANA UN PROJEKTĒŠANA	26
4.1. Sistēmas struktūras modelis.....	26
4.1.1. Sistēmas arhitektūra	26
4.1.2. Sistēmas ER modelis.....	27
4.2. Funkcionālais sistēmas modelis	31
4.2.1. Datu plūsmu modelis	31
5. DATU STRUKTŪRU APRAKSTS	35
6. LIETOTĀJA CEĻVEDIS.....	43
6.1. Sistēmas prasības aparatūrai un programmatūrai	43
6.2. Sistēmas instalācija un palaišana	43
6.3. Programmas apraksts.....	45
7. PROGRAMMATŪRAS LIETOJAMĪBAS TESTĒŠANA	87
NOBEIGUMS	89
INFORMĀCIJAS AVOTI.....	90
PIELIKUMI.....	91
Programmas pirmteksts.....	91

IEVADS

Mūsdienu sabiedrībā arvien vairāk pieaug interese par veselīgu dzīvesveidu un personīgo labsajūtu. Cilvēki sāk pievērsties veselīgākiem paradumiem, piemēram, regulārām fiziskajām aktivitātēm un sabalansētam uzturam. Neskatoties uz šo pozitīvo tendenci, daudzi sastopas ar nozīmīgiem izaicinājumiem – informācijas nepieejamība, vairāku rīku nepieciešamība un risinājumu augstās izmaksas. Lietotājiem bieži nākas izmantot atsevišķas aplikācijas treniņu plānošanai, citas – uztura uzskaitē, un pat mēdz būt vajadzība arī pēc trešā rīka progresa analīzei, kas var būt neērti, laiktietilpīgi un finansiāls slogs.

Pašlaik fitnesa un labsajūtas vidē ir pieejami vairāki risinājumi, tomēr katrs no tiem ir ar saviem ierobežojumiem. Viena no populārākajām aplikācijām ir MyFitnessPal, kas sākotnēji tika izstrādāta kā uztura uzskaitē paredzēta platforma. Tā piedāvā plašu pārtikas datu bāzi un makroelementu izsekošanu, taču tās galvenā uzmanība ir vērsta uz uzturu. Treniņu plānošanas un izpildes izsekošana ir tikai sekundāra funkcionalitāte, un sistēmas pilna versija ir pieejama tikai par maksu. Cits labi zināms risinājums ir Strava, kas ir vērsts uz sportisku aktivitāšu izsekošanu – galvenokārt skrējienus un riteņbraukšanu. Nav daudzveidīgu treniņu formu, piemēram, spēka treniņu, elastības treniņu vai sporta spēļu izsekošana, un tā nenodrošina uztura pārvaldības iespējas. Strong, Hevy un līdzīgās spēka treniņu aplikācijas ir labas to konkrēto darbības jomām, taču tās ir orientētas uz viena veida treniņu reģistrāciju un nepiedāvā uztura izsekošanu vienā sistēmā. Šo risinājumu kopīgais trūkums ir tas, ka visaptverošs risinājums, kas apvieno gan daudzveidīgu treniņu plānošanu, gan detalizētu uztura izsekošanu vienotā vidē, bieži vien ir pieejams tikai maksas versijā. Turklāt lielāka daļa šo sistēmu ir komercializētas lielām korporācijām un nav pielāgotas atsevišķu lietotāju individuālajām vajadzībām.

Lai novērstu šīs problēmas, tika izstrādāts JustFitness – mobilā lietotne, kas satur integrētu risinājumu treniņu un uztura vadībai. Atšķirībā no esošajiem risinājumiem, JustFitness ir: pieejams bez maksas – galvenās funkcionalitātes ir bezmaksas, ar iespējamām abonementa variantiem nākotnē; vienota platforma – gan treniņu plānošana, gan uztura izsekošana vienā vietā; vienkārša un intuitīva – piemērota gan iesācējiem, gan pieredzējušiem fitnesa entuziastiem; muskuļu grupu orientēta – treniņu plānošana pēc muskuļu grupu fokusa; detalizēta progresa izsekošana – treniņu sesiju un uztura dati tiek pilnībā dokumentēti

Sistēma ir izstrādāta, ņemot vērā gan tehniskos, gan lietojamības aspektus, kas ļauj nodrošināt pielāgojamību jebkuram fitnesa entuziastam – no iesācējiem, kuri tikai sāk viņu fitnesa ceļojumu, līdz pieredzējušiem sportistiem, kuri vēlas detalizētu un precīzu izsekošanu. Sistēma piedāvā iespēju izveidot personalizētas treniņu programmas lietojot plašu klāstu ar

vingrinājumiem, reģistrēt izpildītos vingrinājumus ar svariem un atkātojumiem, sekot uzturam visās ēdienreizēs, analizēt progresu laika gaitā un pieņemt informētus lēmumus par savu veselību un fitnesa mērķu sasniegšanu.

1. UZDEVUMA NOSTĀDNE

Kvalifikācijas darba uzdevums ir apvienot gan treniņu plānu, gan uztura pārvaldību vienotā mobilā platformā. Esošās sistēmas padara lietotāju pieredzi apgrūtinātu un neērtu, galvenokārt liekot fokusu tikai uz vienu no iepriekš minētajām funkcijām. Apvienojot abas, sistēma uzlabotu lietotājam pieredzi, novēršot nepieciešamību pielietot divas atsevišķas sistēmas. Sistēmā nepieciešams realizēt iespēju glabāt un pārvaldīt datus par lietotājiem, viņu fiziskajiem parametriem, muskuļu grupām, vingrinājumiem, treniņu plāniem, treniņu sesijām un ēdienu informāciju. Katram lietotāju tipam ir jābūt atbilstošai piekļuvei dažādām funkcijām atkarībā no viņu lomas sistēmā. Sistēmā jānodrošina drošības mehānismi autentifikācijas un autorizācijas nolūkā.

Ir plānotas vairākas funkcijas (skat. 1. att.):



1.1 att. Lietojumgadījuma diagramma

- Viesim (neregistrētam lietotājam) būs iespēja:
 - Apskatīt visu sistēmā pielietoto muskuļu grupu informāciju un to aprakstus;

- Veikt reģistrāciju un autentifikāciju.
- Reģistrētam lietotājam būs iespēja:
 - Rediģēt sava profila datus (vārds, dzimums, augums, svars, mērķa svars);
 - Izveidot, rediģēt un dzēst savus vingrinājumus;
 - Izveidot, rediģēt un dzēst savus treniņu plānus;
 - Sākt un pabeigt treniņu sesijas ar iespēju reģistrēt izpildītos vingrinājumus un svarus tiem;
 - Izveidot, rediģēt un dzēst savus ēdienus;
 - Ievadīt un rediģēt savas maltītes (brokastis, pusdienas, vakariņas, uzkodas);
 - Apskatīt publiski pieejamos vingrinājumus, treniņu plānus un ēdienus;
 - Apskatīt savu treniņu un maltīšu vēsturi un progresu datus.
- Administratoram būs iespēja:
 - Rediģēt un dzēst sistēmas lietotāju datus;
 - Izveidot, rediģēt un dzēst visus sistēmā esošos vingrinājumus;
 - Izveidot, rediģēt un dzēst visus sistēmā esošos ēdienus;
 - Pārvaldīt muskuļu grupu informāciju;
 - Apskatīt lietotāju aktivitātes un izsekošanas datus.

2. PRASĪBU SPECIFIKĀCIJA

2.1. Ieejas un izejas informācijas apraksts

2.1.1. Ieejas informācijas apraksts

Sistēmā tiks nodrošināta šāda ieejas informācija, ko ievadīs lietotājs no tastatūras apstrādes:

1. Informācija par lietotājiem sastāvēs no šādiem datiem:

- Vārds – lietotāja vārds - burtu teksts ar izmēru līdz 50 rakstzīmēm (piem. Jānis)
- E-pasts – lietotāja elektroniskā pasta adrese - burtu teksts ar izmēru līdz 100 rakstzīmēm (piem. janis@gmail.com)
- Parole – parole, ar kuru lietotājs autorizējās sistēmā - burtu teksts ar izmēru līdz 50 rakstzīmēm (piem. Janis!2024)
- Loma – loma, ar kuru noteiks vai lietotājam ir piekļuve administratīvām piekļuvēm – burtu teksts ar izmēru līdz 50 rakstzīmēm (piem. admin)

2. Informācija par lietotāja iestatījumiem sastāvēs no šādiem datiem:

- Valoda – lietotāja vēlamā valoda - burtu teksts (piem. "lv", "en")
- Svara mērvienība – burtu teksts (piem. "kg", "lbs")
- Garuma mērvienība – burtu teksts (piem. "cm", "in")
- Distances mērvienība – burtu teksts (piem. "km", "miles")
- Mērķa svars kg - cipars ar decimālo daļu (piem. 72.0)
- Mērķa kalorijas – dienas kaloriju norma - cipars (piem. 2500)
- Mērķa proteīni – dienas proteīna norma gramos - cipars ar decimālo daļu (piem. 150.5)
- Mērķa tauki – dienas tauku norma gramos - cipars ar decimālo daļu (piem. 70.0)
- Mērķa ogļhidrāti – dienas ogļhidrātu norma gramos - cipars ar decimālo daļu (piem. 250.0)

- Pašreizējais garums – lietotāja pašreizējais garums cm - cipars ar decimālo daļu (piem. 180.5)
- Dzimums – lietotāja dzimums - burtu teksts (piem. "Vīrietis", "Sieviete")
- Dzimšanas datums – lietotāja dzimšanas datums – datums (piem. 1990-05-15)

3. Informācija par svara vēsturi sastāvēs no šādiem datiem:

- Svars – reģistrētais svars kg - cipars ar decimālo daļu (piem. 75.2)
- Datums – diena, kurā svars tika reģistrēts – datums un laiks (piem. 2026-04-27 08:30:00)

4. Informācija par muskuļu grupām sastāvēs no šādiem datiem:

- Nosaukums – muskuļu grupas nosaukums - burtu teksts ar izmēru līdz 30 rakstzīmēm (piem. Krūtis)
- Apraksts – muskuļu grupas detalizēts apraksts - burtu teksts ar izmēru līdz 500 rakstzīmēm

5. Informācija par vingrinājumiem sastāvēs no šādiem datiem:

- Nosaukums – vingrinājuma nosaukums - burtu teksts ar izmēru līdz 50 rakstzīmēm (piem. Krūšu spiešana ar stieni)
- Muskuļu grupa – saistītā muskuļu grupa - burtu teksts (piem. Krūtis)
- Apraksts – vingrinājuma detalizēts apraksts - burtu teksts ar izmēru līdz 500 rakstzīmēm
- Publiski pieejams – vai vingrinājums ir pieejams citiem lietotājiem - Boolean (Jā/Nē)

6. Informācija par treniņu plāniem sastāvēs no šādiem datiem:

- Nosaukums – treniņu plāna nosaukums - burtu teksts ar izmēru līdz 50 rakstzīmēm (piem. Spēka treniņš)
- Apraksts – treniņu plāna detalizēts apraksts - burtu teksts ar izmēru līdz 500 rakstzīmēm
- Vingrinājumi – saraksts ar vingrinājumiem plānā - saites uz vingrinājumiem

- Publiski pieejams – vai plāns ir pieejams citiem lietotājiem - Boolean (Jā/Nē)

7. Informācija par treniņu sesijām sastāvēs no šādiem datiem:

- Datums – treniņu sesijas datums - datums (piem. 2026-04-26)
- Laiks – treniņu sesijas sākuma laiks - laiks (piem. 18:00)
- Ilgums – treniņu sesijas ilgums minūtēs - cipars (piem. 60)
- Vingrinājumi – vingrinājumu saraksts sesijā
 - Vingrinājuma nosaukums
 - Muskuļu grupas
 - Ieteiktie piegājieni dati (informācija no iepriekšējās sesijas)
 - Faktiskā piegājiena dati (kas tiek ievadīti šajā sesijā)

8. Informācija par treniņu piegājieniem (no iepriekšējās sesijas) sastāvēs no šādiem datiem:

- Piegājiena numurs – cipars (piem. 1, 2, 3)
- Ieteiktie atkārtojumi – cipars (piem. 10)
- Ieteiktais svars kg – cipars ar decimālo daļu (piem. 80.0 kg)

9. Informācija par sesijas piegājieniem (faktiskā izpilde) sastāvēs no šādiem datiem:

- Piegājiena numurs – cipars (piem. 1, 2, 3)
- Faktiskais atkārtojumu skaits – cipars (piem. 10)
- Faktiskais lietotais svars kg – cipars ar decimālo daļu (piem. 85.0 kg)
- Faktiskais atpūtas ilgums – cipars sekundēs (piem. 60)

10. Informācija par ūdens ierakstiem sastāvēs no šādiem datiem:

- Daudzums ML – cipars (piem. 50)
- Ieraksta datums – datums (piem. 2026-04-26)

11. Informācija par ēdieniem sastāvēs no šādiem datiem:

- Nosaukums – ēdiena nosaukums - burtu teksts ar izmēru līdz 50 rakstzīmēm (piem. Vista ar rīsiem)
- Proteīns uz 100g – proteīna saturs gramos uz 100g produkta - cipars ar decimālo daļu (piem. 25.5 g)
- Tauki uz 100g – tauku saturs gramos uz 100g produkta - cipars ar decimālo daļu (piem. 8.2 g)
- Oglhidrāti uz 100g – ogļhidrātu saturs gramos uz 100g produkta - cipars ar decimālo daļu (piem. 32.0 g)
- Kalorijas uz 100g – kaloriju saturs uz 100g produkta - cipars (piem. 280 kcal)
- Publiski pieejams – vai ēdiens ir pieejams citiem lietotājiem - Boolean (Jā/Nē)7.

12. Informācija par maltītēm sastāvēs no šādiem datiem:

- Tips – maltītes tips - burtu teksts (Brokastis, Pusdienas, Vakariņas, uzkodas)
- Datums – maltītes datums - datums (piem. 2026-04-26)
- Ēdieni – ēdienu saraksts maltītē - saites uz ēdieniem
- Kopējās kalorijas – maltītes kopējais kaloriju skaits - cipars
- Kopējie proteīni – maltītes kopējais proteīna skaits - cipars ar decimālo daļu
- Kopējie tauki – maltītes kopējais tauku skaits - cipars ar decimālo daļu
- Kopējie ogļhidrāti – maltītes kopējais ogļhidrātu skaits - cipars ar decimālo daļu

2.1.2. Izejas informācijas apraksts

1. Lietotāja profila pārskats – parāda lietotāja personīgo informāciju (vārds, e-pasts, dzimums, augums, pašreizējais svars, mērķi) ar iespēju rediģēt datus. Pārskatā redzams arī svara progresu grafiks.
2. Lietotāja iestatījumu pārskats – forma ar lietotāja pašreizējiem iestatījumiem (valoda, svara mērvienība, garuma mērvienība, distances mērvienība). Lietotājs var mainīt jebkuru no šiem parametriem un saglabāt izmaiņas.
3. Treniņu plānu pārskats – tabula ar visiem lietotāja treniņu plāniem, kur parādīts plāna nosaukums, apraksts, vingrinājumu skaits un iespēja sākt treniņu sesiju.

4. Aktīvās treniņu sesijas pārskats – detalizēts pārskats, kurā redzams pašreizējo vingrinājumu saraksts ar iespēju reģistrēt vingrinājuma datus (vingrinājuma nosaukums, muskuļu grupas, piegājienu saraksts, un katrā piegāzienā atkārtojumi un svars). Pārskatā pieejama iespēja pievienot jaunus piegāzienus vai pabeigt sesiju.
5. Treniņu vēstures pārskats – saraksts ar visām veiktajām treniņu sesijām, kur parāda datumu, laiku, ilgumu un vingrinājumu skaitu. Iespēja klikšķināt uz sesijas, lai redzētu detalizētu informāciju par veiktajiem vingrinājumiem un piegājieniem.
6. Progresu grafiki – vizuālie attēlojumi ar vingrinājuma vai lietotāja svara izmaiņām laika gaitā, ļaujot izsekot svara izmaiņām.
7. Vingrinājumu kataloga pārskats – saraksts ar publiskajiem vingrinājumiem, filtrējami pēc muskuļu grupu, ar iespēju apskatīt vingrinājuma aprakstu un muskuļu grupu.
8. Treniņu plānu kataloga pārskats – saraksts ar publiskajiem treniņu plāniem, filtrējami pēc kategorijas, ar iespēju apskatīt plāna aprakstu un vingrinājumu sarakstu.
9. Maltītes pārskats – tabula ar visām dienas maltītēm (brokastis, pusdienas, vakariņas, uzkodas), kur parādīts katrā maltītē iekļauto ēdienu saraksts ar to daudzumiem (gramos) un kopējie makroelementi (kalorijas, proteīni, tauki, ogļhidrāti).
10. Ēdienu izvēles pārskats – saraksts ar publiskajiem ēdieniem, filtrējami pēc kategorijas, ar iespēju apskatīt ēdiena makroelementu informāciju uz 100g (proteīni, tauki, ogļhidrāti, kalorijas).
11. Ēdiena pievienošanas pārskats – forma, kur lietotājs var pievienot ēdienu maltītei, norādot ēdiena nosaukumu un daudzumu gramos. Sistēma automātiski aprēķina makroelementus pamatojoties uz norādīto daudzumu.
12. Dienas uztura pārskats – kopsavilkums par visu dienas uzņemto kaloriju, proteīna, tauku un ogļhidrātu skaitu ar makroelementu sadalījuma joslu.
13. Nedēļas uztura statistikas pārskats – grafiks, kas parāda vidējās kaloriju, proteīna, tauku un ogļhidrātu vērtības katrai nedēļas dienai.
14. Muskuļu grupu informācijas pārskats – detalizēts apraksts par katru muskuļu grupu ar anatomisku informāciju un tai piesaistīto vingrinājumu sarakstu.

2.2.Funkcionālās prasības

1. Vingrinājuma izveidošana

2.1. tabula

Vingrinājuma izveidošanas funkcionālais pārskats

Identifikators	PS 2.2.1
Ievads	
Ļauj reģistrētam lietotājam izveidot jaunu vingrinājumu ko pievienot treniņu plānam	
Ievaddati	
1. Vingrinājuma nosaukums – obligāts lauks, burtu teksts līdz 50 rakstzīmēm; 2. Apraksts – neobligāts lauks, burtu teksts līdz 500 rakstzīmēm; 3. Muskuļu grupa – obligāts lauks, saite uz muskuļu grupu; 4. Publiskums – obligāts lauks, boolean (patiess/aplams).	
Apstrāde	
1. Sistēma validē ieejas datus (formāts, obligātie lauki); 2. Sistēma pārbauda, vai nosaukums jau neeksistē; 3. Ja validācija sekmīga un nosaukums unikāls, sistēma izveido jaunu ierakstu vingrinājumu tabulā; 4. Sistēma izveido saiti vingrinājuma un muskuļu grupu starptabulā.	
Izvaddati	
1. Ja sekmīgi – veiksmes ziņojums ar izveidotā vingrinājuma ID 2. Ja nosaukums pastāv – informatīvs paziņojums	
Kļūdas paziņojums	
1. Vingrinājums ar tādu nosaukumu jau eksistē jūsu lietotājam	
Izsaucamās prasības	
Var veikt reģistrēti lietotāji	

2. Treniņu plāna izveidošana

2.2. tabula

Treniņu plāna izveidošanas funkcionālais pārskats

Identifikators	PS 2.2.2
Ievads	
Ļauj lietotājam izveidot jaunu treniņu plānu, ko vēlāk varēs uzsākt	
Ievaddati	
1. Plāna nosaukums – obligāts lauks, burtu teksts līdz 50 rakstzīmēm 2. Apraksts – neobligāts lauks, burtu teksts līdz 500 rakstzīmēm 3. Vingrinājumi – obligāts lauks, saraksts ar vingrinājumu ID 4. Publiskums – obligāts lauks, boolean (patiess/aplams)	
Apstrāde	
1. Sistēma pārbauda obligātos laukus; 2. Validē nosaukuma un apraksta formātu; 3. Pārbauda, vai plāns ar tādu nosaukumu jau neeksistē; 4. Validē, ka atlasītie vingrinājumi eksistē; 5. Saglabā treniņu plānu un tā vingrinājumus.	
Izvaddati	
1. Ja sekmīgi – treniņu plāns veiksmīgi pievienots. Lietotājs tiek aizvadīts uz treniņu uzskaites lapu, kur redzams jaunizveidotais plāns 2. Ja nosaukums pastāv – informatīvs paziņojums	
Kļūdas paziņojums	
1. Treniņu plāns ar tādu nosaukumu jau eksistē jūsu lietotājam 2. Jāatlasa vismaz 1 vingrinājums	
Izsaucamās prasības	
Var veikt reģistrēti lietotāji	

3. Ēdiena izveidošana

2.3. tabula

Ēdiena izveidošanas funkcionālais pārskats

Identifikators	PS 2.2.3
Ievads	
Ļauj reģistrētam lietotājam vai administratoram izveidot jaunu ēdienu ar makroelementu informāciju uz 100g	
Ievaddati	
1. Ēdiena nosaukums – obligāts lauks, burtu teksts līdz 50 rakstzīmēm 2. Kalorijas uz 100g – obligāts lauks, cipars (piem. 280) 3. Proteīni uz 100g – obligāts lauks, cipars ar decimālo daļu (piem. 25.5) 4. Tauki uz 100g – obligāts lauks, cipars ar decimālo daļu (piem. 8.2) 5. Oglhidrāti uz 100g – obligāts lauks, cipars ar decimālo daļu (piem. 32.0) 6. Publiskums – obligāts lauks, boolean	
Apstrāde	
1. Sistēma validē ieejas datus (formāts, obligātie lauki, pozitīvi skaitļi) 2. Sistēma pārbauda, vai nosaukums jau lietotājam neeksistē; 3. Sistēma izveido jaunu ierakstu ēdienu tabulā ar lietotāja ID (ja reģistrēts) vai administrācijas ID (ja administrators) 4. Sistēma saglabā makroelementu datus uz 100g	
Izvaddati	
1. Ja sekmīgi – veiksmes ziņojums ar izveidotā ēdiena ID 2. Ja nosaukums pastāv – informatīvs paziņojums	
Kļūdas paziņojums	
1. Ēdiens ar tādu nosaukumu jau eksistē jūsu lietotājam	
Izsaucamās prasības	
Var veikt reģistrēti lietotāji	

4. Ēdiena pievienošana maltītei

2.4. tabula

Ēdiena pievienošana maltītei funkcionālais pārskats

Identifikators	PS 2.2.4
Ievads	
Ļauj reģistrētam lietotājam pievienot ēdienu konkrētai maltītei, norādot daudzumu gramos, un sistēma automātiski aprēķina makroelementus	
Ievaddati	
1. Maltītes ID – obligāts lauks, saite uz maltīti 2. Ēdiena ID – obligāts lauks, saite uz ēdienu 3. Daudzums izvēlētajā mērvienībā – obligāts lauks, cipars ar decimālo daļu (piem. 150.5) 4. Mērvienība – obligāts lauks, ENUM: g, kg, ml, l	
Apstrāde	
1. Sistēma validē ieejas datus (formāts, obligātie lauki) 2. Sistēma atlasa ēdiena makroelementu datus no ēdienu tabulas 3. Sistēma aprēķina makroelementus pamatojoties uz daudzumu: 4. Kalorijas = kalorijas_uz_100g * (daudzums / 100) 5. Proteīni = proteīni_uz_100g * (daudzums / 100) 6. Tauki = tauki_uz_100g * (daudzums / 100) 7. Ogļhidrāti = ogļhidrāti_uz_100g * (daudzums / 100) 8. Sistēma izveido ierakstu maltītes ēdienu starptabulā ar aprēķinātajiem makroelementiem	
Izvaddati	
1. Ja sekmīgi – veiksmes ziņojums + aprēķinātie makroelementi 2. Ja nepareizi dati – informatīvs paziņojums	
Kļūdas paziņojums	
1. Nepareiza formāta dati ievadīti	
Izsaucamās prasības	
Var veikt reģistrēti lietotāji	

5. Jauno lietotāju kontu pievienošana un reģistrācija:

5.1. Jānodrošina iespēja reģistrēt jaunu lietotāja kontu bez administratora starpniecības.

- 5.2. Jāļauj izveidot jaunu kontu, izmantojot reģistrācijas formu ar vārdu, e-pasta adresi, paroli, dzimumu, augumu, svaru un mērķa svaru.
- 5.3. Ja obligātie lauki nav aizpildīti, neļaut izveidot kontu un izvadīt paziņojumu.
- 5.4. Ja formāts neatbilst ieejas informācijas apraksta 1. punktā minētajam, neļaut pievienot kontu un izvadīt paziņojumu.
- 5.5. Ja e-pasta adrese jau pastāv sistēmā, neļaut pievienot kontu un izvadīt paziņojumu.
- 5.6. Pēc veiksmīgas reģistrācijas lietotāju automātiski novirzīt uz autorizētu lapu.
6. Lietotāja autentifikācija un autorizācija:
 - 6.1. Lietotājiem jānodrošina autentifikācija sistēmā, izmantojot formu ar e-pasta adresi un paroli.
 - 6.2. Ja e-pasta adrese sakrīt ar datu bāzē esošā lietotāja e-pasta adresi un parole ir pareiza, novirzīt lietotāju uz viņa konta galveno pārskatu (mājas lapu).
 - 6.3. Ja e-pasta adrese vai parole nesakrīt ar sistēmā esošo, izvadīt paziņojumu un neļaut piekļūt kontam.
 - 6.4. Sistēmai jānodrošina JWT vai sesiju pamatā esoša autorizācija, lai pēc autentifikācijas lietotājs paliktu pieslēgts.
 - 6.5. Lietotājam jābūt iespējai atslēgties no sistēmas jebkurā brīdī.
7. Lietotāja profila rediģēšana:
 - 7.1. Reģistrētam lietotājam jābūt iespējai rediģēt savu profila datus (vārds, dzimums, augums, pašreizējais svars, mērķa svars).
 - 7.2. Administratoram jābūt iespējai rediģēt jebkura lietotāja datus.
 - 7.3. Profila rediģēšanai ir jāpiedāvā forma ar esošajiem datiem jau aizpildītiem.
 - 7.4. Ja formāts neatbilst ieejas informācijas apraksta 1. punktā minētajam, neļaut saglabāt izmaiņas un izvadīt paziņojumu.
 - 7.5. Pirms jaunas informācijas saglabāšanas ir jāizvada brīdinājuma ziņojums ar iespēju saglabāt vai atcelt veiktās izmaiņas.
 - 7.6. Pēc veiksmīgas rediģēšanas jāparāda paziņojums par sekmīgu atjauninājumu.
8. Lietotāja kontu dzēšana:

- 8.1. Administratoram jābūt iespējai dzēst lietotāja kontu.
- 8.2. Lietotājam jābūt iespējai dzēst savu kontu.
- 8.3. Pirms kontu dzēšanas jāparāda brīdinājuma paziņojums ar iespēju apstiprināt vai atcelt darbību.
- 8.4. Pēc dzēšanas jāparāda paziņojums par sekmīgu dzēšanu.
9. Vingrinājumu pārvaldīšana:
 - 9.1. Reģistrētam lietotājam jābūt iespējai izveidot, rediģēt un dzēst savus vingrinājumus.
 - 9.2. Administratoram jābūt iespējai izveidot, rediģēt un dzēst sistēmā esošos vingrinājumus.
 - 9.3. Jāļauj izveidot jaunu vingrinājuma ierakstu, izmantojot formu ar nosaukumu, muskuļu grupu, aprakstu un publiskuma statusu.
 - 9.4. Ja obligātie lauki nav aizpildīti, neļaut izveidot ierakstu un izvadīt paziņojumu.
 - 9.5. Ja formāts neatbilst ieejas informācijas apraksta 3. punktā minētajam, neļaut pievienot vingrinājumu un izvadīt paziņojumu.
 - 9.6. Vingrinājuma rediģēšanai ir jāpiedāvā forma ar esošajiem datiem jau aizpildītiem.
 - 9.7. Pirms vingrinājuma dzēšanas jāparāda brīdinājuma paziņojums.
10. Treniņu plānu pārvaldīšana:
 - 10.1. Reģistrētam lietotājam jābūt iespējai izveidot, rediģēt un dzēst savus treniņu plānus.
 - 10.2. Administratoram jābūt iespējai izveidot, rediģēt un dzēst sistēmā esošos treniņu plānus.
 - 10.3. Jāļauj izveidot jaunu plāna ierakstu, izmantojot formu ar nosaukumu, aprakstu, vingrinājumu sarakstu un publiskuma statusu.
 - 10.4. Ja obligātie lauki nav aizpildīti, neļaut izveidot ierakstu un izvadīt paziņojumu.
 - 10.5. Jāļauj pievienot un noņemt vingrinājumus no treniņu plāna rediģēšanas laikā.
 - 10.6. Plāna rediģēšanai ir jāpiedāvā forma ar esošajiem datiem jau aizpildītiem.
 - 10.7. Pirms plāna dzēšanas jāparāda brīdinājuma paziņojums.

11. Treniņu sesiju pārvaldīšana:

- 11.1. Reģistrētam lietotājam jābūt iespējai sākt treniņu sesiju, rediģēt sesijas datus un dzēst nepabeigtas sesijas.
- 11.2. Sākot treniņu sesiju, jāparāda treniņu plāna vingrinājumi ar ieteiktajiem piegājieniem no iepriekšējās sesijas (ja tāda pastāv) kā informāciju.
- 11.3. Lietotājam jāievada faktiskā piegājiena dati (faktiskais atkārtojumu skaits, faktiskais svars, faktiskais atpūtas ilgums) katram piegājenam.
- 11.4. Jāļauj pievienot jaunus piegājenus esošam vingrinājumam sesijas laikā.
- 11.5. Pēc sesijas pabeigšanas jāparāda paziņojums ar sesijas kopsavilkumu.
- 11.6. Jānodrošina iespēja apskatīt un rediģēt nesen izpildītas sesijas (mainīt faktiskos piegājienus datus).

12. Ēdienu pārvaldīšana:

- 12.1. Reģistrētam lietotājam jābūt iespējai izveidot, rediģēt un dzēst savus ēdienus.
- 12.2. Administratoram jābūt iespējai izveidot, rediģēt un dzēst sistēmā esošos ēdienus.
- 12.3. Jāļauj izveidot jaunu ēdiena ierakstu, izmantojot formu ar nosaukumu, makroelementu saturu uz 100g (proteīni, tauki, ogļhidrāti, kalorijas) un publiskuma statusu.
- 12.4. Ja obligātie lauki nav aizpildīti, neļaut izveidot ierakstu un izvadīt paziņojumu.
- 12.5. Ja formāts neatbilst ieejas informācijas apraksta 6. punktā minētajam, neļaut pievienot ēdienu un izvadīt paziņojumu.
- 12.6. Ēdiena rediģēšanai ir jāpiedāvā forma ar esošajiem datiem jau aizpildītiem.
- 12.7. Pirms ēdiena dzēšanas jāparāda brīdinājuma paziņojums.

13. Maltīšu un uztura izsekošana:

- 13.1. Reģistrētam lietotājam jābūt iespējai ievadīt un rediģēt savus ēdienus konkrētām maltītēm (brokastis, pusdienas, vakariņas, uzkodas).
- 13.2. Jāļauj pievienot ēdienus maltītei, norādot ēdiena nosaukumu un daudzumu gramos.

- 13.3. Sistēmai automātiski jāaprēķina kopējie makroelementi (kalorijas, proteīni, tauki, ogļhidrāti) katrai maltītei, pamatojoties uz norādīto daudzumu.
- 13.4. Jānodrošina iespēja apskatīt kopējo dienas makroelementu saturu.
- 13.5. Jāļauj dzēst ēdienus no maltītes.
- 13.6. Jānodrošina iespēja apskatīt maltīšu vēsturi iepriekšējām dienām.
- 14. Muskuļu grupu informācija:
 - 14.1. Visiem lietotājiem (viesi, reģistrēti lietotāji, administratori) jābūt iespējai apskatīt visu sistēmā pieejamo muskuļu grupu informāciju un aprakstus.
 - 14.2. Administratoram jābūt iespējai rediģēt muskuļu grupu informāciju.
 - 14.3. Muskuļu grupu pārskatā jārāda ar grupu saistītie vingrinājumi.
- 15. Vingrinājumu katalogs:
 - 15.1. Visiem lietotājiem jābūt iespējai apskatīt publiski pieejamos vingrinājumus.
 - 15.2. Jānodrošina filtrēšana pēc muskuļu grupas.
 - 15.3. Jānodrošina iespēja meklēt vingrinājumus pēc nosaukuma.
 - 15.4. Informācijai jābūt pieejamai tabulas vai saraksta formātā.
- 16. Treniņu plānu katalogs:
 - 16.1. Visiem lietotājiem jābūt iespējai apskatīt publiski pieejamos treniņu plānus.
 - 16.2. Jānodrošina iespēja apskatīt plāna detaļas, tajā skaitā iekļauto vingrinājumu sarakstu.
 - 16.3. Reģistrētam lietotājam jābūt iespējai kopēt publisku plānu uz savu kontu.
 - 16.4. Informācijai jābūt pieejamai tabulas vai saraksta formātā.
- 17. Ēdienu katalogs:
 - 17.1. Visiem lietotājiem jābūt iespējai apskatīt publiski pieejamos ēdienus ar makroelementu informāciju uz 100g.
 - 17.2. Jānodrošina filtrēšana pēc kategorijas.
 - 17.3. Jānodrošina iespēja meklēt ēdienus pēc nosaukuma.

- 17.4. Informācijai jābūt pieejamai tabulas vai saraksta formātā.
18. Progresu izsekošana:
- 18.1. Reģistrētam lietotājam jābūt iespējai apskatīt vingrinājuma svāri izmaiņas laika gaitā ar vizuāliem grafikiem.
- 18.2. Jānodrošina iespēja apskatīt treniņu vēsturi ar detaļām par katru sesiju.
- 18.3. Jānodrošina iespēja apskatīt nedēļas uztura statistiku ar vidējiem makroelementu skaitļiem.
19. Datu drošība un validācija:
- 19.1. Visu lietotāja ievadīto datu validācijai ir jābūt gan klienta, gan servera pusē.
- 19.2. Parolēm jābūt šifrētām datu bāzē (minimum bcrypt vai līdzīgs algoritms).
- 19.3. Jūtīgiem datiem jābūt pārsūtītiem ar šifrētu savienojumu (HTTPS).
- 19.4. Lietotāju sesijām jābūt ar beigu laiku.

2.3. Nefunkcionālās prasības

1. Sistēmas saskarnes valodai jābūt pieejamai latviešu valodā;
2. Sistēmai jānodrošina ērta lietojamība mobilajās ierīcēs ar dažādiem ekrāna izmēriem;
3. Lietotnes saskarnei jābūt pielāgotai skārienekrāna vadībai, lai pogas, ievades lauki un izvēlnes būtu viegli nospiežamas;
4. Sistēmas dizainam jābūt tumši zaļos un dzeltenos toņos, saglabājot vienotu vizuālo stilu visās lietotnes sadaļās;
5. Sistēmai jāsniedz lietotājam informatīvi paziņojumi par veiksmīgiem un neveiksmīgiem procesiem, piemēram, konta izveidi, datu saglabāšanu vai kļūdainu ievadi;
6. Sistēmai ir jādarbojas gan uz IOS, gan Android ierīcēm;

7. Lietotāja pieslēgšanās formai jāizskatās līdzīgi sekojošai bildei (skat. 2.1. att.)

Pieslēgšanās

Epasts

Vārds

Parole

Dzimšanas datums

Garums

Svars

Pieslēgties

2.1. att. Lietotāja pieslēgšanās forma

8. Treniņu plānu sarakstam ir jāizskatās līdzīgi sekojošai bildei (skat. 2.2. att.)

TRENIŅI

IZVEIDOT TRENIŅU

Tavi treniņu plāni

Treniņš 1	SĀKT
	REDIĢĒT
Treniņš 2	SĀKT
	REDIĢĒT

2.2. att. Treniņu plānu saraksts

9. Maltīšu sarakstam ir jāizskatās līdzīgi sekojošai bildei (skat. 2.3. att.)

UZTURS

<

Šodien

>

Brokastis

Olas100g

Pusdienas

Siermaize50g

Vakariņas

Kartupeļi100g

Kotlete150g

Uzkodas

Šokolādes batons25g

2.3. att. Maltīšu saraksts

3. UZDEVUMA RISINĀŠANAS LĪDZEKĻU IZVĒLES PAMATOJUMS

Sistēmas izstrādē tika izmantoti mūsdienīgi un plaši atbalstīti rīki un ietvari, kas nodrošina gan drošu, gan efektīvu lietojamību tīmekļa lietotnēm. Izvēlētie tehnoloģiskie risinājumi tika izvēlēti, ņemot vērā to saderību, paplašināšanas iespējas, aktīvo izstrādātāju kopienu un ilgtermiņa uzturēšanu.

Lietotāja saskarnes daļai tika izvēlēta **React Native** bibliotēka, kas ļauj veidot dinamisku un uz komponentiem balstītu lietotāja saskarni. React nodrošina ātru informācijas atspoguļošanu, efektīvu DOM manipulāciju (caur Virtual DOM), kā arī komponentu atkārtotu izmantošanu. Šie faktori ir būtiski, lai izstrādātu modernu, atsaucīgu un viegli uzturamu lietotāju saskarni.

Servera pusē tiks izmantots **ExpressJS** ietvars (**Node.js** vide), kas ir viens no populārākajiem un vieglākajiem **JavaScript** servera puses ietvariem. ExpressJS nodrošina elastīgu arhitektūru **RESTful API** izveidei, maršrutu (routing) pārvaldībai, vidusslāņa (middleware) integrācijai, kā arī autentifikācijas, autorizācijas un datu validācijas risinājumu izmantošanai. ExpressJS priekšrocība ir arī plašais pieejamo pakotņu klāsts (piemēram, **jsonwebtoken**, **bcrypt**, **express-validator**, **cors** u.c.), kas ļauj ātri un efektīvi paplašināt funkcionalitāti.

Datu bāzes pārvaldībai izmantota **MySQL**, pateicoties tās stabilitātei un plašajam atbalstam. Tā ir piemērota strukturētai informācijas uzglabāšanai un nodrošina augstu veikspēju arī ar lieliem datu apjomiem.

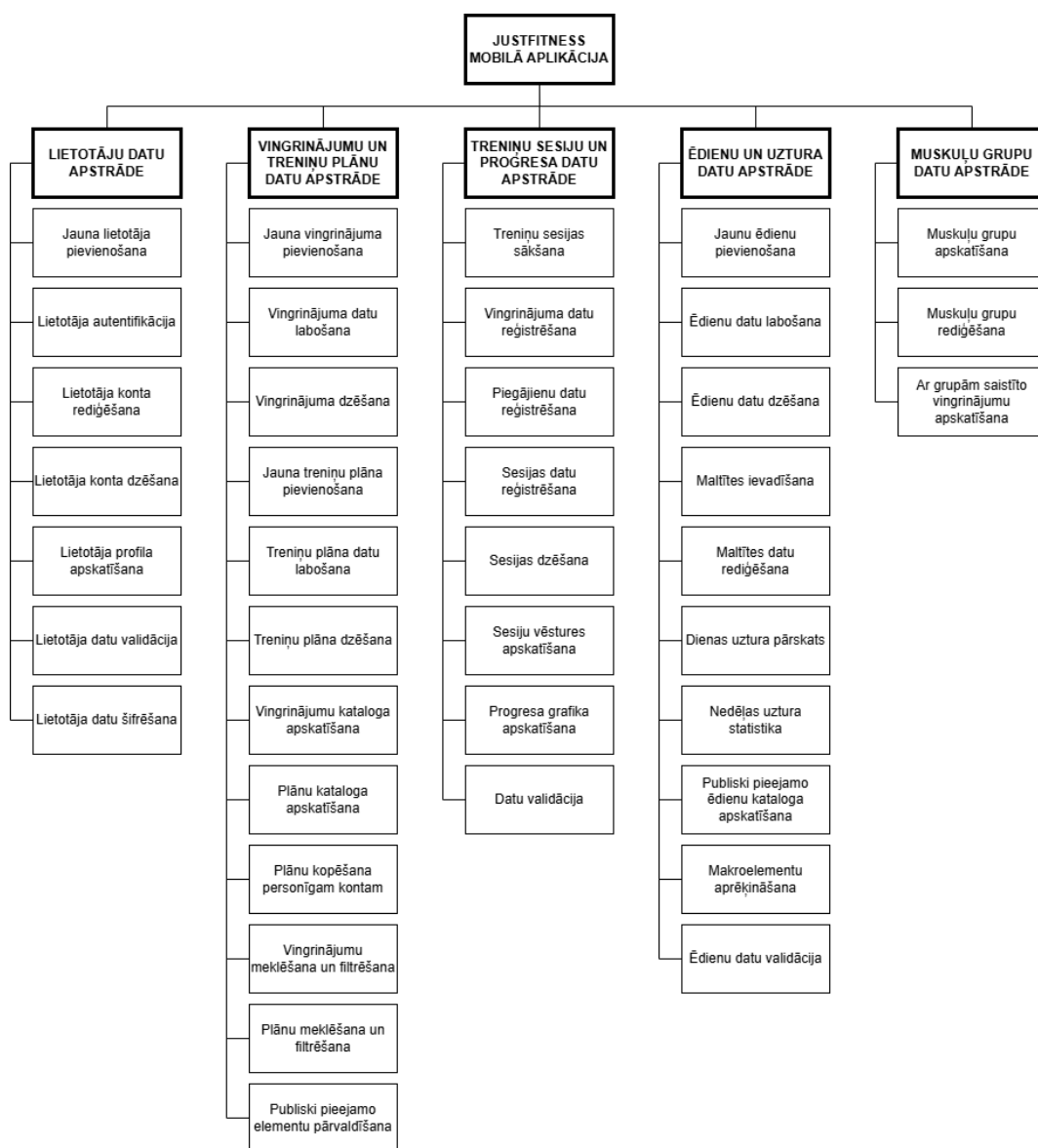
Diagrammu un modeļu veidošanai izmantota tiešsaistes platforma **draw.io**, kas nodrošina ērtu ER un klašu diagrammu veidošanu, eksportēšanu un koplietošanu.

4. PROGRAMMATŪRAS PRODUKTA MODELĒŠANA UN PROJEKTĒŠANA

4.1. Sistēmas struktūras modelis

4.1.1. Sistēmas arhitektūra

Sistēma tiks veidota no **piecām apakšsistēmām** (skat. 4.1. att.): lietotāju datu apstrādes apakšsistēmas, vingrinājumu un treniņu plānu datu apstrādes apakšsistēmas, treniņu sesiju un progresu izsekošanas apakšsistēmas, ēdienu un uztura izsekošanas apakšsistēmas un muskuļu grupu informācijas apakšsistēmas.



4.1. att. Funkcionālās dekompozīcijas diagramma

Lietotāju datu apstrādes apakšsistēmas moduļi nodrošinās administratoram un lietotājiem nepieciešamo funkcionalitāti lietotāju kontu pārvaldībai, autentifikācijai un profila rediģēšanai.

Vingrinājumu un treniņu plānu datu apstrādes apakšsistēma sastāvēs no moduļiem, kas ļaus reģistrētiem lietotājiem un administratoriem pārvaldīt vingrinājumus un treniņu plānus, filtrēt un meklēt publiskos vingrinājumus un plānus, kā arī kopēt publiskus plānus savai kontam.

Treniņu sesiju un progresu izsekošanas apakšsistēma nodrošinās funkcionalitāti treniņu sesiju sākšanai, rediģēšanai un dzēšanai, piegājienu un vingrinājumu datu reģistrēšanai, vēstures apskatīšanai un progresu grafiku veidošanai.

Ēdienu un uztura izsekošanas apakšsistēma ļaus lietotājiem pārvaldīt ēdienus, pievienot ēdienus maltītēm, izsekot dienas un nedēļas uztura statistiku, kā arī apskatīt publiski pieejamos ēdienus.

Muskuļu grupu informācijas apakšsistēma nodrošinās visiem lietotājiem iespēju apskatīt muskuļu grupu informāciju, anatomiskos aprakstus un ar grupām saistītos vingrinājumus.

4.1.2. Sistēmas ER modelis

Sistēmas ER-modelis sastāv no 11 entitijām (skat. 4.2. att.), kas nodrošina pamat informācijas uzglabāšanu un apstrādi. Tie ir:

"LIETOTĀJS", kas satur informāciju par sistēmas lietotājiem. Tās atribūtu kopums ietver lietotāja vārdu, e-pasta adresi, paroli un lomu.

"LIETOTĀJA_IESTATĪJUMI", kas satur personīgos iestatījumus katram lietotājam. Tās atribūtu kopums ietver valodu, svara mērvienību, garuma mērvienību, distances mērvienību, mērķa svaru, mērķa kaloriju saturu, mērķa proteīnu saturu, mērķa tauku saturu, mērķa ogļhidrātu saturu, pašreizējo svaru, pašreizējo garumu, dzimumu un dzimšanas datumu.

"SVARA_VĒSTURE ", kas satur vēsturi par lietotāja svara izmaiņām. Tās atribūtu kopums ietver svaru un datumu.

"MUSKUĻU_GRP", kas satur informāciju par muskuļu grupām. Tās atribūtu kopums ietver muskuļu grupas nosaukumu (piem. "Krūtis", "Mugura", "Kājas") un detalizētu aprakstu.

"VINGRINĀJUMS", kas satur informāciju par vingrinājumiem. Tās atribūtu kopums ietver vingrinājuma nosaukumu, aprakstu un publiskuma statusu.

"TRENIŅU_PLĀNS", kas satur informāciju par treniņu plāniem. Tās atribūtu kopums ietver plāna nosaukumu, aprakstu un publiskuma statusu.

"TRENIŅU_SESIJA", kas satur informāciju par veiktajām treniņu sesijām. Tās atribūtu kopums ietver sesijas sākuma laiku un sesijas pabeigšanas laiku.

"TRENIŅU_PIEGĀJIENS", kas satur informāciju par treniņa vingrinājuma piegājieniem. Tās atribūtu kopums ietver piegājiena indeksa numuru, atkārtojumu reizes, svaru un atpūtas laiku.

"SESIJU_PIEGĀJIENS", kas satur informāciju par treniņa sesijas vingrinājuma piegājieniem. Tās atribūtu kopums ietver piegājiena indeksa numuru, atkārtojumu reizes, svaru un atpūtas laiku

"ŪDENS", kas satur informāciju par ūdens ierakstiem. Tās atribūtu kopums ietver daudzumu ML un ievades datumu.

"ĒDIENS", kas satur informāciju par ēdieniem. Tās atribūtu kopums ietver ēdiena nosaukumu, makroelementu saturu uz 100g (kalorijas, proteīni, tauki, ogļhidrāti) un publiskuma statusu.

"MALTĪTE", kas satur informāciju par atsevišķām maltītēm. Tās atribūtu kopums ietver maltītes tipu (brokastis, pusdienas, vakariņas, uzkodas), datumu un lietotāja atsauci.

Starp LIETOTĀJU un TREIŅU_PLĀNU pastāv bināra saite ar kardinalitāti "viens pret daudziem", jo viens lietotājs var veidot daudz treniņu plānu, bet viens plāns pieder tikai vienam lietotājam.

Starp VINGRINĀJUMU un TREIŅU_PLĀNU pastāv bināra saite ar kardinalitāti "daudzi pret daudziem" caur TREIŅU_VINGRINĀJUMS, jo viens treniņu plāns satur daudz vingrinājumu, un viens vingrinājums var būt iekļauts daudzos plānos.

Starp TREIŅU_VINGRINĀJUMS un TREIŅU_PIEGĀJIENS pastāv bināra saite ar kardinalitāti "viens pret daudziem", jo viens treniņu vingrinājums var satur daudz ieteikto piegājienu, bet viens piegājiens pieder vienam treniņu vingrinājumam.

Starp LIETOTĀJU un TREIŅU_SESIJU pastāv bināra saite ar kardinalitāti "viens pret daudziem", jo viens lietotājs var veikt daudz treniņu sesiju, bet viena sesija pieder tikai vienam lietotājam.

Starp TREIŅU_PLĀNU un TREIŅU_SESIJU pastāv bināra saite ar kardinalitāti "viens pret daudziem", jo viena treniņu sesija ir balstīta uz vienu treniņu plānu, bet viens plāns var satur daudz sesiju.

Starp VINGRINĀJUMU un TREIŅU_SESIJU pastāv bināra saite ar kardinalitāti "daudzi pret daudziem" caur SESIJAS_VINGRINĀJUMS, jo viena sesija var satur daudz vingrinājumu, un viens vingrinājums var parādīties daudzās sesijās.

Starp SESIJAS_VINGRINĀJUMS un SESIJAS_PIEGĀJIENS pastāv bināra saite ar kardinalitāti "viens pret daudziem", jo viena vingrinājuma izpilde var satur daudz faktisko piegājienu, bet viens piegājiens pieder vienai vingrinājuma izpildei.

Starp LIETOTĀJU un ŪDENI pastāv bināra saite ar kardinalitāti "viens pret daudziem", jo viens lietotājs var veikt vairākus ūdens ierakstus, bet viens ūdens ieraksts pieder tikai vienam lietotājam.

Starp LIETOTĀJU un ĒDIENU pastāv bināra saite ar kardinalitāti "viens pret daudziem", jo viens lietotājs var veidot daudz personīgo ēdienu, bet viens ēdiens pieder tikai vienam lietotājam.

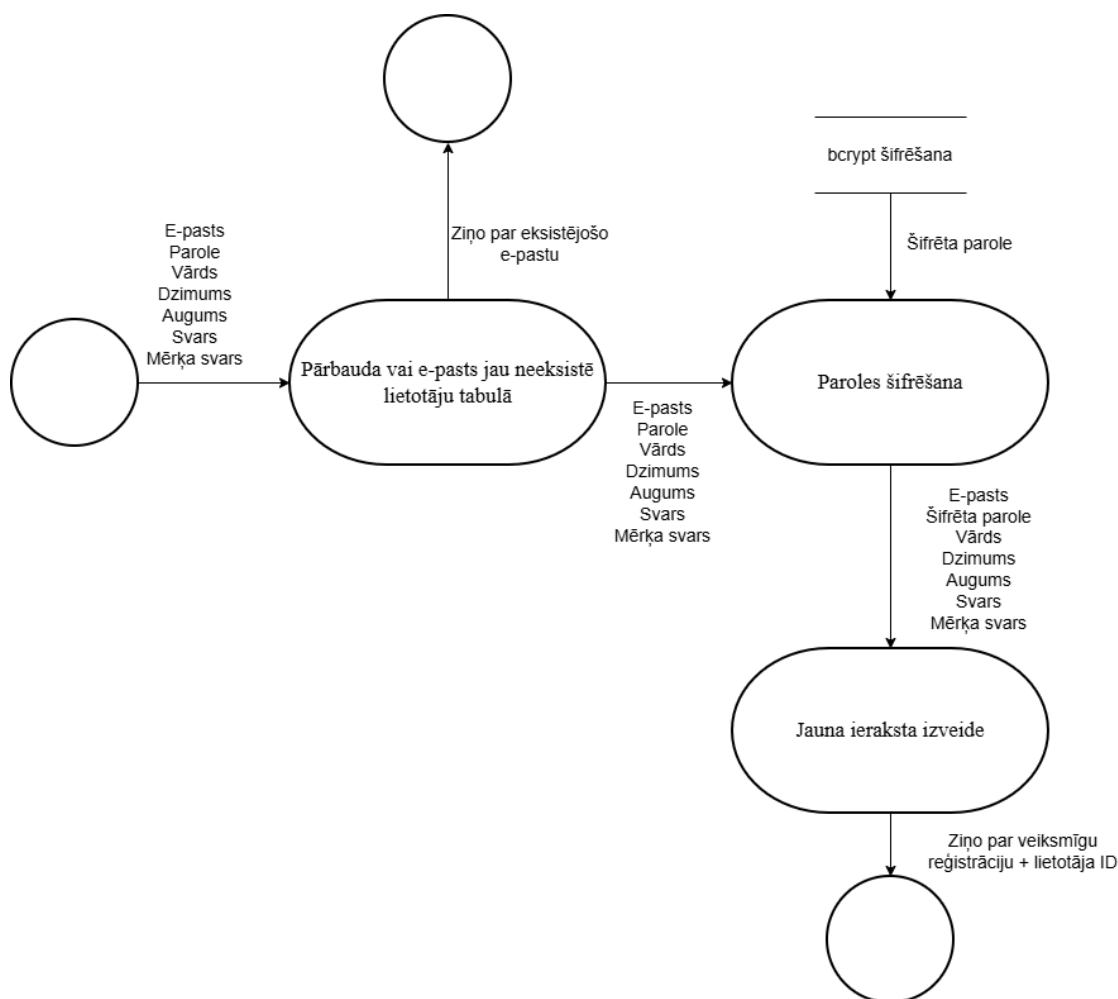
Starp LIETOTĀJU un MALTĪTI pastāv bināra saite ar kardinalitāti "viens pret daudziem", jo viens lietotājs var veidot daudz maltīšu, bet viena maltīte pieder tikai vienam lietotājam.

Starp ĒDIENU un MALTĪTI pastāv bināra saite ar kardinalitāti "daudzi pret daudziem" caur MALTĪTES_ĒDIENS, jo viena maltīte var satur daudz ēdienu, un viens ēdiens var būt iekļauts daudzās maltītēs.

4.2.Funkcionālais sistēmas modelis

4.2.1. Datu plūsmu modelis

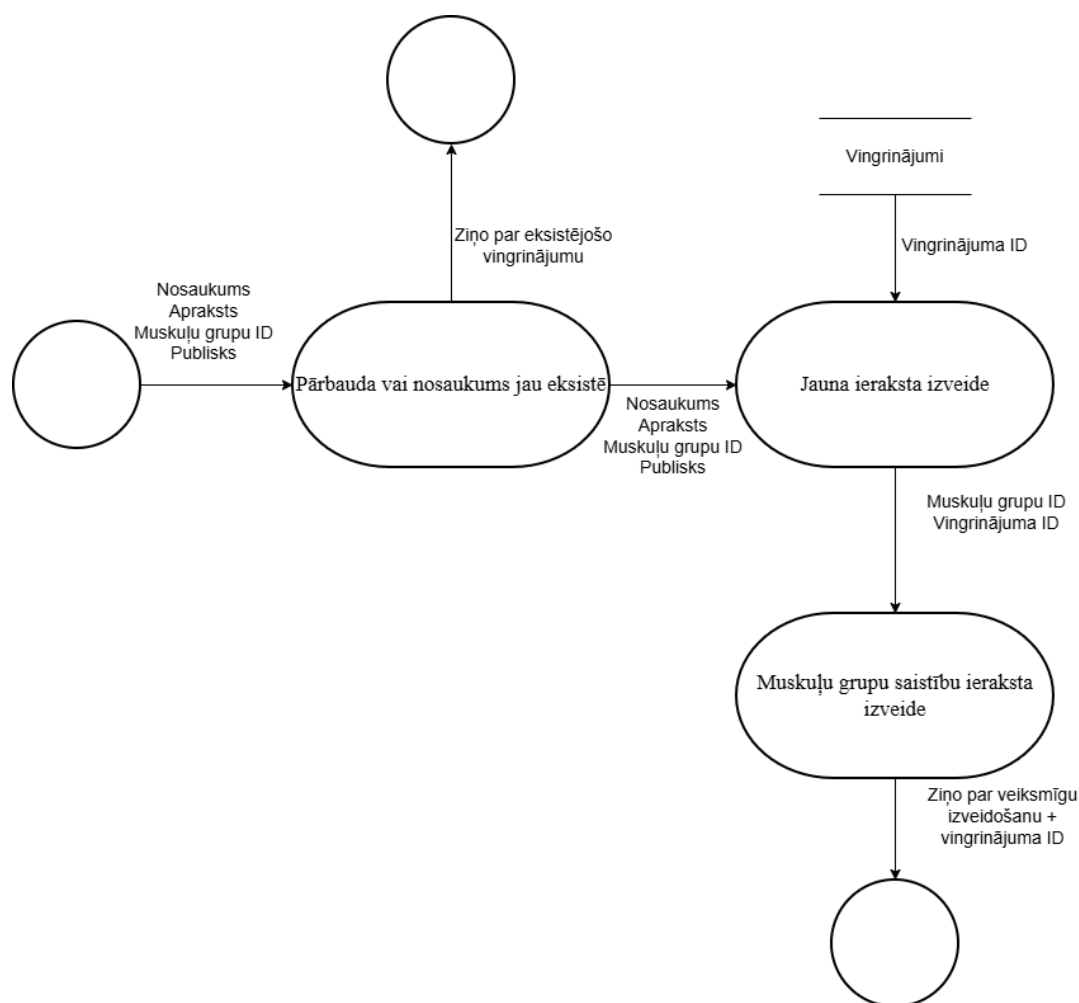
Lietotāja reģistrācija (skat. 4.3. att.)



4.3. att. Lietotāja reģistrācijas datu plūsmu diagramma

Saskarne padod lietotāja datus (e-pasta adrese, parole, vārds, dzimums, augums, svars, mērķa svars). Sistēma validē ieejas datus, pārbauda, vai e-pasta adrese jau neeksistē lietotāju tabulā. Ja validācija ir sekmīga un e-pasta adrese ir unikāla, sistēma šifrē paroli, izveido jaunu ierakstu lietotāju tabulā ar padotajiem datiem un reģistrācijas datumu, tiek parādīts veiksmes ziņojums. Ja e-pasta adrese jau pastāv, tiek izvadīts informatīvais ziņojums par jau eksistējošu e-pastu.

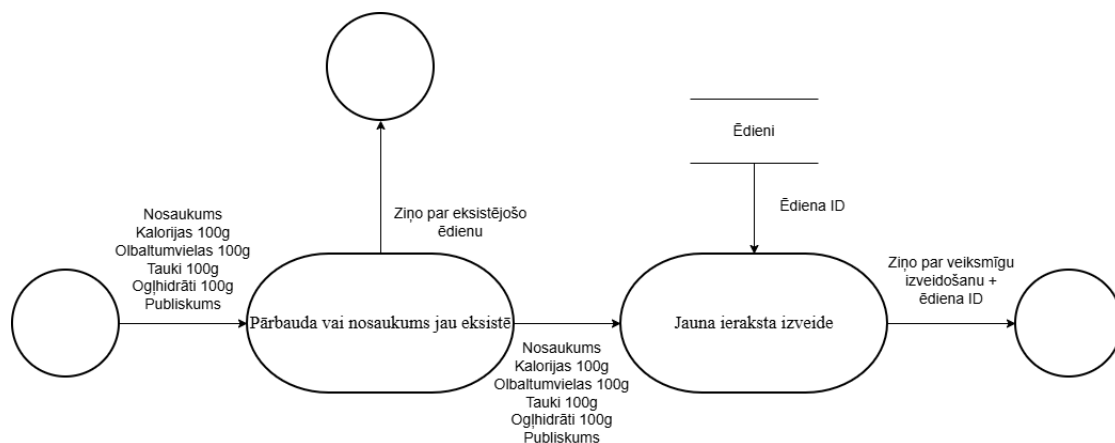
Vingrinājuma izveidošana (skat. 4.4. att.)



4.4. att. Vingrinājuma izveidošanas datu plūsmu diagramma

Saskarne padod vingrinājuma datus (nosaukums, apraksts, muskuļu grupa, publiskuma statuss). Sistēma validē ieejas datus. Ja validācija ir sekmīga, sistēma pārbauda, vai vingrinājuma nosaukums jau neeksistē vingrinājumu tabulā. Ja nosaukums ir unikāls, sistēma izveido jaunu ierakstu vingrinājumu tabulā, pēc tam izveido saiti vingrinājuma un muskuļu grupu starptabulā ar atbilstošajām ID un primārās muskuļu grupas mainīgo. Tiek parādīts veiksmes ziņojums ar izveidotā vingrinājuma ID. Ja nosaukums jau pastāv, tiek izvadīts informatīvais ziņojums.

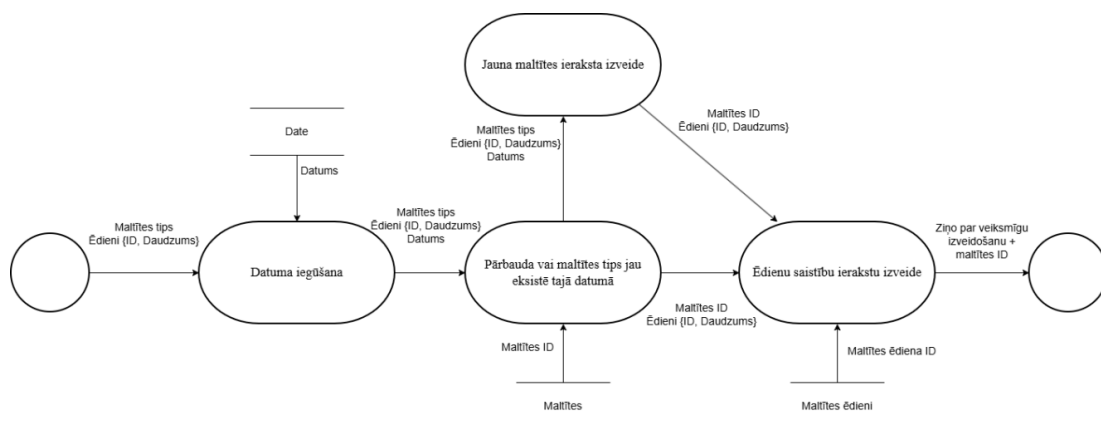
Ēdiena izveidošana (skat. 4.5. att.)



4.5. att. Ēdiena izveidošanas datu plūsmu diagramma

Saskarne padod ēdiena datus (nosaukums, kalorijas uz 100g, proteīni uz 100g, tauki uz 100g, ogļhidrāti uz 100g, publiskuma statuss) un lietotāja ID. Sistēma validē ieejas datus. Ja validācija ir sekmīga, sistēma pārbauda, vai vingrinājuma nosaukums jau neeksistē vingrinājumu tabulā. Ja nosaukums ir unikāls, sistēma izveido jaunu ierakstu ēdiena tabulā ar padotajiem datiem. Tiek parādīts veiksmes ziņojums ar izveidotā ēdiena ID. Ja nosaukums jau pastāv, tiek izvadīts informatīvais ziņojums.

Ēdienu pievienošana maltītei (skat. 4.6. att.)



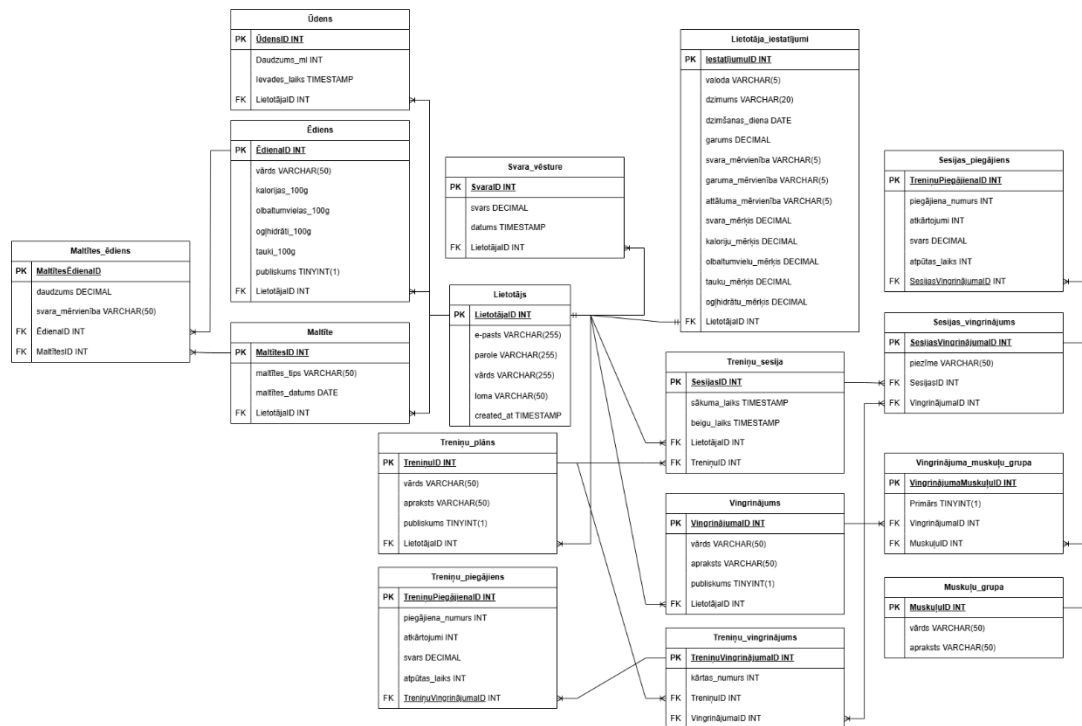
4.6. att. Ēdienu pievienošana maltītei datu plūsmu diagramma

Saskarne padod maltītes tipu, datumu un ēdienu sarakstu (ar ID un daudzumiem gramos) un lietotāja ID. Sistēma validē ieejas datus. Ja validācija ir sekmīga, sistēma pārbauda vai ar padoto maltītes tipu eksistē tajā dienā maltītes ieraksts. Ja ir maltītes ieraksts tad sistēma katram ēdienam izveido ierakstu maltītes ēdiena starptabulā ar maltītes ID, ēdiena ID un daudzumu. Tiek parādīts veiksmes ziņojums ar atjaunināto ēdienu sarakstu pie maltītēm. Ja nav maltītes

ieraksts tad sistēma izveido jaunu ierakstu maltīšu tabulā ar lietotāja ID, maltītes tipu un datumu un veic iepriekš minētās ēdienu ierakstu izveides.

5. DATU STRUKTŪRU APRAKSTS

Datubāzes projektēšanas rezultātā tika izveidotas vairākas tabulas, un tika definēta to relācija jeb saistība (skat. 4.7. att.).



4.7. att. Datubāzes tabulu shēma

Datubāze sastāv no 15 tabulām, kas satur informāciju par lietotājiem, viņu iestatījumiem, svara vēsturi, muskuļu grupām, vingrinājumiem, treniņu plāniem, treniņu sesijām un ēdienu informāciju:

- tabula "lietotājs" sastāv no 6 laukiem un ir savienota ar tabulām "lietotāja_iestatījumi" ar saiti viens-pret-viens, "svara_vēsture", "treniņa_plāns", "treniņa_sesija", "vingrinājums", "maltīte", "ēdiens" un "ūdens" ar saiti viens-pret-daudziem;
- tabula "lietotāja_iestatījumi" sastāv no 14 laukiem un ir savienota ar tabulu "lietotājs" ar saiti viens-pret-viens;
- tabula "svara_vēsture" sastāv no 4 laukiem un ir savienota ar tabulu "lietotājs" ar saiti viens-pret-daudziem;
- tabula "muskuļu_grupa" sastāv no 3 laukiem un ir savienota ar tabulu "vingrinājums" ar saiti daudzi-pret-daudziem caur starptabulu "vingrinājuma_muskuļu_grupa";

- tabula "vingrinājuma_muskuļa_grupa" sastāv no 4 laukiem un ir starptabula daudzi-pret-daudziem relācijā starp tabulām "vingrinājums" un "muskuļu_grupa";
- tabula "vingrinājums" sastāv no 5 laukiem un ir savienota ar tabulām "lietotājs" ar saiti viens-pret-daudziem, "muskuļu_grupa" ar saiti daudzi-pret-daudziem caur starptabulu "vingrinājuma_muskuļa_grupa", un "trenaža_plāns" ar saiti daudzi-pret-daudziem caur starptabulu "trenaža_vingrinājums";
- tabula "trenaža_plāns" sastāv no 5 laukiem un ir savienota ar tabulu "lietotājs" un "trenaža_sesija" ar saiti viens-pret-daudziem, "vingrinājums" ar saiti daudzi-pret-daudziem caur starptabulu "trenaža_vingrinājums";
- tabula "trenaža_vingrinājums" sastāv no 6 laukiem un ir starptabula daudzi-pret-daudziem relācijā starp tabulām "trenaža_plāns" un "vingrinājums", kā arī ar "trenaža_piegājiens" ar saiti viens-pret-daudziem;
- tabula "trenaža_piegājiens" sastāv no 6 laukiem un ir savienota ar tabulu "trenaža_vingrinājums" ar saiti viens-pret-daudziem;
- tabula "trenaža_sesija" sastāv no 5 laukiem un ir savienota ar tabulām "lietotājs" un "trenaža_plāns" ar saiti viens-pret-daudziem;
- tabula "sesijas_vingrinājums" sastāv no 4 laukiem un ir starptabula daudzi-pret-daudziem relācijā starp tabulām "vingrinājums" un "trenaža_sesija", "sesijas_piegājiens" ar saiti viens-pret-daudziem;
- tabula "sesijas_piegājiens" sastāv no 6 laukiem un ir savienota ar tabulu "sesijas_vingrinājums" ar saiti viens-pret-daudziem;
- tabula "ūdens" sastāv no 4 laukiem un ir savienota ar tabulu "lietotājs" ar saiti viens-pret-daudziem;
- tabula "ēdiens" sastāv no 8 laukiem un ir savienota ar tabulu "maltīte" ar saiti daudzi-pret-daudziem caur starptabulu "maltītes_ēdiens", savienota ar tabulu "lietotājs" ar saiti viens-pret-daudziem;
- tabula "maltīte" sastāv no 4 laukiem un ir savienota ar tabulu "ēdiens" ar saiti daudzi-pret-daudziem caur starptabulu "maltītes_ēdiens", savienota ar tabulu "lietotājs" ar saiti viens-pret-daudziem;

- tabula "maltītes_ēdiens" sastāv no 5 laukiem un ir starptabula daudzi-pret-daudziem relācijā starp tabulām "maltīte" un "ēdiens".

Tabula “**Lietotājs**” satur sistēmas lietotāja pamatdatus. (skat. 4.1. tabula)

4.1. tabula

Tabulas “**Lietotājs**” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	LietotājaID	INT	-	Lietotāja unikālais identifikators
2.	e-pasts	VARCHAR	255	Lietotāja e-pasta adrese
3.	parole	VARCHAR	255	Lietotāja parole
4.	vārds	VARCHAR	255	Lietotāja vārds
5.	loma	VARCHAR	50	Lietotāja loma
6.	created_at	TIMESTAMP	-	Ieraksta izveides datums un laiks

Tabula “**Lietotāja_iestatījumi**” satur lietotāja profila, mērvienību un uztura mērķu iestatījumus. (skat. 4.2. tabula)

4.2. tabula

Tabulas “**Lietotāja_iestatījumi**” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	IestatījumuID	INT	-	Iestatījumu ieraksta identifikators
2.	valoda	VARCHAR	5	Lietotāja izvēlētā valoda
3.	dzimums	VARCHAR	20	Lietotāja dzimums
4.	dzimšanas_diena	DATE	-	Lietotāja dzimšanas diena
5.	garums	DECIMAL	-	Lietotāja auguma vērtība
6.	svara_mērvienība	VARCHAR	5	Izvēlētā svara mērvienība
7.	garuma_mērvienība	VARCHAR	5	Izvēlētā garuma mērvienība
8.	attāluma_mērvienība	VARCHAR	5	Izvēlētā attāluma mērvienība
9.	svara_mērķis	DECIMAL	-	Lietotāja vēlamais svars
10.	kaloriju_mērķis	DECIMAL	-	Dienas kaloriju mērķis
11.	olbaltumvielu_mērķis	DECIMAL	-	Dienas olbaltumvielu mērķis
12.	tauku_mērķis	DECIMAL	-	Dienas tauku mērķis
13.	ogļhidrātu_mērķis	DECIMAL	-	Dienas ogļhidrātu mērķis
14.	LietotājaID	INT	-	Saistītā lietotāja identifikators

Tabula “**Svara_vēsture**” glabā lietotāja svara izmaiņu vēstures ierakstus. (skat. 4.3. tabula)

4.3. tabula

Tabulas “**Svara_vēsture**” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	SvaraID	INT	-	Svara vēstures ieraksta identifikators
2.	svars	DECIMAL	-	Reģistrētais svars
3.	datums	TIMESTAMP	-	Svara ieraksta datums un laiks
4.	LietotājaID	INT	-	Saistītā lietotāja identifikators

Tabula “**Muskuļu_grupa**” satur sistēmā izmantoto muskuļu grupu katalogu. (skat. 4.4. tabula)

4.4. tabula

Tabulas “**Muskuļu_grupa**” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	MuskuļuID	INT	-	Muskuļu grupas identifikators
2.	vārds	VARCHAR	50	Muskuļu grupas nosaukums
3.	apraksts	VARCHAR	50	Muskuļu grupas apraksts

Tabula “**Vingrinājums**” satur lietotāja vai sistēmas pieejamos vingrinājumus. (skat. 4.5. tabula)

4.5. tabula

Tabulas “**Vingrinājums**” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	VingrinājumaID	INT	-	Vingrinājuma identifikators
2.	vārds	VARCHAR	50	Vingrinājuma nosaukums
3.	apraksts	VARCHAR	50	Vingrinājuma apraksts
4.	publiskums	TINYINT	1	Vai vingrinājums ir publiski pieejams
5.	LietotājaID	INT	-	Lietotāja identifikators, kas izveidoja vingrinājumu

Tabula “**Vingrinājuma_muskuļu_grupa**” sasaista vingrinājumus ar muskuļu grupām un norāda primāro noslodzi. (skat. 4.6. tabula)

4.6. tabula

Tabulas “**Vingrinājuma_muskuļu_grupa**” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	VingrinājumaMuskuļuID	INT	-	Sasaistes ieraksta identifikators
2.	Primārs	TINYINT	1	Norāda, vai muskuļu grupa ir primārā
3.	VingrinājumaID	INT	-	Saistītā vingrinājuma identifikators
4.	MuskuļuID	INT	-	Saistītās muskuļu grupas identifikators

Tabula “**Treniņu_plāns**” satur lietotāja izveidotos treniņu plānus. (skat. 4.7. tabula)

4.7. tabula

Tabulas “**Treniņu_plāns**” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	TreniņuID	INT	-	Treniņu plāna identifikators
2.	vārds	VARCHAR	50	Treniņu plāna nosaukums
3.	apraksts	VARCHAR	50	Treniņu plāna apraksts
4.	publiskums	TINYINT	1	Vai treniņu plāns ir publiski pieejams
5.	LietotājaID	INT	-	Lietotāja identifikators, kas izveidoja plānu

Tabula “**Treniņu_vingrinājums**” sasaista treniņu plānu ar konkrētiem vingrinājumiem. (skat. 4.8. tabula)

4.8. tabula

Tabulas “**Treniņu_vingrinājums**” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	TreniņuVingrinājumaID	INT	-	Treniņu vingrinājuma identifikators
2.	Kārtas_numurs	INT	-	Vingrinājuma kārtas numurs treniņu plānā
5.	TreniņuID	INT	-	Saistītā treniņu plāna identifikators
6.	VingrinājumaID	INT	-	Saistītā vingrinājuma identifikators

Tabula “**Treniņu_piegājiens**” satur plānotos piegājienu katram treniņu plānā iekļautajam vingrinājumam. (skat. 4.9. tabula)

4.9. tabula

Tabulas “**Treniņu_piegājiens**” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	TreniņuPiegājienaID	INT	-	Piegājiena identifikators
2.	piegājiena numurs	INT	-	Piegājiena kārtas numurs
3.	atkārtojumi	INT	-	Plānotais atkārtojumu skaits
4.	svars	DECIMAL	-	Plānotais svars
5.	atpūtas laiks	INT	-	Atpūtas ilgums starp piegājieniem
6.	TreniņuVingrinājumaID	INT	-	Saistītā treniņu vingrinājuma identifikators

Tabula “**Treniņu_sesija**” glabā informāciju par lietotāja veiktajām treniņu sesijām. (skat. 4.10. tabula)

4.10. tabula

Tabulas “**Treniņu_sesija**” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	SesijasID	INT	-	Treniņu sesijas identifikators
2.	sākuma_laiks	TIMESTAMP	-	Sesijas sākuma datums un laiks
3.	beigu_laiks	TIMESTAMP	-	Sesijas beigu datums un laiks
4.	LietotājaID	INT	-	Sesijas lietotāja identifikators
5.	TreniņuID	INT	-	Saistītā treniņu plāna identifikators

Tabula “**Sesijas_vingrinājums**” glabā sesijā izpildītos vingrinājumus. (skat. 4.11. tabula)

4.11. tabula

Tabulas “**Sesijas_vingrinājums**” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	SesijasVingrinājumaID	INT	-	Sesijas vingrinājuma identifikators
2.	piezīme	VARCHAR	50	Piezīmes par vingrinājuma izpildi
3.	SesijasID	INT	-	Saistītās sesijas identifikators
4.	VingrinājumaID	INT	-	Saistītā vingrinājuma identifikators

Tabula “**Sesijas_piegājiens**” satur faktiskos piegājienu, kas veikti sesijas laikā. (skat. 4.12. tabula)

4.12. tabula

Tabulas “**Sesijas_piegājiens**” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	TreniņuPiegājienaID	INT	-	Sesijas piegājiena identifikators
2.	piegājiena numurs	INT	-	Piegājiena kārtas numurs
3.	atkārtojumi	INT	-	Faktiskais atkārtojumu skaits
4.	svars	DECIMAL	-	Faktiski izmantotais svars
5.	atpūtas laiks	INT	-	Faktiskais atpūtas ilgums
6.	SesijasVingrinājumaID	INT	-	Saistītā sesijas vingrinājuma identifikators

Tabula “**Ūdens**” satur ievadītos ūdens ierakstus. (skat. 4.13. tabula)

4.13. tabula

Tabulas “**Ūdens**” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	ŪdensID	INT	-	Ūdeņa ieraksta identifikators
2.	daudzums ml	INT	-	Ūdens daudzums ml
3.	ievades laiks	TIMESTAMP	-	Ūdens ieraksta datums un laiks
4.	LietotājaID	INT	-	Lietotāja identifikators

Tabula “**Ēdiens**” satur ēdienu uzturvērtības datus uz 100 gramiem. (skat. 4.14. tabula)

4.14. tabula

Tabulas “**Ēdiens**” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	ĒdienaID	INT	-	Ēdiena identifikators
2.	vārds	VARCHAR	50	Ēdiena nosaukums
3.	kalorijas_100g	DECIMAL	-	Kaloriju daudzums uz 100 g
4.	olbaltumvielas_100g	DECIMAL	-	Olbaltumvielu daudzums uz 100 g
5.	ogļhidrāti_100g	DECIMAL	-	Ogļhidrātu daudzums uz 100 g
6.	tauki_100g	DECIMAL	-	Tauku daudzums uz 100 g
7.	publiskums	TINYINT	1	Vai ēdiens ir publiski pieejams
8.	LietotājaID	INT	-	Ēdiena izveidotāja identifikators

Tabula “**Maltīte**” satur lietotāja maltīšu ierakstus ar maltītes tipu un datumu. (skat. 4.15. tabula) (skat. 4.15. tabula)

4.15. tabula

Tabulas “**Maltīte**” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	MaltītesID	INT	-	Maltītes identifikators
2.	maltītes tips	VARCHAR	50	Maltītes tips
3.	maltītes datums	DATE	-	Maltītes datums
4.	LietotājaID	INT	-	Lietotāja identifikators

Tabula “**Maltītes_ēdiens**” ir starptabula starp maltītēm un ēdieniem un glabā pievienotā ēdiena daudzumu. (skat. 4.16. tabula)

4.16. tabula

Tabulas “**Maltītes_ēdiens**” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	MaltītesĒdienaID	INT	-	Starpieraksta identifikators
2.	daudzums	DECIMAL	-	Maltītei pievienotā ēdiena daudzums
3.	svara mērvienība	VARCHAR	50	Daudzuma mērvienība
4.	ĒdienaID	INT	-	Saistītā ēdiena identifikators
5.	MaltītesID	INT	-	Saistītās maltītes identifikators

6. LIETOTĀJA CEĻVEDIS

6.1. Sistēmas prasības aparatūrai un programmatūrai

Lietotnes JustFitness lietošanai ir nepieciešams dators vai portatīvais dators izstrādes un servera palaišanas vajadzībām, kā arī mobilā ierīce vai emulatori lietotnes lietošanai.

Lai sistēmu būtu iespējams palaist jūsu datoram ir jāatbilst sekojošām prasībām:

- Windows 10 vai jaunāks;
- macOS 11 vai jaunāks;
- Node.js 20.19.x vai jaunāks.

Tā ir piemērota ērtai lietošanai lielākajā daļā ierīču, ja tās atbilst sekojošām prasībām:

- IOS 15.1 vai jaunāks;
- Android 7.0 vai jaunāks.

6.2. Sistēmas instalācija un palaišana

Lai instalētu un palaistu sistēmu, lietotājam vispirms ir jāparūpējas par sekojoša programmnodrošinājuma instalēšanu uz izmantojamās ierīces:

- Visual Studio Code vai līdzīgs koda redaktors;
- Expo Go aplikācija uz izvēlētajā mobilās ierīces;
- Laragon vai cits MySQL bāzēts lokālās datubāzes serveris;
- Node.js;
- Git.

1. Projekta klonēšana no GitHub:

1.1. Atveriet terminālu un izpildiet komandu (skat. 6.1. att. Git Clone):

```
git clone https://github.com/22DP3HEisu/JustFitness_Mobile.git
```

6.1. att. Git Clone

1.2. Ieejiet projekta mapē (skat. 6.2. att. Projekta mape):

```
cd JustFitness_Mobile/
```

6.2. att. Projekta mape

2. Sistēmas pamatu uzstādīšana:

2.1. Palaidiet sistēmas uzstādīšanas komandu (skat. 6.3. att. Sistēmas uzstādīšana):

```
npm run setup
```

6.3. att. Sistēmas uzstādīšana

2.2. Atveriet .env failu iekš “\Backend” mapes un iestatiet sistēmas un datubāzes konfigurācijas informāciju.

3. Palaidiet datubāzes migrācijas, lai izveidotu tabulas (skat. 6.4. att. Datubāzes migrācijas):

```
npm run setup-db -w Backend
```

6.4. att. Datubāzes migrācijas

4. Aplikācijas palaišana:

4.1. Palaidiet aplikāciju no projektu saknes (skat. 6.5. att. Aplikācijas palaišana):

```
npm start
```

6.5. att. Aplikācijas palaišana

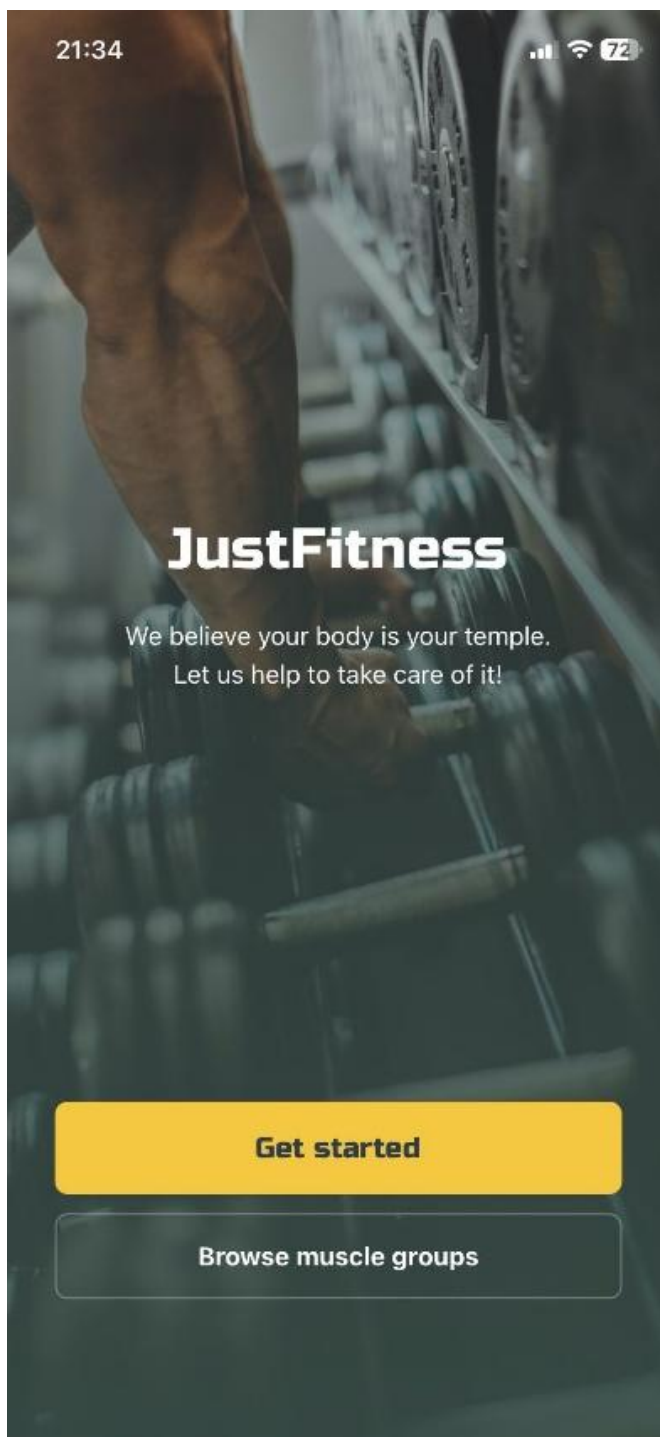
4.2. Ar mobīlo ierīci noskenējiet doto QR kodu no konsoles, lai atvērtu aplikāciju iekš Expo Go aplikācijas (skat. 6.6. att. QR kods)



6.6. att. QR kods

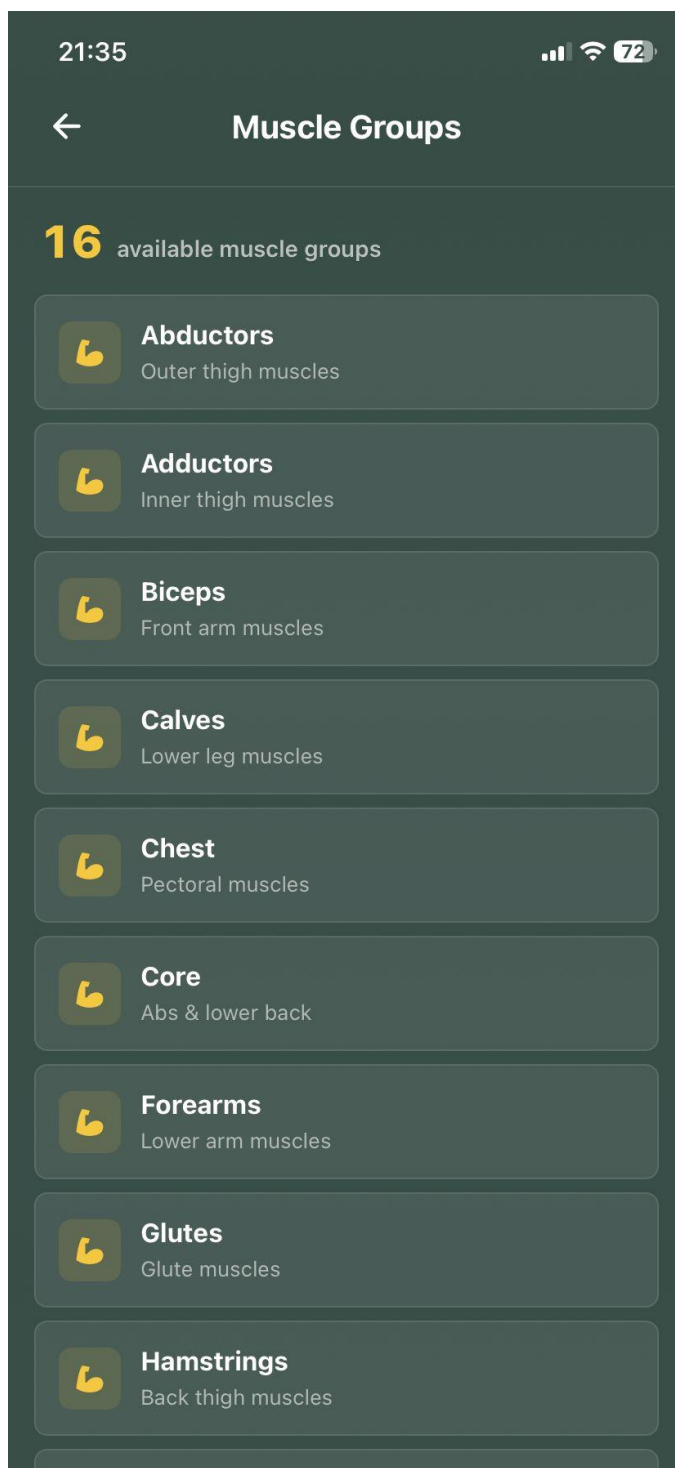
6.3. Programmas apraksts

Sākotnējā lapā lietotājs redz lietotnes nosaukumu JustFitness, īsu motivējošu tekstu un divas galvenās darbības. Poga “Get started” novirza lietotāju uz konta izveides procesu, savukārt poga “Browse muscle groups” ļauj bez pieslēgšanās apskatīt sistēmā pieejamo muskuļu grupu informāciju. Šis skats paredzēts jauna lietotāja iepazīstināšanai ar lietotni. (skat. 6.1. att.)



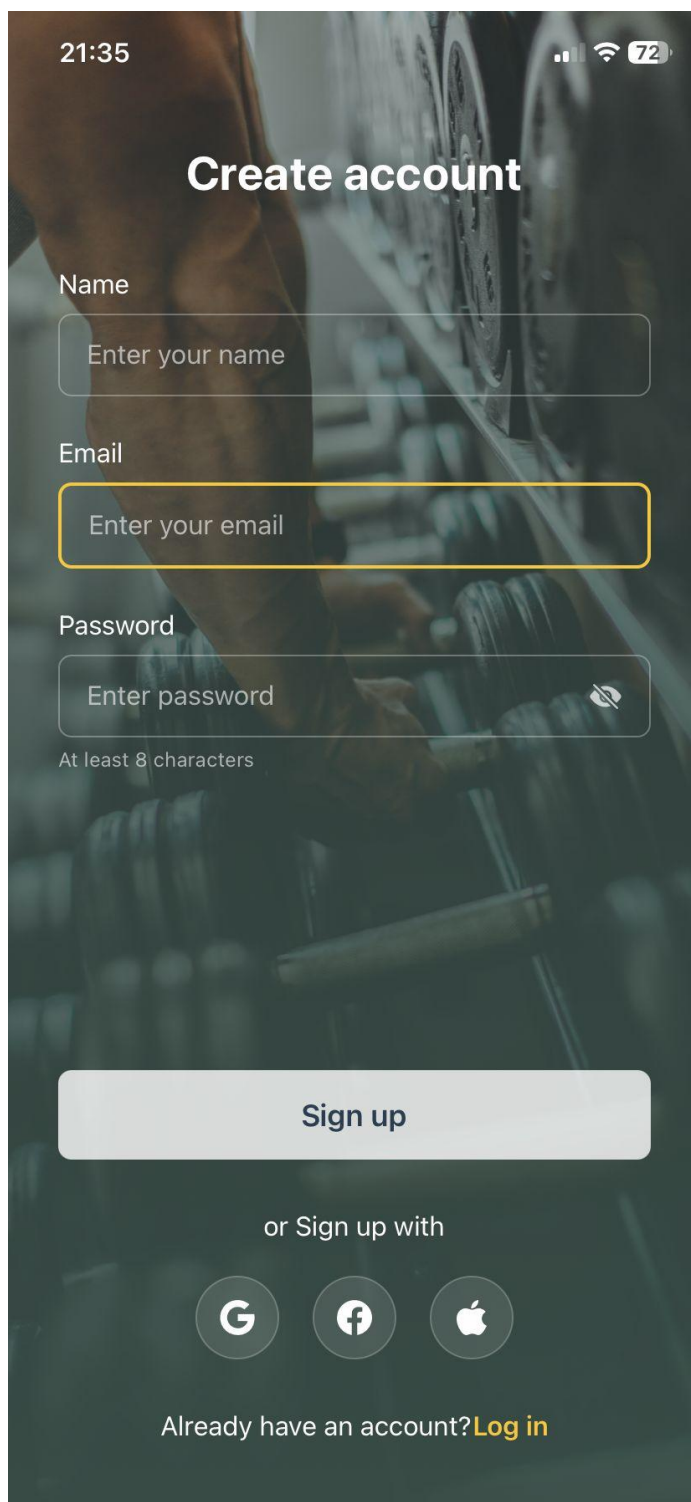
6.1. att. Sākotnējā lapa

Muskuļu grupu skatā tiek attēlots sistēmā pieejamo muskuļu grupu katalogs. Katrs ieraksts satur muskuļu grupas nosaukumu un īsu aprakstu, piemēram, “Biceps”, “Chest”, “Core” u.c. Lietotājs var pārlūkot sarakstu un iepazīties ar muskuļu grupām arī bez autorizācijas. Augšējā kreisajā stūrī pieejama atpakaļ poga, kas atgriež lietotāju iepriekšējā skatā. (skat. 6.2. att.)



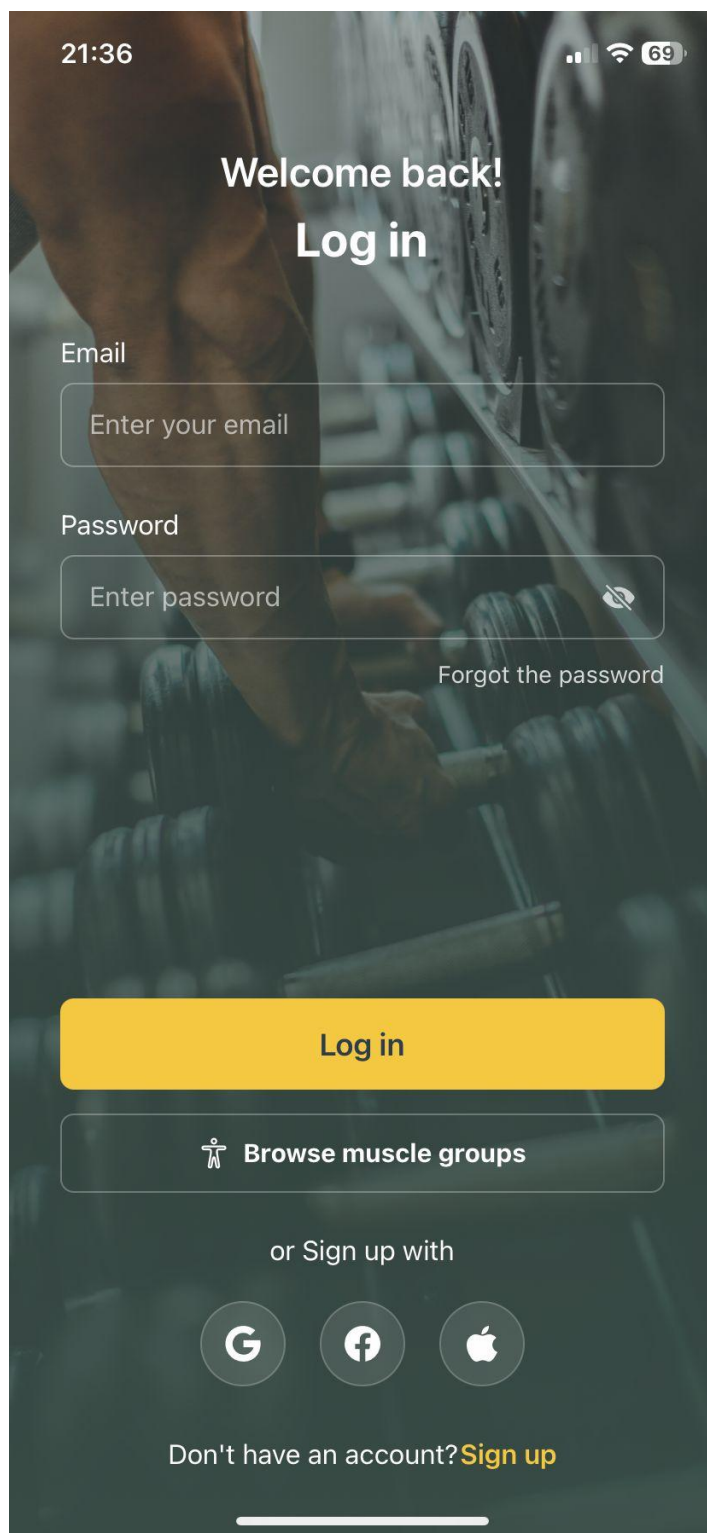
6.2. att. Muskuļu grupu saraksts

Reģistrācijas lapā lietotājs var izveidot jaunu kontu, ievadot vārdu, e-pasta adresi un paroli. Paroles laukā ir redzama prasība par minimālo paroles garumu. Ja lietotājs neaizpilda obligātos laukus vai ievada nederīgu informāciju, sistēma parāda kļūdas paziņojumu. Poga “Sign up” nosūta reģistrācijas datus serverim. Ja lietotājam jau ir konts, saite “Log in” novirza uz pieslēgšanās lapu. (skat. 6.3. att.)

A mobile application registration screen with a dark background featuring a blurred image of a person's arm and a mechanical gear. At the top, the status bar shows the time 21:35, signal strength, Wi-Fi, and 72% battery. The title "Create account" is centered in white. Below it are three input fields: "Name" with placeholder "Enter your name", "Email" with placeholder "Enter your email" (highlighted with a yellow border), and "Password" with placeholder "Enter password" and a toggle icon. A note "At least 8 characters" is below the password field. A large grey "Sign up" button is centered below the fields. Underneath is the text "or Sign up with" followed by three circular icons for Google, Facebook, and Apple. At the bottom, the text "Already have an account? Log in" is displayed, with "Log in" in yellow.

6.3. att. Reģistrācijas lapa

Pieslēgšanās lapā lietotājs ievada savu e-pasta adresi un paroli. Paroles ievades laukā ir pieejama ikona paroles redzamības pārslēgšanai. Poga “Log in” veic autentifikāciju. Ja dati ir pareizi, lietotājs tiek novirzīts uz galveno paneli. Ja parole vai e-pasts neatbilst datubāzē esošajiem datiem, tiek parādīts kļūdas paziņojums. Lapā pieejama arī saite uz reģistrācijas lapu. (skat. 6.4. att.)

A mobile application login screen with a dark background featuring a blurred image of a person's arm and a dumbbell. At the top, the status bar shows the time 21:36, signal strength, Wi-Fi, and 69% battery. The main heading reads "Welcome back! Log in". Below this are two input fields: "Email" with the placeholder "Enter your email" and "Password" with the placeholder "Enter password" and a toggle icon. A link "Forgot the password" is positioned to the right of the password field. A large yellow "Log in" button is centered below the inputs. Underneath is a button with a person icon and the text "Browse muscle groups". Below that, the text "or Sign up with" is followed by three circular social media icons: Google, Facebook, and Apple. At the bottom, the text "Don't have an account? Sign up" is displayed, with "Sign up" in yellow. A horizontal line is at the very bottom of the screen.

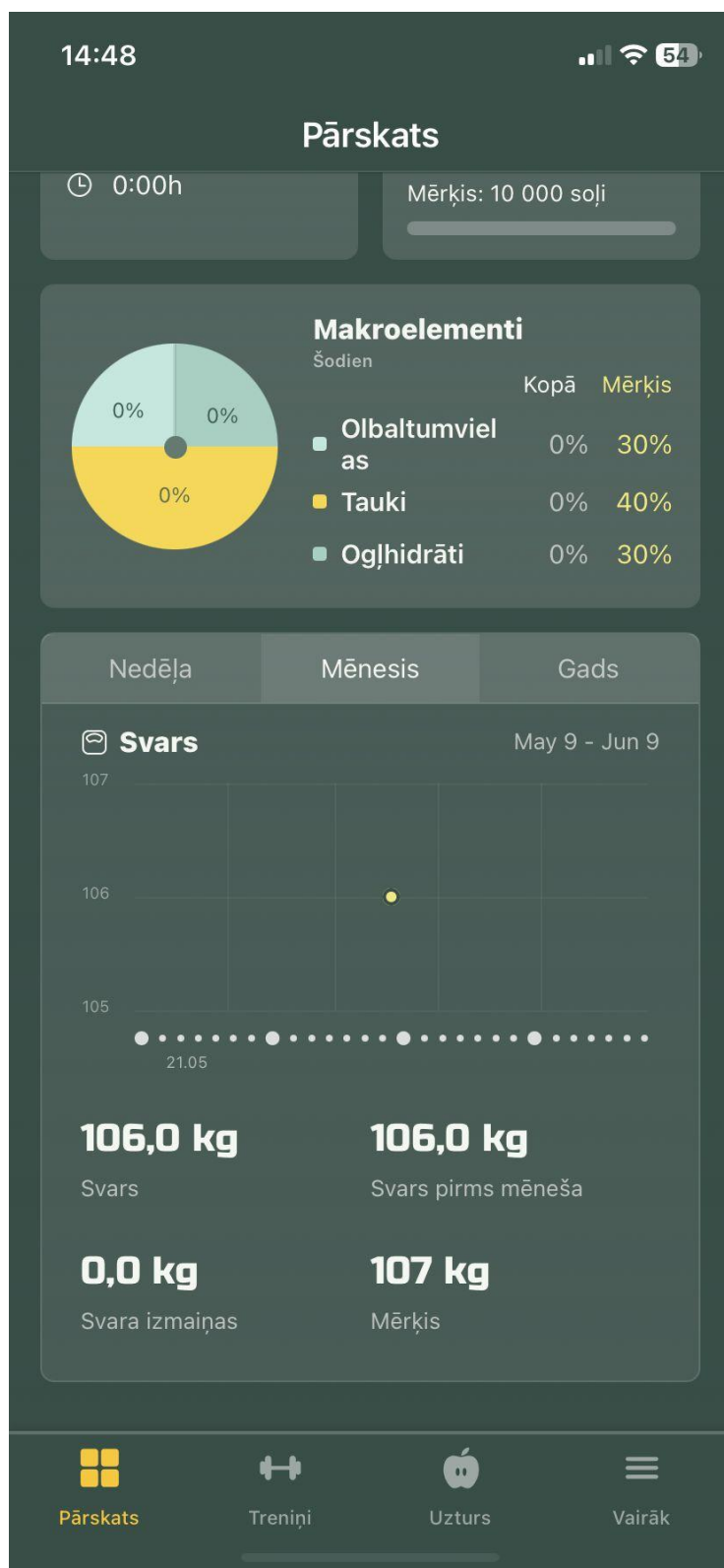
6.4. att. Pieslēgšanās lapa

Galvenajā panelī lietotājs redz dienas kopsavilkumu. Tajā tiek attēlots kaloriju mērķis, atlikušās kalorijas, ēdienreizēs uzņemtās kalorijas, treniņu dati un soļu mērķis. Zemāk redzams makroelementu sadalījums, kurā tiek parādīti proteīni, tauki un ogļhidrāti. Apakšējā navigācijas joslā lietotājs var pārslēgties starp paneļa, treniņu, uztura un papildu izvēlnes sadaļām. (skat. 6.5. att.)



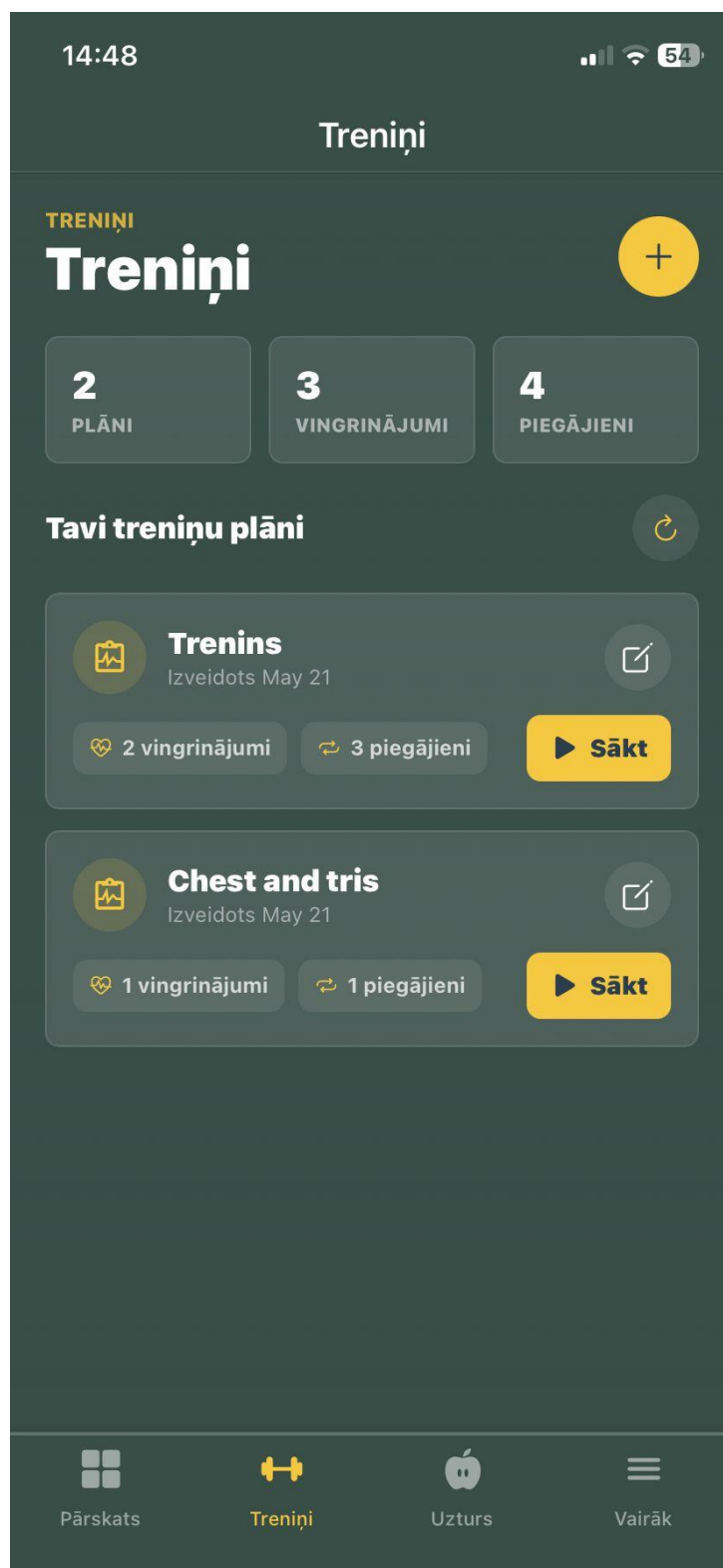
6.5. att. Galvenais panelis

Svara sadaļā lietotājs redz svara izmaiņu grafiku izvēlētajā periodā. Ir iespējams pārslēgt periodu pa nedēļu, mēnesi vai gadu. Zem grafika tiek parādīts pašreizējais svars, salīdzinājums ar iepriekšējo periodu, svara izmaiņa un mērķa svars. Šī sadaļa ļauj lietotājam pārraudzīt progresu ilgākā laika posmā. (skat. 6.6. att.)



6.6. att. Galvenā paneļa svara grafiks

Treniņu sadaļā tiek parādīts lietotāja treniņu plānu pārskats. Augšpusē redzams kopējais plānu, vingrinājumu un setu skaits. Ar dzeltenu “+” pogu lietotājs var izveidot jaunu treniņu. Katram treniņa plānam ir redzams nosaukums, vingrinājumu skaits, setu skaits, rediģēšanas poga un poga “Sākt”, ar kuru var sākt treniņa sesiju. (skat. 6.7. att.)



6.7. att. Treniņu sadaļa

Treniņa izveides formā lietotājs ievada treniņa nosaukumu un aprakstu. Sadaļā “Vingrinājumi” var pievienot vingrinājumus, izmantojot pogu “Pievienot”. Katram vingrinājumam var ievadīt atkārtojumu skaitu un svaru, kā arī pievienot vairākus setus. Poga “Izveidot treniņu” saglabā treniņa plānu datubāzē. (skat. 6.8. att.)

14:49

← Izveidot treniņu

Treniņa nosaukums *

piem., ķermeņa augšdaļa, kāju diena, pilns ķe...

Apraksts (neobligāts)

Pievienojiet piezīmes par šo treniņu...

Vingrinājumi + Pievienot

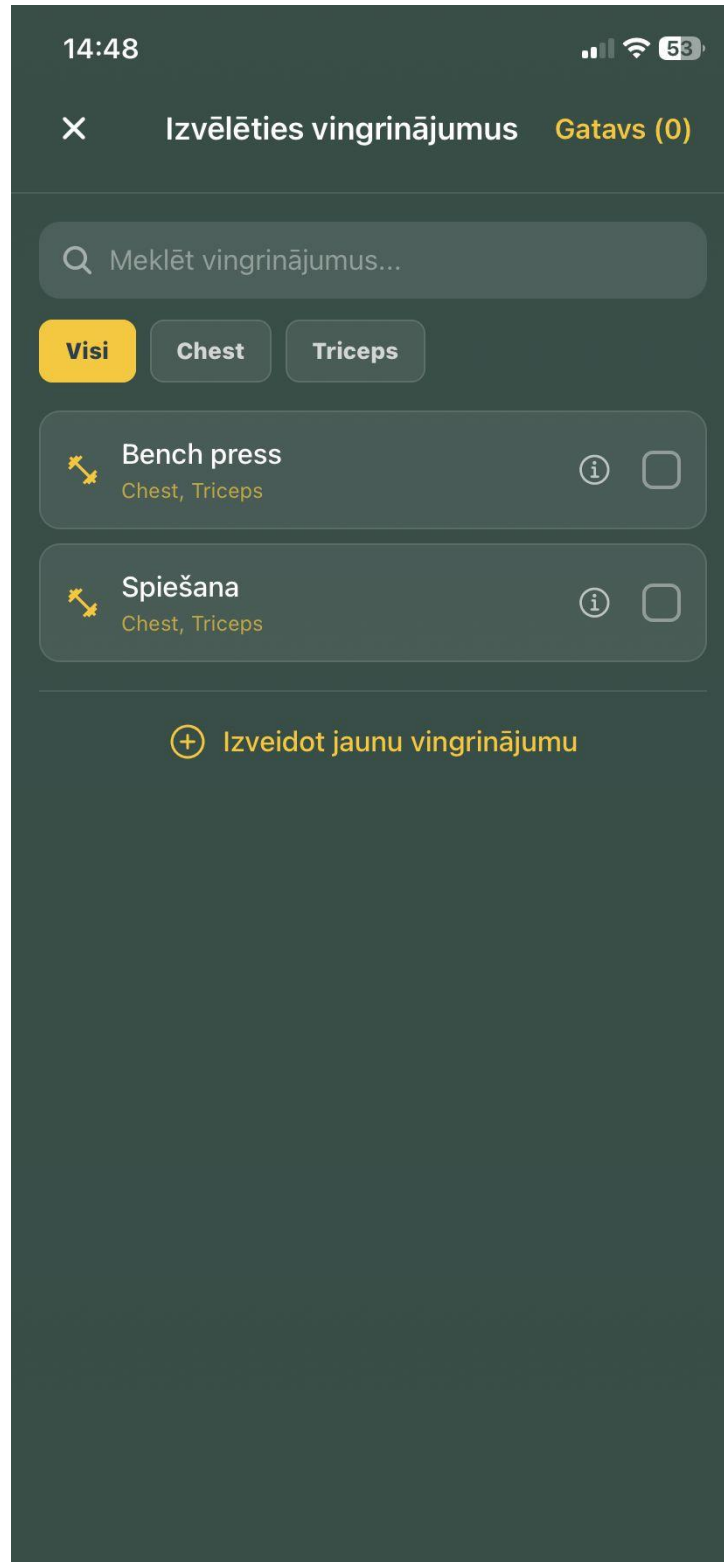
PIEGĀJIE NS	ATKĀRTOJUMI	SVARS
1	10	—
2	10	—

+ Pievienot piegājieni

✓ Izveidot treniņu

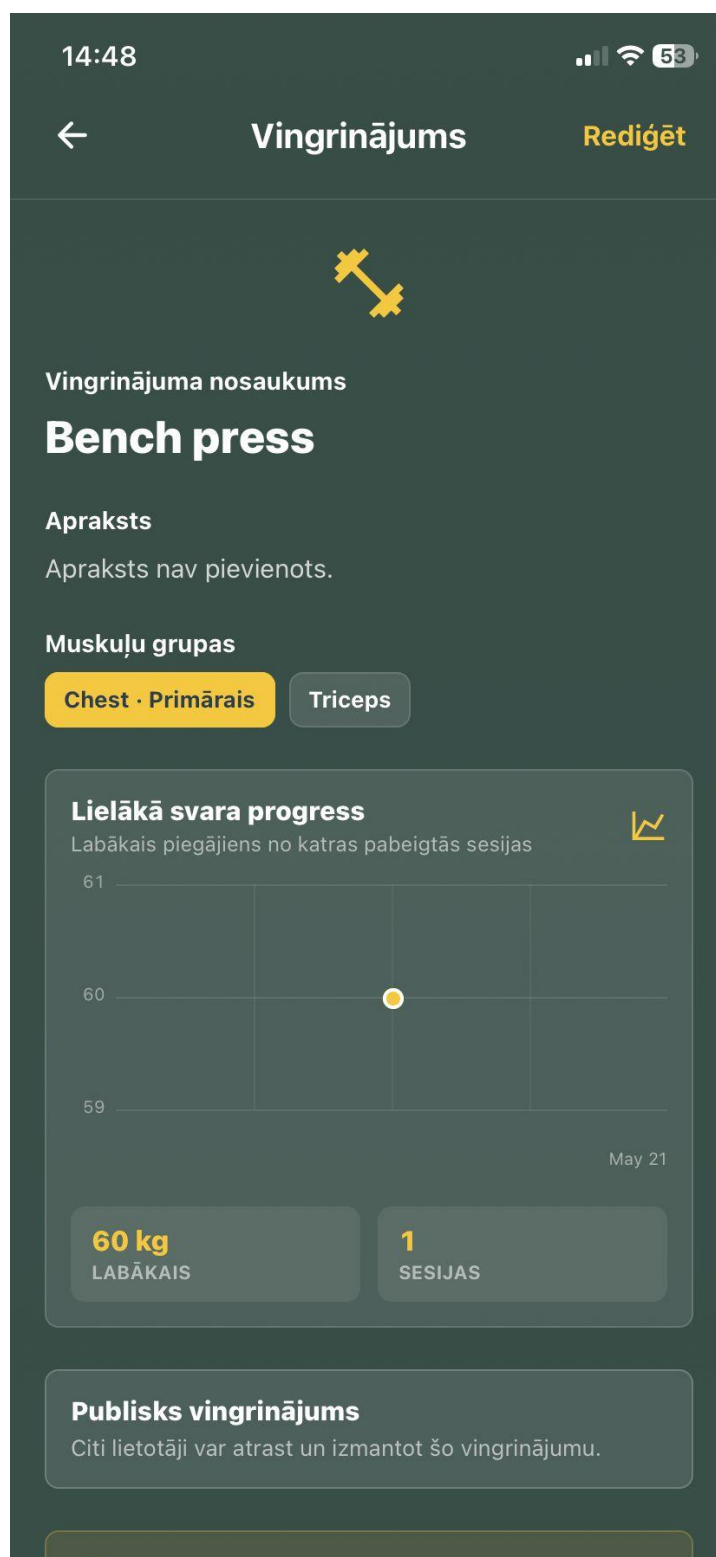
6.8. att. Treniņa izveidošana

Šajā skatā lietotājs izvēlas vingrinājumus, ko pievienot treniņu plānam. Augšā ir meklēšanas lauks, kas ļauj atrast vingrinājumu pēc nosaukuma. Zem tā atrodas muskuļu grupu filtri. Katram vingrinājumam ir informācijas ikona un izvēles lauks. Lietotājs var atzīmēt vairākus vingrinājumus un apstiprināt izvēli ar pogu “Gatavs”. (skat. 6.9. att.)



6.9. att. Vingrinājumu izvēle treniņam

Vingrinājuma informācijas skatā tiek parādīts vingrinājuma nosaukums, apraksts un piesaistītās muskuļu grupas. Ja vingrinājumam ir treniņu vēsture, sadaļā “Lielākā svara progress” tiek parādīti dati par labāko svaru. Redzams arī vingrinājuma privātuma statuss, piemēram, “Privāts vingrinājums”. Augšējā labajā stūrī ir poga “Rediģēt”, kas atver rediģēšanas formu. (skat. 6.10. att.)



6.10. att. Vingrinājuma informācija

Vingrinājuma rediģēšanas skatā lietotājs var mainīt vingrinājuma nosaukumu, aprakstu un galveno muskuļu grupu. Galvenās muskuļu grupas lauks atver izvēles logu, kur lietotājs izvēlas vienu primāro muskuļu grupu. Augšējā labajā stūrī pieejama poga “Atcelt”, kas atceļ rediģēšanu un atgriež iepriekšējā skatā. (skat. 6.11. att.)

The screenshot displays the 'Rediģēt vingrinājumu' (Edit exercise) screen. At the top, the time is 14:48 and the battery is at 53%. The screen has a dark green background with white text. The title 'Rediģēt vingrinājumu' is centered at the top, with a back arrow on the left and an 'Atcelt' (Cancel) button on the right. Below the title, there are three main sections: 1. 'Vingrinājuma nosaukums' (Exercise name) with a text input field containing 'Bench press'. 2. 'Apraksts' (Description) with a text input field containing 'Kā izpildīt šo vingrinājumu...'. 3. 'Muskuļu grupas' (Muscle groups) with two rows: 'Chest' (selected) and '1 secondary selected'. Below these is a 'Lielākā svara progress' (Maximum weight progress) section. It features a line graph showing a single data point at 60 kg on May 21. At the bottom, there are two summary boxes: '60 kg LABĀKAIS' (Maximum weight) and '1 SESIJAS' (Sessions).

14:48 53

← Rediģēt vingrinājumu Atcelt

Vingrinājuma nosaukums

Bench press

Apraksts

Kā izpildīt šo vingrinājumu...

Muskuļu grupas

Chest >

1 secondary selected >

Lielākā svara progress

Labākais piegājiens no katras pabeigtās sesijas

61

60

59

May 21

60 kg LABĀKAIS

1 SESIJAS

6.11. att. Vingrinājuma rediģēšana

Rediģēšanas skata turpinājumā lietotājs redz papildu muskuļu grupu izvēli, svara progresu sadaļu, privātuma slēdzi un darbību pogas. Slēdzis ļauj noteikt, vai vingrinājums būs privāts vai publiski pieejams citiem lietotājiem. Poga “Saglabāt izmaiņas” saglabā labojumus, bet poga “Dzēst vingrinājumu” dzēš vingrinājumu pēc apstiprinājuma. (skat. 6.12. att.)



6.12. att. Vingrinājuma papildu dati

Jauna vingrinājuma formā lietotājam jāievada vingrinājuma nosaukums un jāizvēlas galvenā muskuļu grupa. Papildus var ievadīt aprakstu un izvēlēties sekundārās muskuļu grupas. Slēdzis “Publisks vingrinājums” nosaka, vai vingrinājumu varēs izmantot citi lietotāji. Poga “Izveidot vingrinājumu” saglabā ierakstu datubāzē. (skat. 6.13. att.)



14:48

← Izveidot vingrinājumu



Vingrinājuma nosaukums *

piem., spiešana guļus, pietupieni, pievilkšanās

Apraksts (neobligāts)

Kā izpildīt šo vingrinājumu...

Galvenā muskuļu grupa *

 Pieskarieties, lai izvēlētos galveno muskuļu grupu >

Papildu muskuļu grupas (neobligāti)

 Pieskarieties, lai izvēlētos papildu muskuļu grupas >

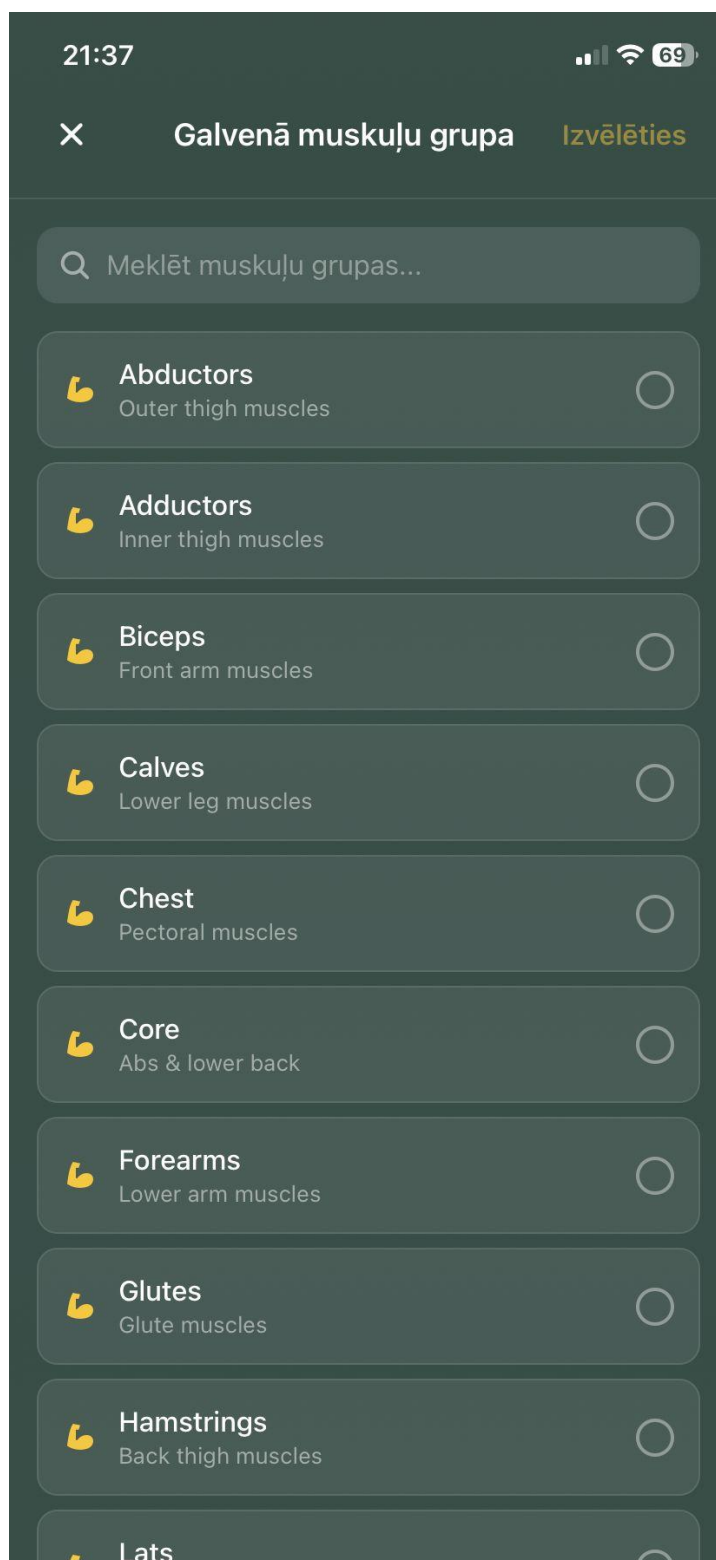
Publisks vingrinājums ☐

Tikai tu vari atrast un izmantot šo vingrinājumu.

 Šo vingrinājumu varēs pievienot jebkuram treniņam.

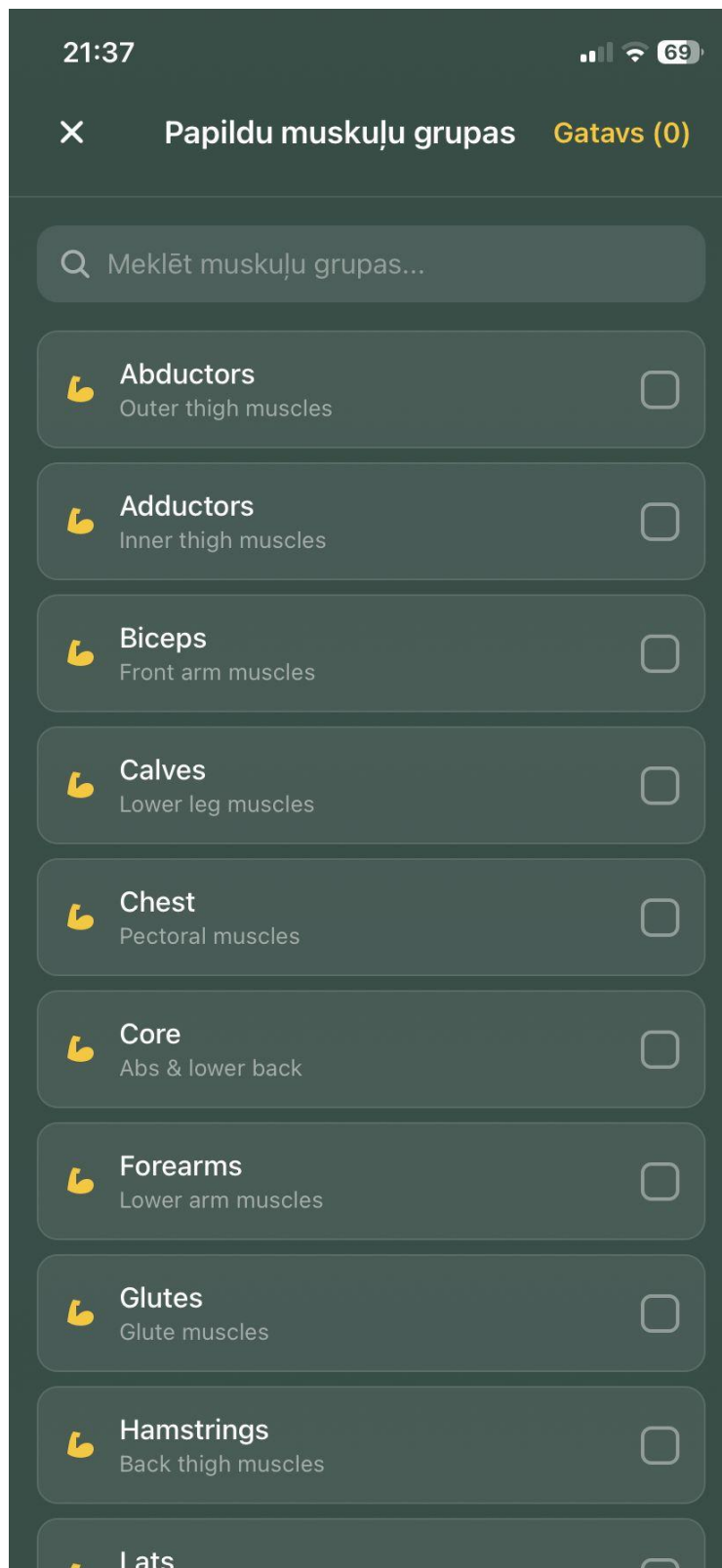
6.13. att. Jauna vingrinājuma izveidošana

Galvenās muskuļu grupas izvēles logā tiek parādīts muskuļu grupu saraksts ar meklēšanas lauku. Lietotājs var izvēlēties tikai vienu galveno muskuļu grupu, kas raksturo vingrinājuma primāro noslodzi. Pēc izvēles veikšanas poga “Izvēlēties” apstiprina izvēli un atgriež lietotāju vingrinājuma formā. (skat. 6.14. att.)



6.14. att. Galvenās muskuļu grupas izvēle

Papildu muskuļu grupu logā lietotājs var izvēlēties vairākas sekundārās muskuļu grupas. Katram ierakstam ir izvēles lauks. Meklēšanas lauks ļauj ātrāk atrast vajadzīgo muskuļu grupu. Poga “Gatavs” saglabā izvēlētās papildu muskuļu grupas un aizver izvēles logu. (skat. 6.15. att.)



6.15. att. Papildu muskuļu grupu izvēle

Treniņa rediģēšanas skatā lietotājs var mainīt treniņa nosaukumu, aprakstu un vingrinājumu sarakstu. Katram vingrinājumam var labot setu atkārtojumu skaitu un svaru, kā arī pievienot jaunus setus. Poga “Saglabāt izmaiņas” saglabā izmaiņas. Šis skats tiek izmantots esoša treniņa plāna labošanai. (skat. 6.16. att.)

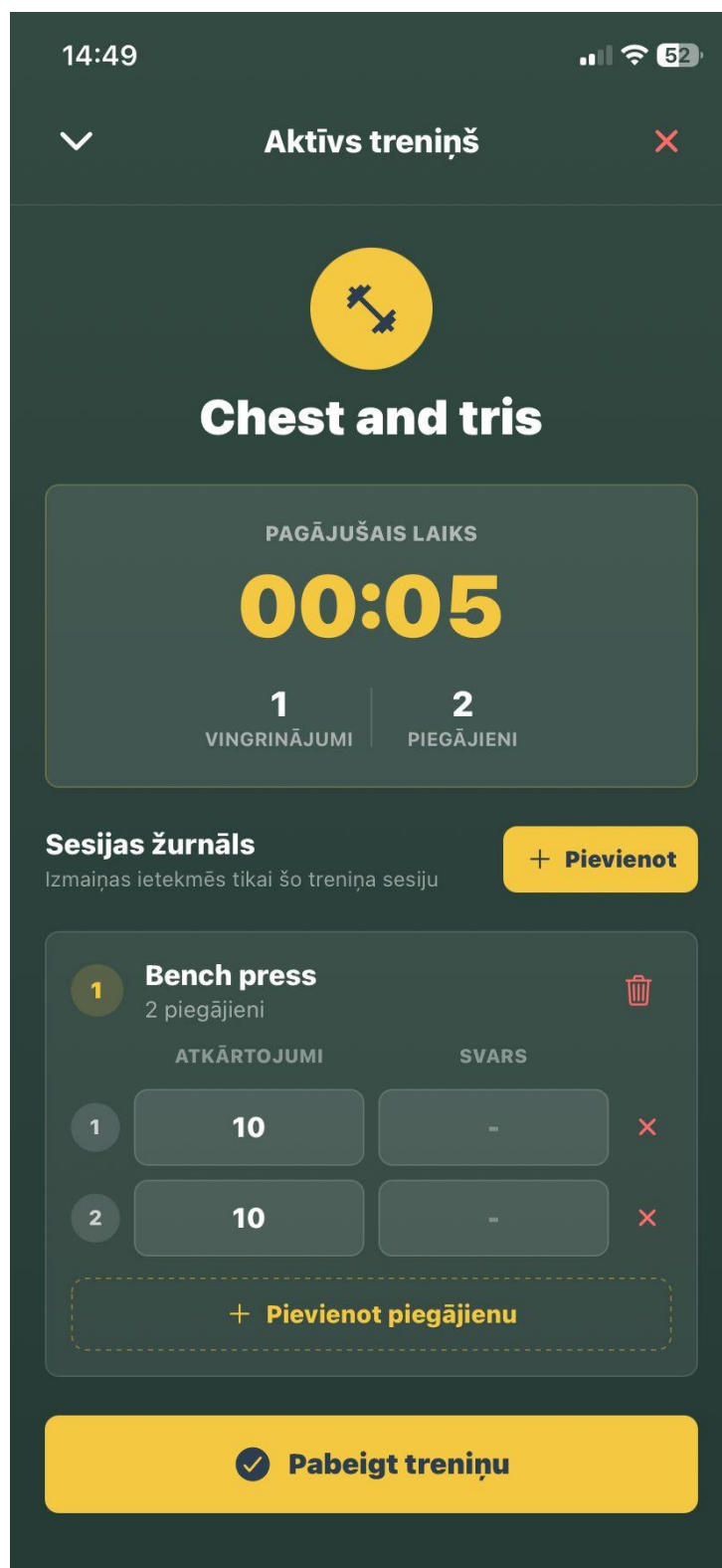
The screenshot shows a mobile app interface for editing a workout. At the top, the status bar displays the time 21:39, signal strength, Wi-Fi, and 69% battery. The app header has a back arrow and the title 'Rediģēt treniņu'. The main form consists of three sections: 1. 'Treniņa nosaukums *' (Workout name) with a text input field containing 'Test'. 2. 'Apraksts (neobligāts)' (Description) with a text input field containing 'Add notes about this workout...'. 3. 'Vingrinājumi' (Exercises) section, which includes a '+ Pievienot' (Add) button and two exercise cards. Each card has a yellow circle with a number (1 and 2), a title ('aaa' and 'Testt'), and a table with columns 'SET S', 'ATKĀRTOJUMI' (Repetitions), and 'SVARS' (Weight). Exercise 1 has two sets, both with 10 repetitions and no weight. Exercise 2 has one set with 10 repetitions and no weight. Each card also has a '+ Pievienot setu' (Add set) button. At the bottom is a large yellow button with a checkmark and the text 'Save Changes'.

SET S	ATKĀRTOJUMI	SVARS
1	10	—
2	10	—

SET S	ATKĀRTOJUMI	SVARS
1	10	—

6.16. att. Treniņa rediģēšana

Aktīvā treniņa skatā lietotājs redz pašreizējās treniņa sesijas nosaukumu, pagājušo laiku, vingrinājumu skaitu un setu skaitu. Sadaļā “Sesijas žurnāls” tiek attēloti treniņā esošie vingrinājumi. Lietotājs var ievadīt faktiskos atkārtojumus un svaru katram setam. Poga “Pievienot” ļauj sesijas laikā pievienot vēl vingrinājumus. (skat. 6.17. att.)



14:49 📶 52

▼ **Aktīvs treniņš** ✕



Chest and tris

PAGĀJUŠAIS LAIKS

00:05

1 | **2**
VINGRINĀJUMI | PIEGĀJIENI

Sesijas žurnāls + Pievienot

Izmaiņas ietekmēs tikai šo treniņa sesiju

1 Bench press 
2 piegājieni

	ATKĀRTOJUMI	SVARS	
1	10	—	✕
2	10	—	✕

+ Pievienot piegājienu

 **Pabeigt treniņu**

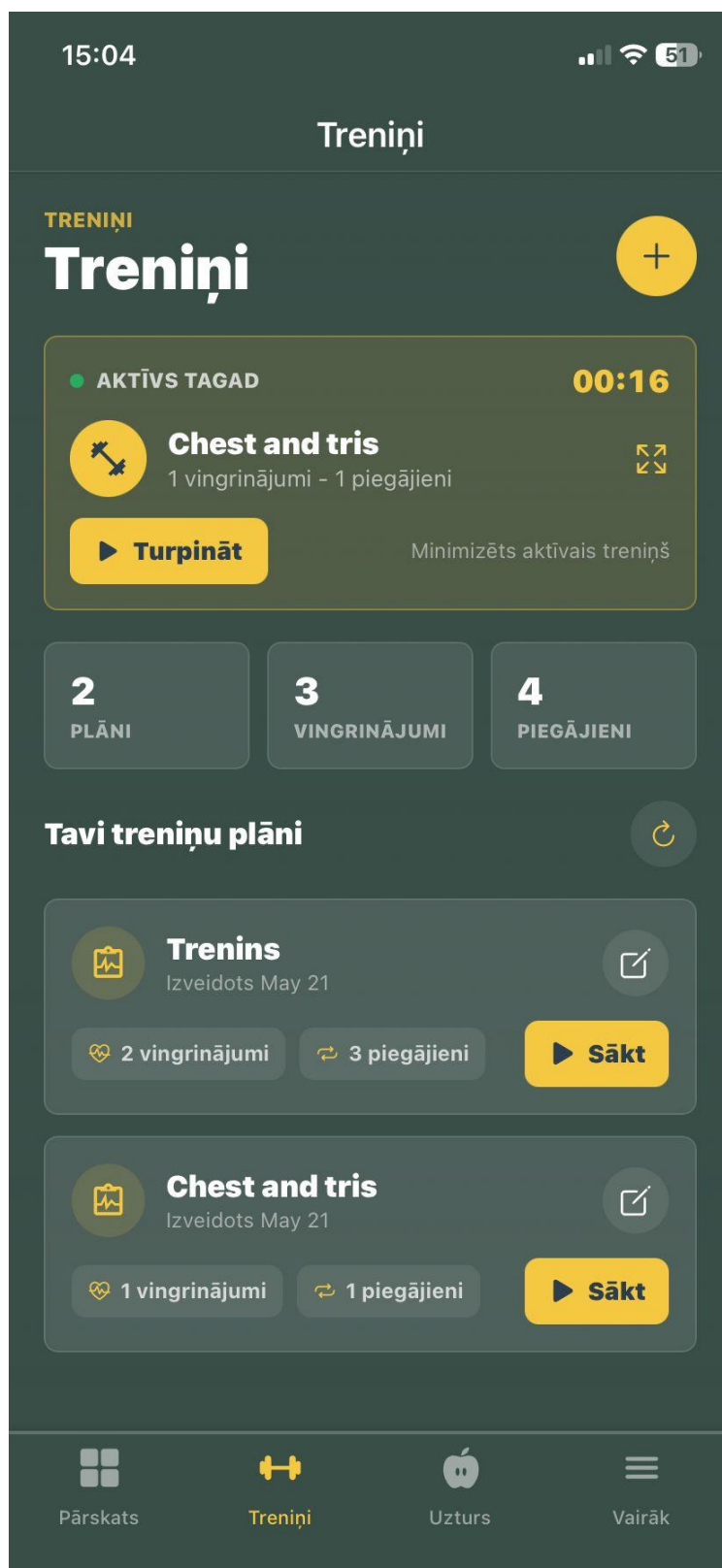
6.17. att. Aktīvs treniņš

Ja lietotājs mēģina atcelt aktīvu treniņu, sistēma parāda apstiprinājuma logu. Tajā ir norādīts, ka darbība atcels aktīvo treniņa sesiju. Lietotājs var izvēlēties “Atcelt treniņu”, lai dzēstu sesiju, vai “Turpināt treniņu”, lai aizvērtu logu un turpinātu darbu. Šis brīdinājums novērš nejaušu datu zaudēšanu. (skat. 6.18. att.)



6.18. att. Treniņa atcelšanas apstiprinājums

Ja lietotājam ir nepabeigta treniņa sesija, treniņu sadaļas augšpusē tiek parādīts aktīvais treniņš. Tajā redzams treniņa nosaukums, pagājušais laiks un poga “Turpināt”. Tas ļauj lietotājam atgriezties aktīvajā sesijā arī pēc pārvietošanās uz citām lietotnes sadaļām. (skat. 6.19. att.)



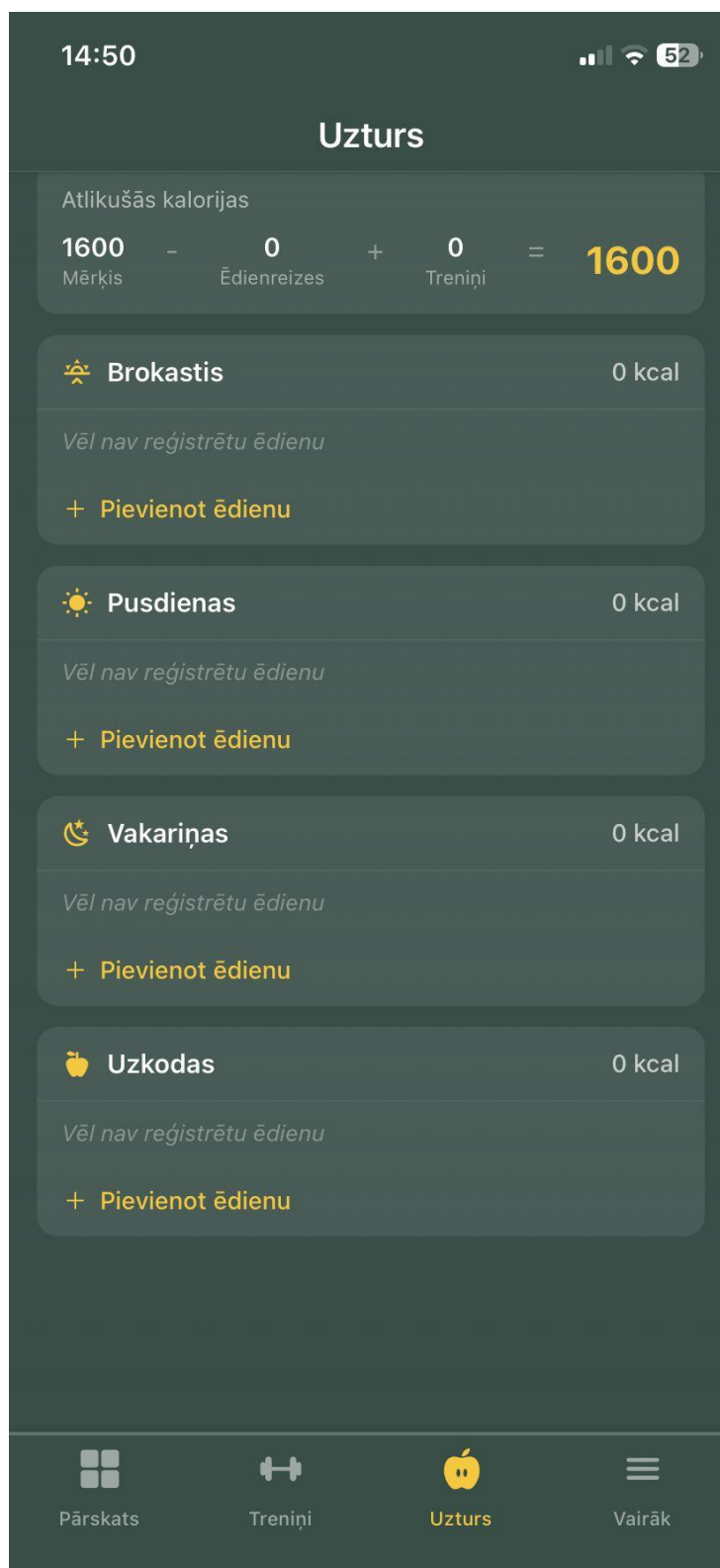
6.19. att. Treniņu sadaļa ar aktīvu sesiju

Uztura sadaļā lietotājs redz izvēlētās dienas uztura pārskatu. Augšpusē ir datuma izvēle un makroelementu sadalījums. Tiek attēlots ūdens patēriņš, ātrās pogas ūdens daudzuma pievienošanai un pielāgots ievades lauks. Zemāk redzamas atlikušās kalorijas, kaloriju mērķis un maltīšu sadaļas. (skat. 6.20. att.)



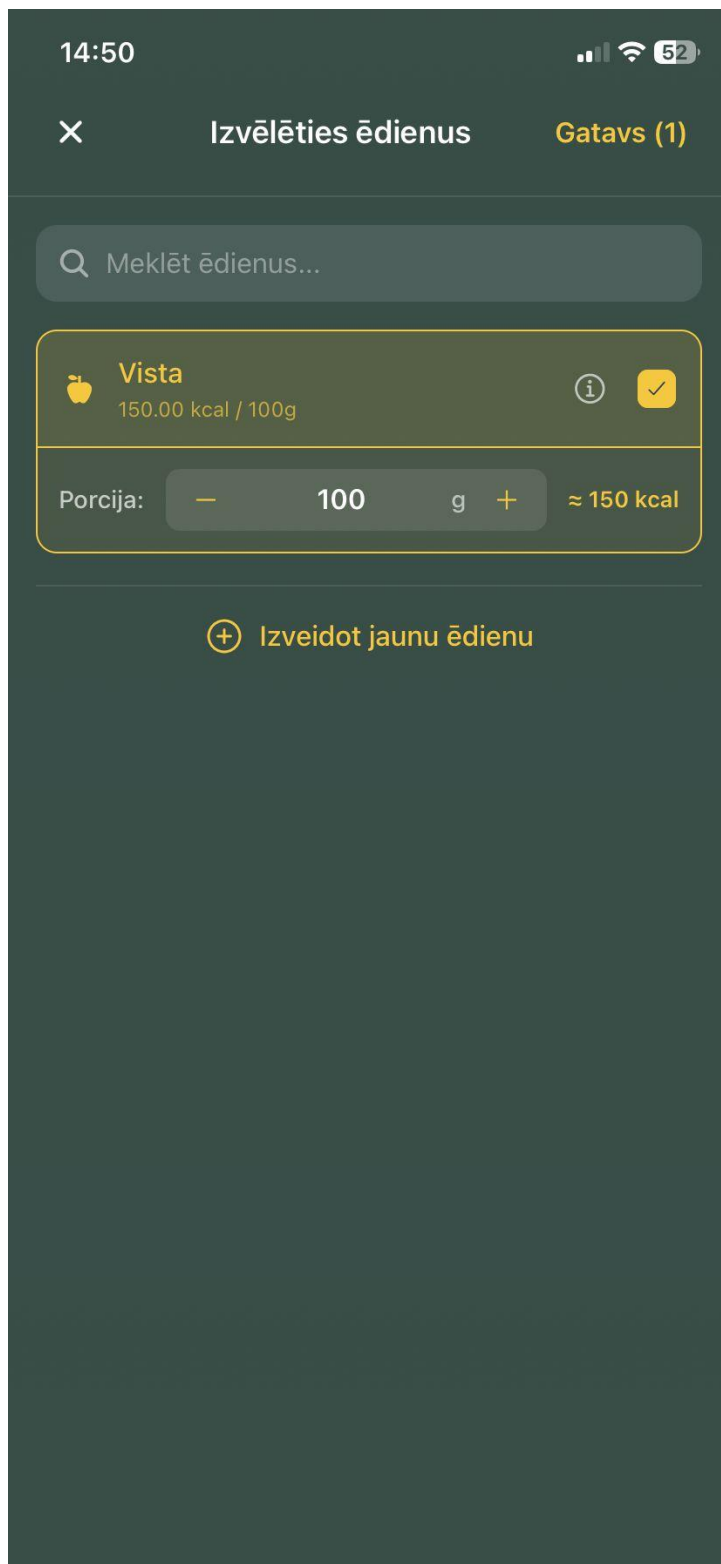
6.20. att. Uztura pārskats

Maltīšu sarakstā ēdieni ir sadalīti brokastīs, pusdienās, vakariņās un uzkodās. Pie katras maltītes tiek parādītas uzņemtās kalorijas un pievienotie ēdieni. Saite “Pievienot ēdienu” ļauj pievienot izvēlētajai maltītei jaunu ēdienu no kataloga. Sistēma automātiski pārreķina kalorijas un makroelementus. (skat. 6.21. att.)



6.21. att. Dienas maltīšu saraksts

Ēdienu izvēles skatā lietotājs var meklēt datubāzē esošos ēdienus un izvēlēties tos pievienošanai maltītei. Katram ēdienam redzams nosaukums un uzturvērtības informācija. Ja nepieciešamais ēdiens vēl nav izveidots, lietotājs var nospiegt “Izveidot jaunu ēdienu” un atvērt ēdiena izveides formu. (skat. 6.22. att.)



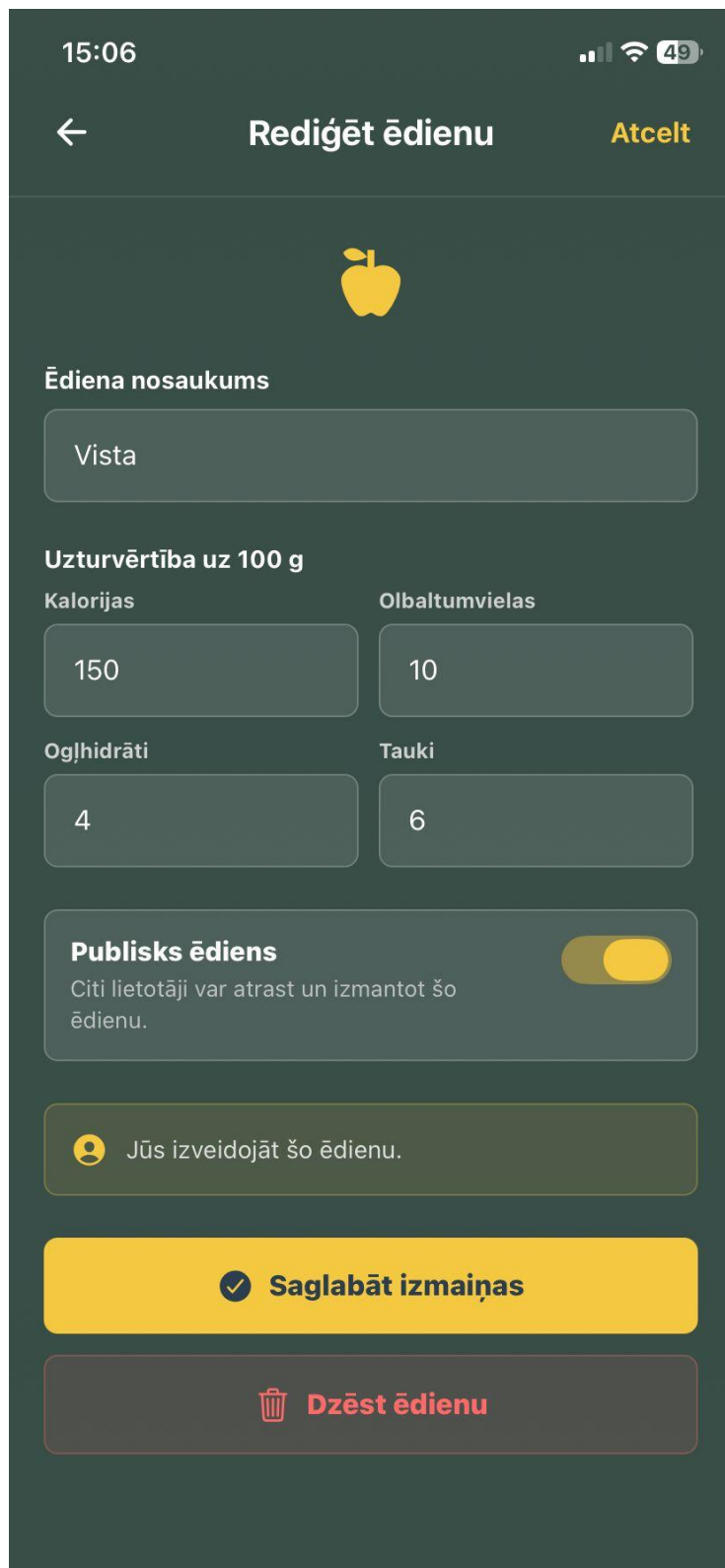
6.22. att. Ēdienu izvēle maltītei

Ēdiena informācijas skatā tiek parādīts ēdiena nosaukums un uzturvērtība uz 100 gramiem: kalorijas, proteīni, ogļhidrāti un tauki. Redzams arī ēdiena privātuma statuss. Augšējā labajā stūrī pieejama poga “Rediģēt”, kas ļauj rediģēt ēdiena datus. (skat. 6.23. att.)



6.23. att. Ēdiena informācija

Ēdiena rediģēšanas skatā lietotājs var mainīt ēdiena nosaukumu, kalorijas, proteīnus, ogļhidrātus un taukus uz 100 gramiem. Slēdzis “Privāts ēdiens” nosaka, vai ēdiens būs pieejams tikai izveidotājam. Poga “Saglabāt izmaiņas” saglabā labojumus, bet poga “Dzēst ēdienu” dzēš ierakstu pēc apstiprinājuma. (skat. 6.24. att.)



15:06

← Rediģēt ēdienu Atcelt



Ēdiena nosaukums

Vista

Uzturvērtība uz 100 g

Kalorijas 150

Olbaltumvielas 10

Ogļhidrāti 4

Tauki 6

Publiskais ēdiens ☒

Citi lietotāji var atrast un izmantot šo ēdienu.

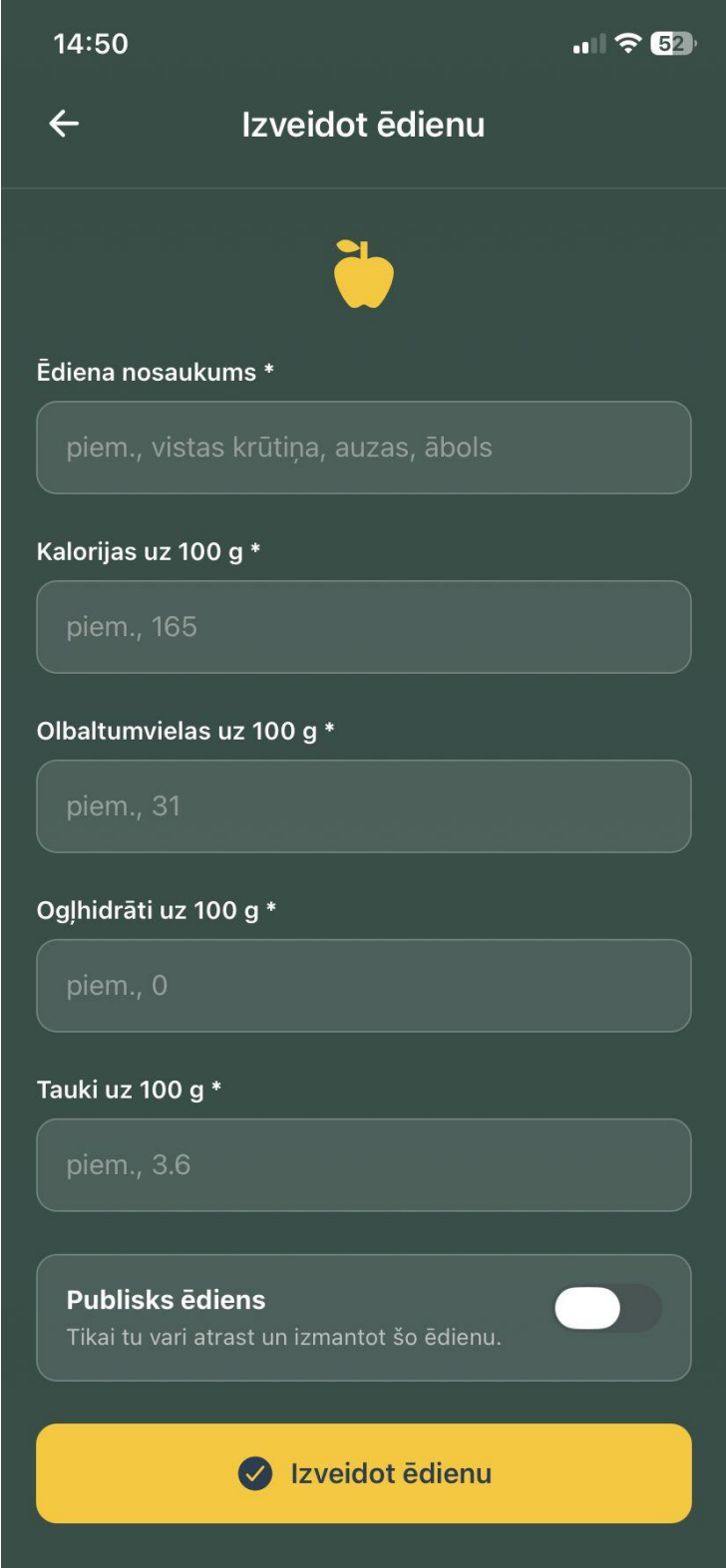
 Jūs izveidojāt šo ēdienu.

✓ Saglabāt izmaiņas

 Dzēst ēdienu

6.24. att. Ēdiena rediģēšana

Jauna ēdiena formā lietotājs ievada ēdiena nosaukumu un uzturvērtības datus uz 100 gramiem: kalorijas, proteīnus, ogļhidrātus un taukus. Slēdzis “Publisks ēdiens” nosaka ēdiena pieejamību citiem lietotājiem. Poga “Izveidot ēdienu” saglabā ēdienu datubāzē, lai to varētu izmantot maltīšu veidošanā. (skat. 6.25. att.)



14:50 📶 52

← **Izveidot ēdienu**



Ēdiena nosaukums *

piem., vistas krūtiņa, auzas, ābols

Kalorijas uz 100 g *

piem., 165

Olbaltumvielas uz 100 g *

piem., 31

Ogļhidrāti uz 100 g *

piem., 0

Tauki uz 100 g *

piem., 3.6

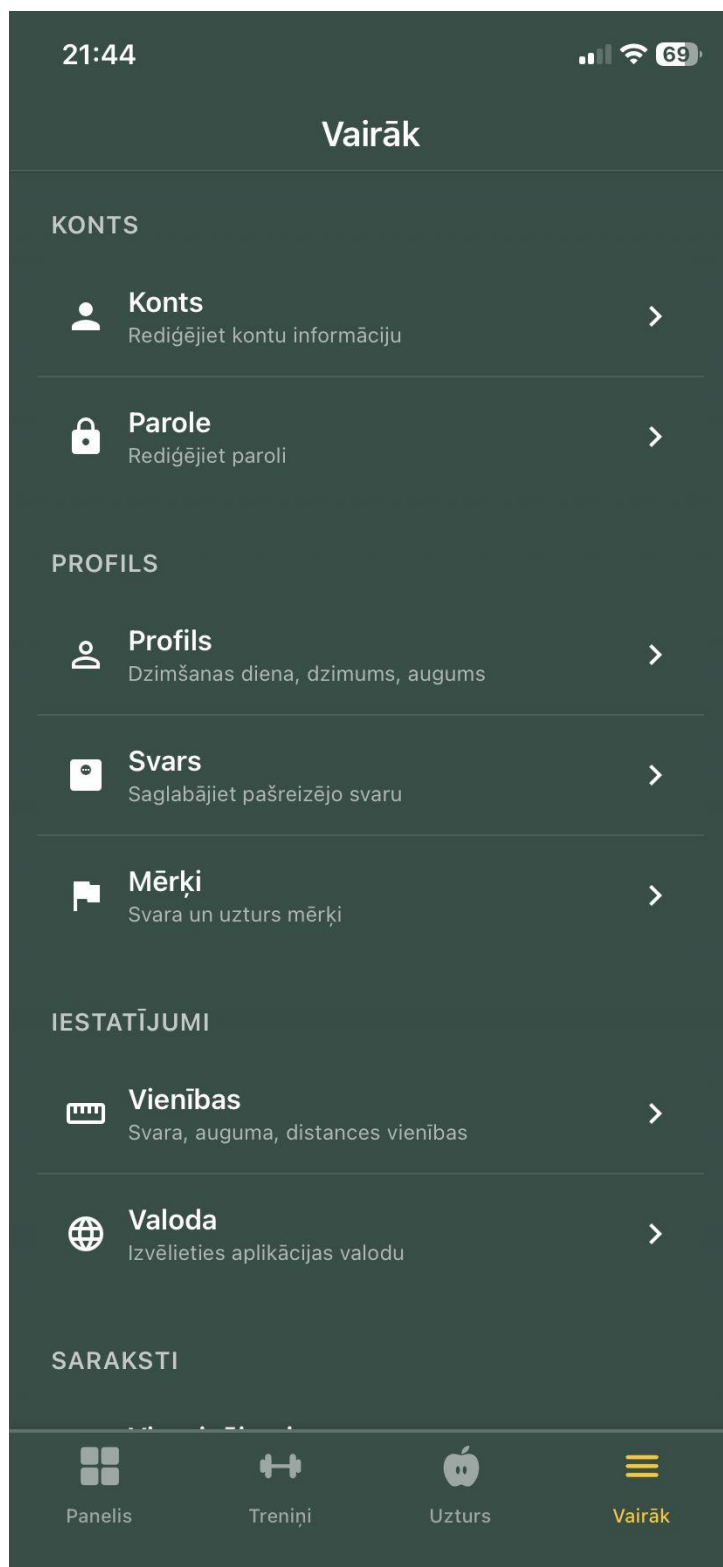
Publisks ēdiens ☐

Tikai tu vari atrast un izmantot šo ēdienu.

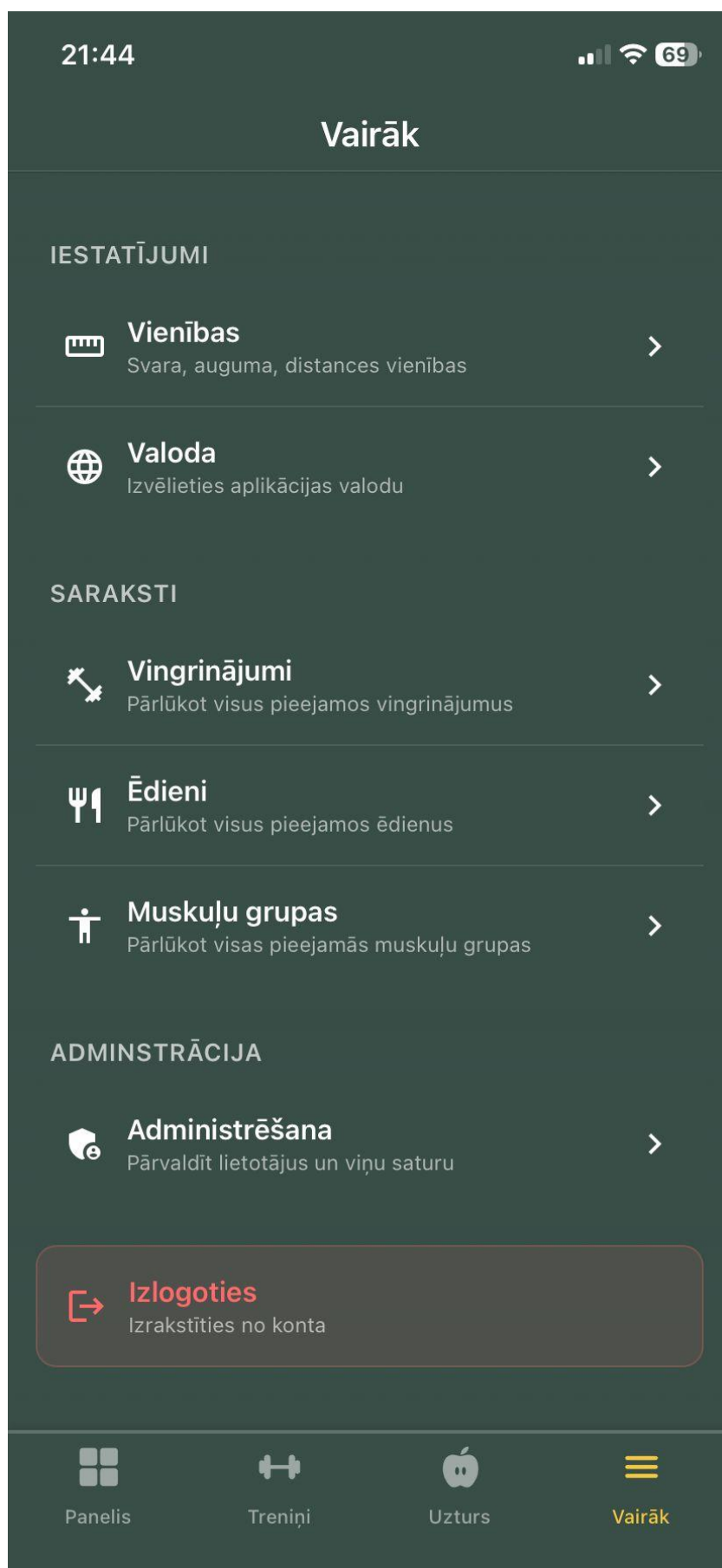
☒ **Izveidot ēdienu**

6.25. att. Jauna ēdiena izveidošana

Papildu izvēlnē ir apkopotas lietotāja konta un iestatījumu sadaļas. Tajā pieejama konta informācijas rediģēšana, paroles maiņa, profila dati, pašreizējā svara ievade, mērķu rediģēšana, mērvienību izvēle, valodas maiņa, vingrinājumu, ēdienu un muskuļu grupu saraksti, administrēšanas lapa un izrakstīšanās poga. Šī sadaļa darbojas kā lietotāja personīgo iestatījumu centrs. (skat. 6.26., 6.27. att.)



6.26. att. Papildu izvēlne



6.27. att. Papildu izvēlne (2)

Konta sadaļā lietotājs var mainīt savu vārdu un e-pasta adresi. Ievades lauki ir aizpildīti ar pašreizējiem datiem. Poga “Saglabāt konta datus” nosūta izmaiņas serverim. Ja ievadīts nederīgs e-pasts vai tukšs vārds, sistēma parāda validācijas kļūdu. (skat. 6.28. att.)

21:44

← Konts

Konta informācija

Vārds

Herberts

E-pasts

h.eisulis@gmail.com

 **Saglabāt konta datus**

 Jūs varat rediģēt savu vārdu un e-pasta adresi

6.28. att. Konta datu rediģēšana

Paroles maiņas skatā lietotājs ievada veco paroli, jauno paroli un jaunās paroles apstiprinājumu. Pie paroles prasībām tiek norādīts minimālais garums un paroļu sakritība. Pēc pogas “Mainīt paroli” nospiešanas sistēma pārbauda veco paroli un saglabā jauno paroli šifrētā veidā. (skat. 6.29. att.)

21:45

← Paroles maiņa

Ievadiet savu vecumu paroli un izvēlieties jaunu paroli

Vecā parole

Ievadiet vecā parole

Jaunā parole

Ievadiet jaunā parole

Apstipriniet jauno paroli

Ievadiet apstipriniet jauno paroli

Paroles prasības:

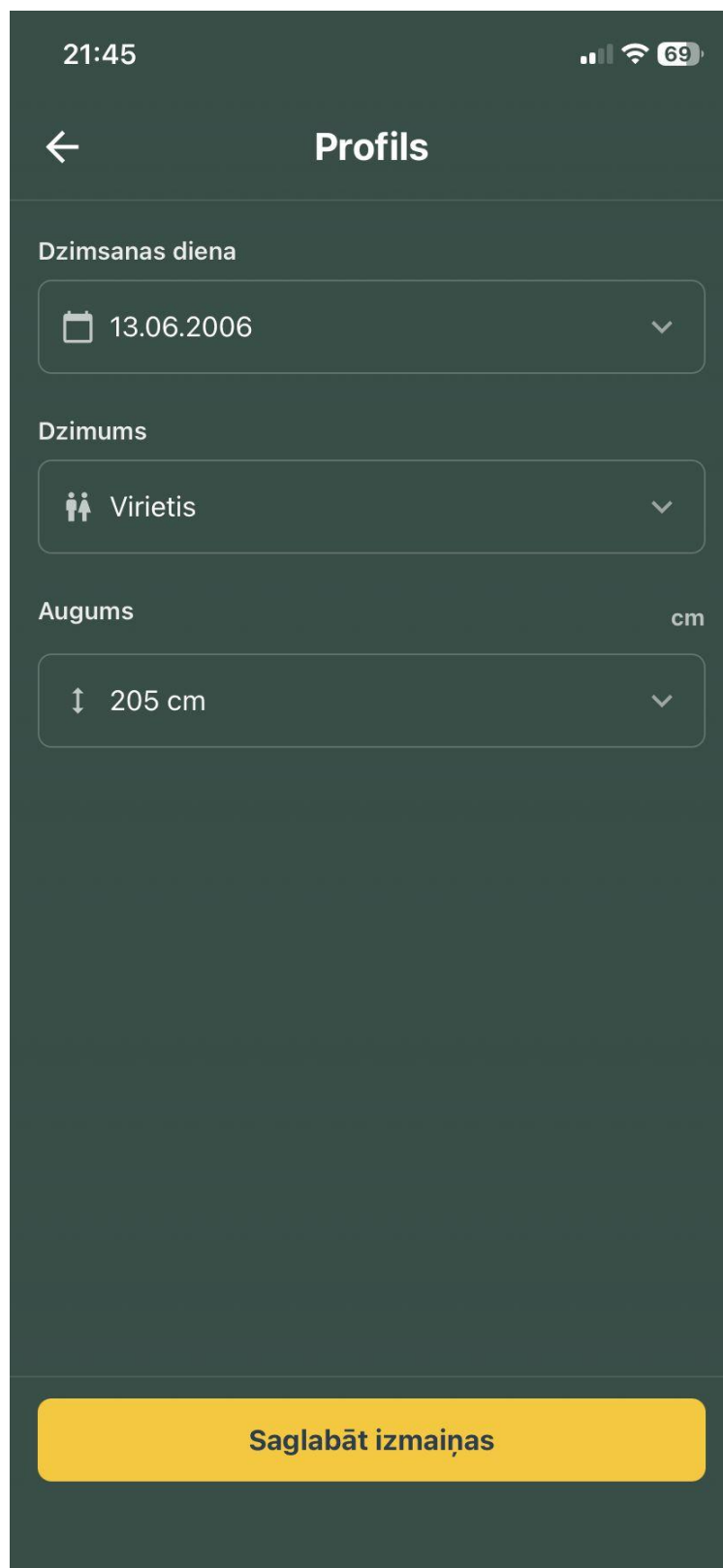
- O Vismaz 8 rakstzīmes (0)
- O Paroles sakrīt

Pēc paroles maiņas jums būs jāpierakstās atkārtoti

🔒 Mainīt paroli

6.29. att. Paroles maiņa

Profila sadaļā lietotājs var mainīt dzimšanas dienu, dzimumu un augumu. Katrs lauks atver izvēles logu, kas palīdz ievadīt datus vienotā formātā. Poga “Saglabāt izmaiņas” saglabā profila datus. Šī informācija tiek izmantota lietotāja personalizācijai. (skat. 6.30. att.)



21:45

← Profils

Dzimšanas diena

13.06.2006

Dzimums

Virietis

Augums cm

205 cm

Saglabāt izmaiņas

6.30. att. Profila rediģēšana

Dzimšanas dienas izvēles logs tiek atvērts no profila sadaļas. Lietotājs izvēlas datumu, izmantojot izvēles riteni. Poga “Gatavs” apstiprina izvēli, bet “Atcelt” aizver logu bez izmaiņu saglabāšanas. Šāds ievades veids samazina kļūdainu datuma formātu risku. (skat. 6.31. att.)

21:45

← Profils

Dzimšanas diena

13.06.2006

Dzimums

Virietis

Augums cm

205 cm

Atcelt	Dzimšanas diena	Gatavs
10	March	2003
11	April	2004
12	May	2005
13	June	2006
14	July	2007
15	August	2008
16	September	2009

6.31. att. Dzimšanas dienas izvēle

Dzimuma izvēles logā lietotājs izvēlas vienu no piedāvātajām vērtībām. Izvēles logs nodrošina vienotu datu ievades formātu un samazina kļūdainu vērtību ievades iespēju. (skat. 6.32. att.)

The screenshot shows a mobile application interface with a dark theme. At the top, the status bar displays the time 21:46, signal strength, Wi-Fi, and battery level at 69%. The app's header is titled 'Profils' with a back arrow on the left. Below the header, there are three sections for profile information: 'Dzimšanas diena' (Date of birth) with a date picker showing '13.06.2006', 'Dzimums' (Gender) with a dropdown menu showing 'Vīrietis' (Male), and 'Augums' (Height) with a dropdown menu showing '205 cm'. At the bottom, there is a navigation bar with three buttons: 'Atcelt' (Cancel), 'Dzimums' (Gender), and 'Gatavs' (Done). Below the navigation bar, a modal dialog is open for selecting a gender. It has a title 'Izvelieties dzimumu' (Select gender) and three options: 'Vīrietis' (Male), 'Sieviete' (Female), and 'Cits' (Other). The 'Vīrietis' option is currently selected and highlighted.

6.32. att. Dzimuma izvēle

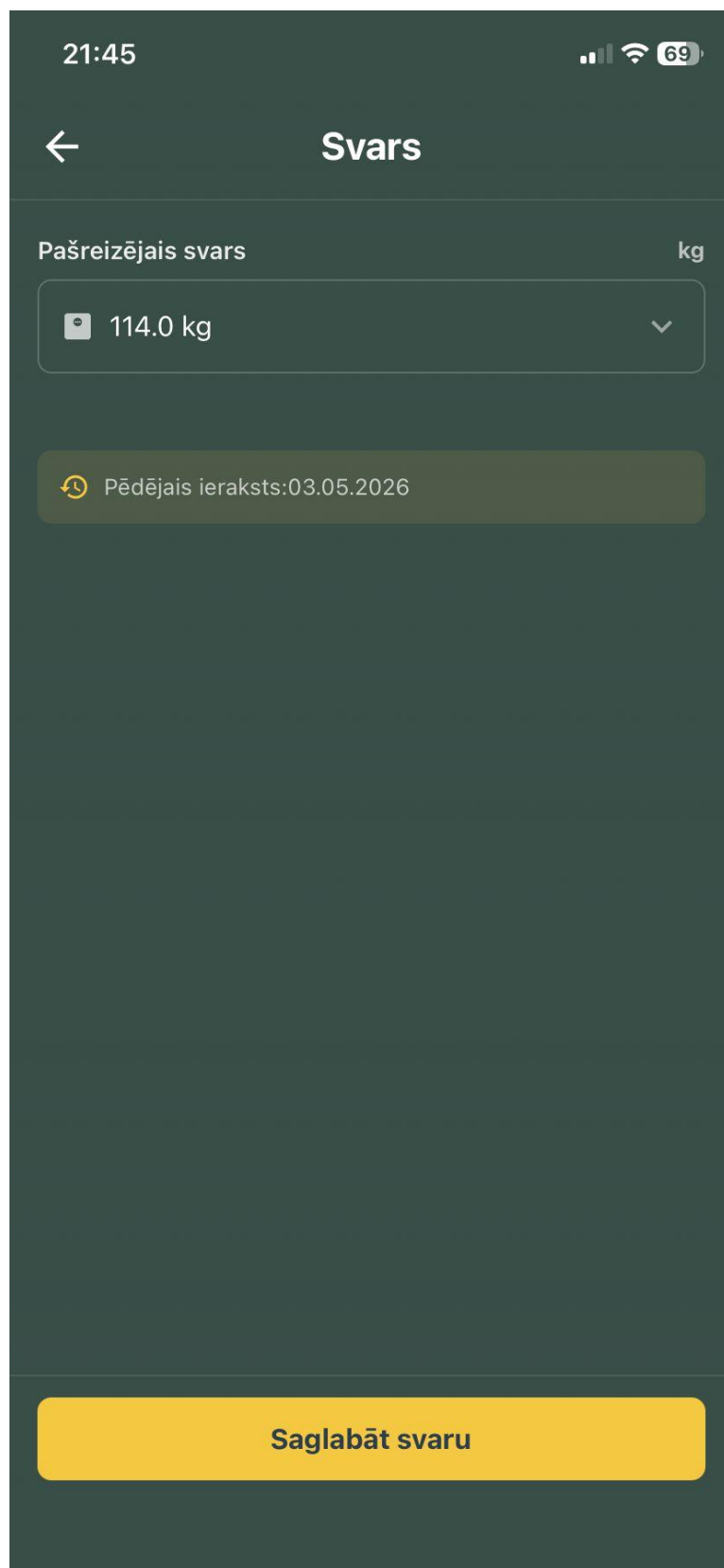
Auguma izvēles logā lietotājs norāda savu augumu. Izvēle tiek veikta ar ritināmu sarakstu, un pēc pogas “Gatavs” nospiešanas vērtība tiek ievietota profila formā. (skat. 6.33. att.)

The screenshot shows a mobile application interface with a dark theme. At the top, the status bar displays the time 21:46, signal strength, Wi-Fi, and battery level at 69%. The app's title bar shows a back arrow and the word "Profils". Below this, there are three input fields: "Dzimsanas diena" (Date of birth) with a calendar icon and the date 13.06.2006; "Dzimums" (Gender) with a person icon and the selection "Virietis" (Male); and "Augums" (Height) with a double-headed arrow icon and the value 205 cm. At the bottom, there is a selection menu with three tabs: "Atcelt" (Cancel), "Augums" (Height), and "Gatavs" (Done). The "Augums" tab is active, showing a list of height options from 202 cm to 208 cm. The option "205 cm" is highlighted with a dark background.

Atcelt	Augums	Gatavs
	202 cm	
	203 cm	
	204 cm	
	205 cm	
	206 cm	
	207 cm	
	208 cm	

6.33. att. Auguma izvēle

Svara ievades skatā lietotājs var saglabāt pašreizējo svaru. Sistēma parāda pēdējā svara ieraksta datumu un ļauj pievienot jaunu ierakstu. Saglabātie dati tiek izmantoti svara progresā grafikā. (skat. 6.34. att.)



21:45

← Svars

Pašreizējais svars kg

114.0 kg

Pēdējais ieraksts: 03.05.2026

Saglabāt svaru

6.34. att. Svara ievade

Svara izvēles logā lietotājs precīzi norāda svaru ar decimālo daļu. Pēc vērtības izvēles lietotājs nospiež “Gatavs”, un izvēlētais svars tiek ievietots svara ievades formā. (skat. 6.35. att.)

The screenshot shows a mobile application interface for selecting weight. At the top, the status bar displays the time 21:45, signal strength, Wi-Fi, and 69% battery. The app title 'Svars' is centered at the top. Below the title, there is a section labeled 'Pašreizējais svars' (Current weight) with a value of 114.0 kg. A dropdown menu is open, showing a list of weight options: 112, 113, 114, 115, 116, and 117. The option 114 is highlighted. To the right of the list, there is a decimal input field with '.0' and a unit 'kg'. Below the list, there is a button labeled 'Gatavs' (Ready). At the bottom, there is a navigation bar with three buttons: 'Atcelt' (Cancel), 'Svars' (Weight), and 'Gatavs' (Ready).

Weight (kg)	Decimal	Unit
112		
113		
114	.0	kg
115	.1	
116	.2	
117		

6.35. att. Svara izvēles logs

Mērķu rediģēšanas skatā lietotājs var norādīt mērķa svaru, kaloriju mērķi, proteīna, tauku un ogļhidrātu mērķus, kā arī aktivitātes mērķus. Šie dati tiek izmantoti galvenajā panelī un uztura pārskatā. (skat. 6.36. att.)

22:25

66

←

Mērķi

Svara mērķis

Mērķa svars

kg

 115.0 kg

▼

Uztura mērķi

Kaloriju mērķis (kcal)

 3500.00

Proteīna mērķis (g)

 200.00

Tauku mērķis (g)

 50.00

Ogļhidrātu mērķis (g)

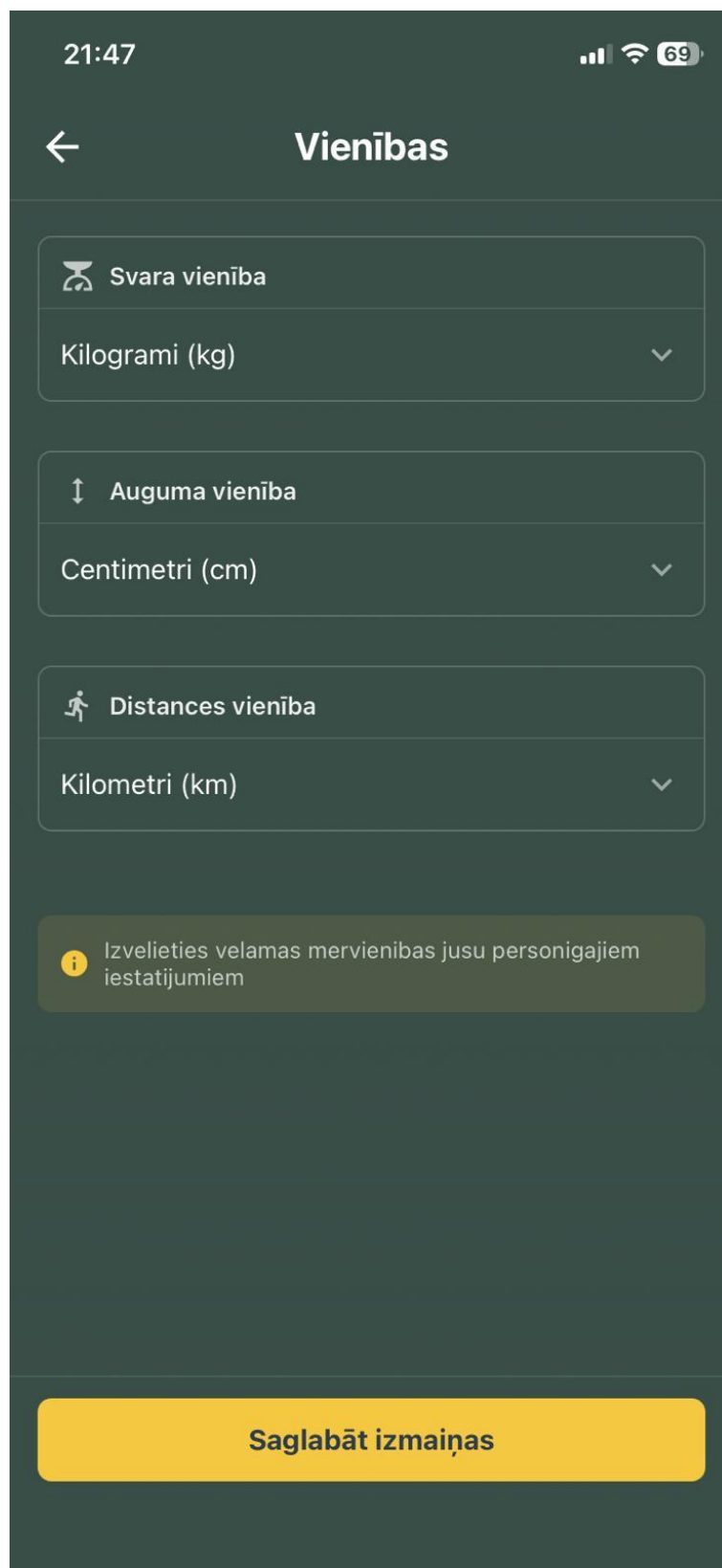
 225.00

Ameklētivitātes mērķi

Saglabāt mērķus

6.36. att. Mērķu rediģēšana

Mērvienību rediģēšanas skatā lietotājs izvēlas svara, auguma un distances mērvienības. Izvēlētās mērvienības tiek izmantotas citās lietotnes sadaļās, piemēram, svara, profila un mērķu pārskatos. (skat. 6.37. att.)



21:47

← Vienības

 Svara vienība

Kilogrami (kg) ▾

 Auguma vienība

Centimetri (cm) ▾

 Distances vienība

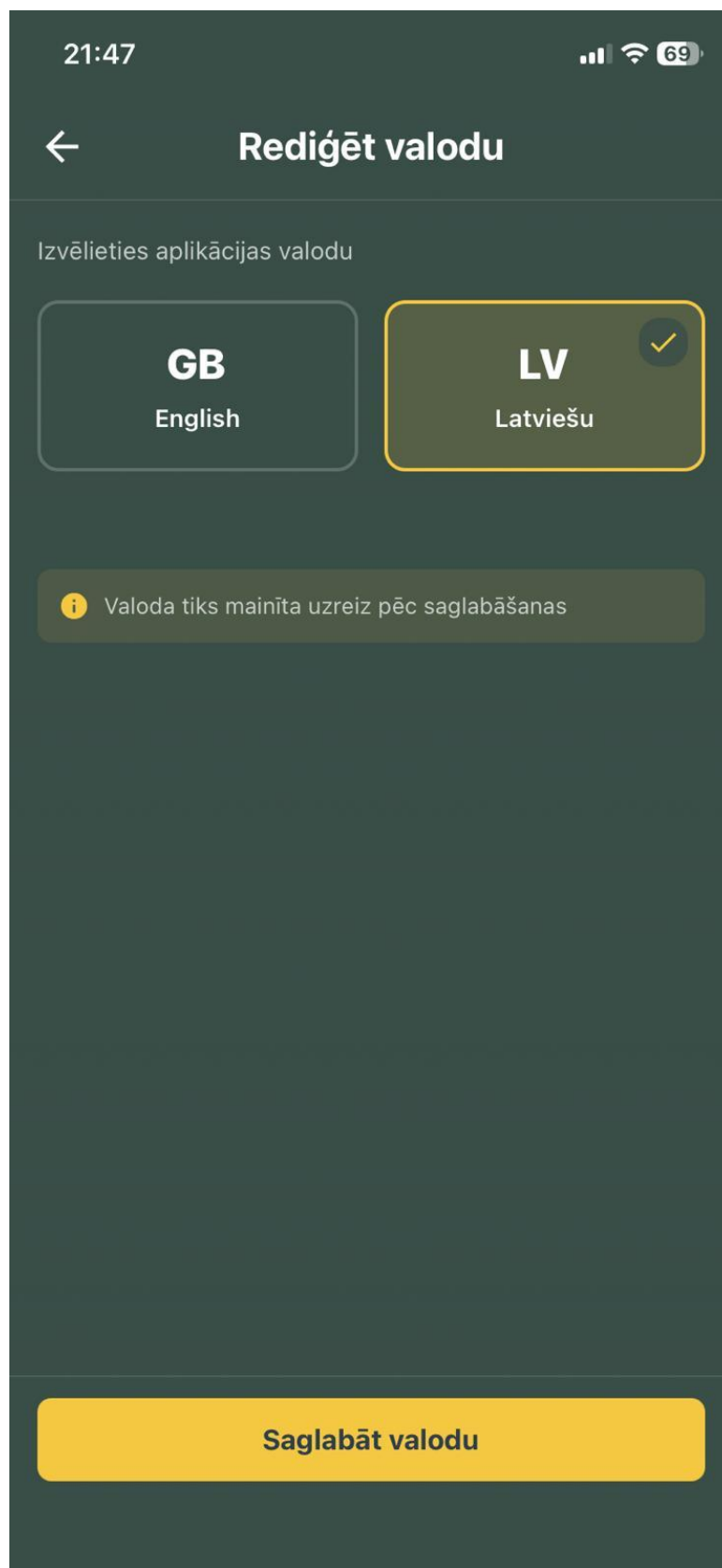
Kilometri (km) ▾

 Izvelieties velamas mervienības jūsu personīgajiem iestatījumiem

Saglabāt izmaiņas

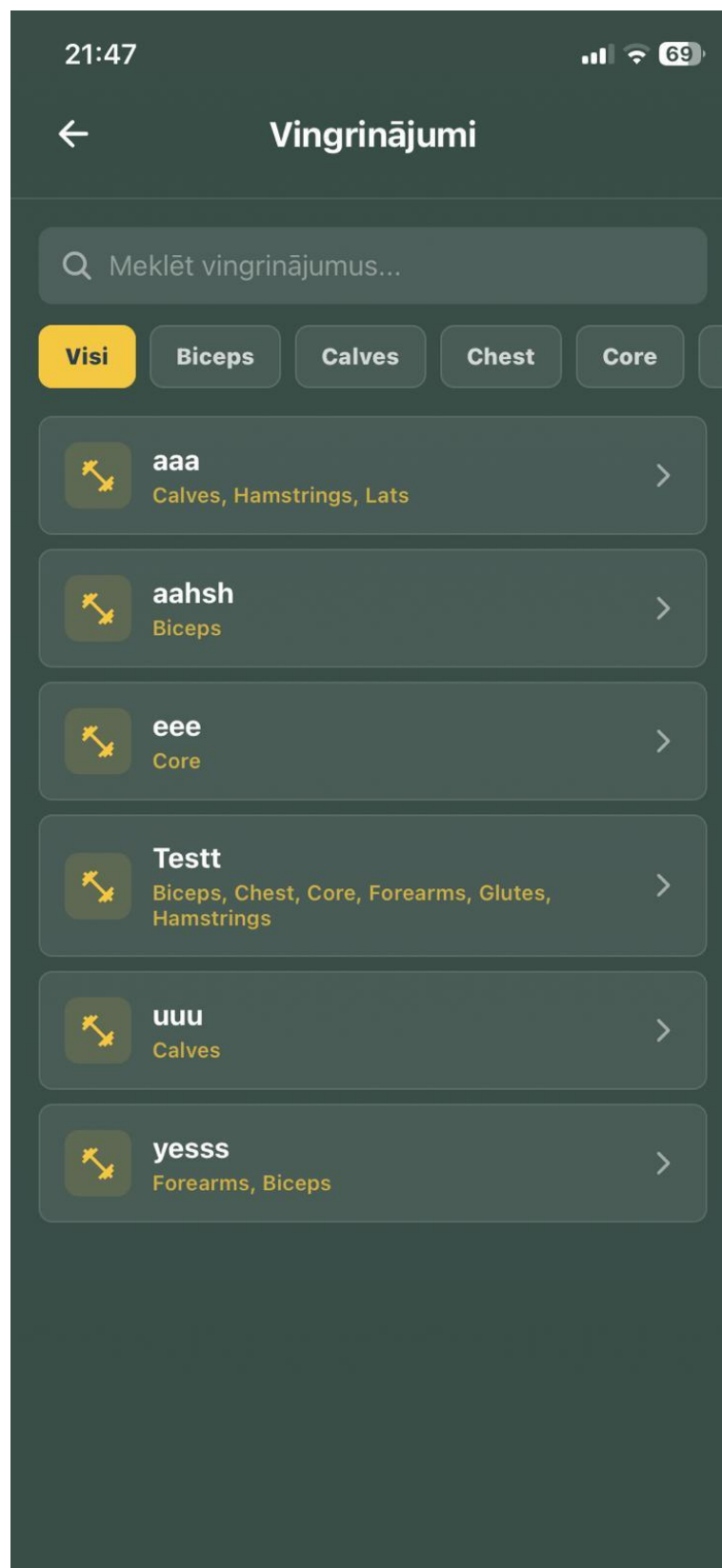
6.37. att. Mērvienību rediģēšana

Valodas rediģēšanas skatā lietotājs izvēlas lietotnes saskarnes valodu. Pieejama latviešu un angļu valoda. Pēc pogas “Saglabāt valodu” nospiešanas sistēma saglabā lietotāja izvēli. (skat. 6.38. att.)



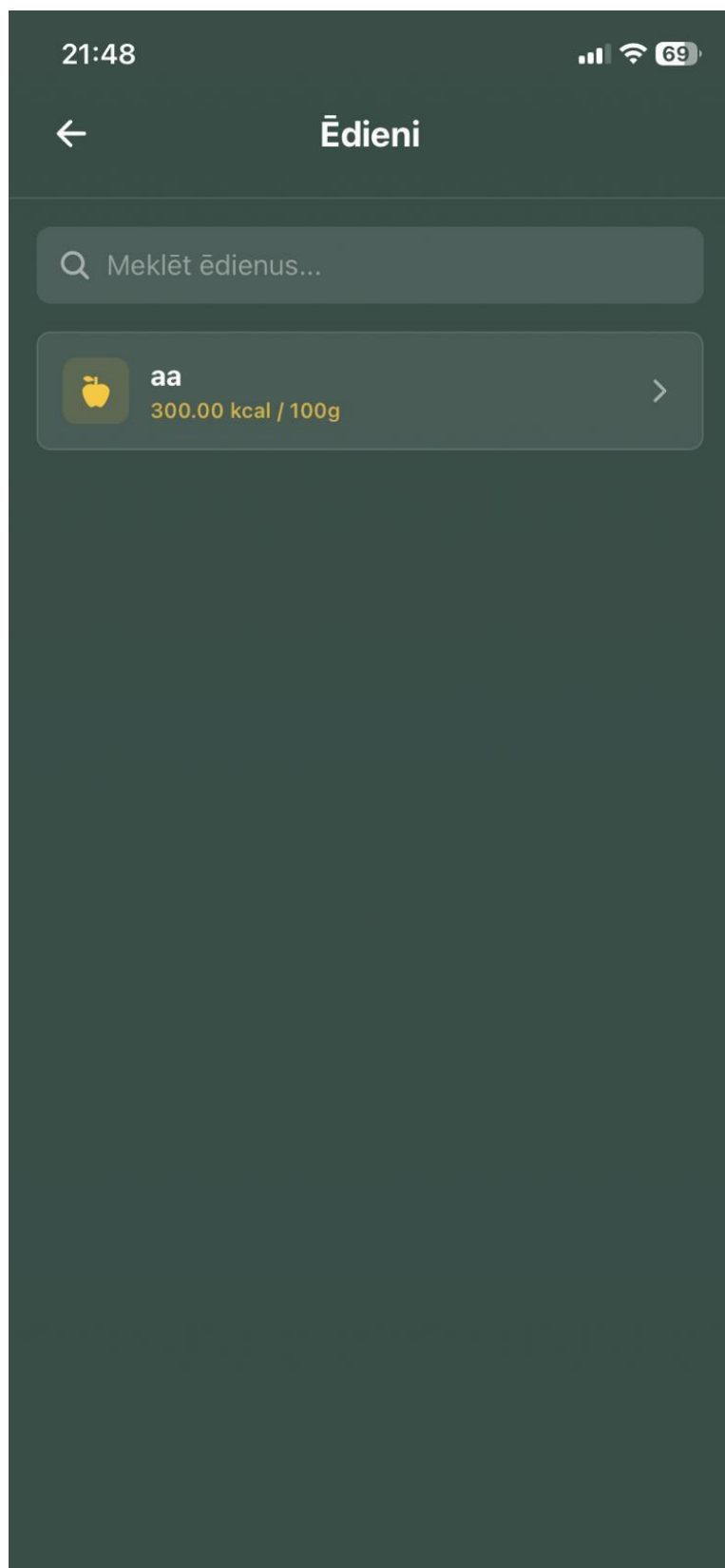
6.38. att. Valodas rediģēšana

Vingrinājumu katalogā lietotājs var pārlūkot sistēmā pieejamos vingrinājumus. Vingrinājumus iespējams meklēt pēc nosaukuma un filtrēt pēc muskuļu grupas. Nospiežot uz vingrinājuma, tiek atvērts tā detalizētais skats. (skat. 6.39. att.)



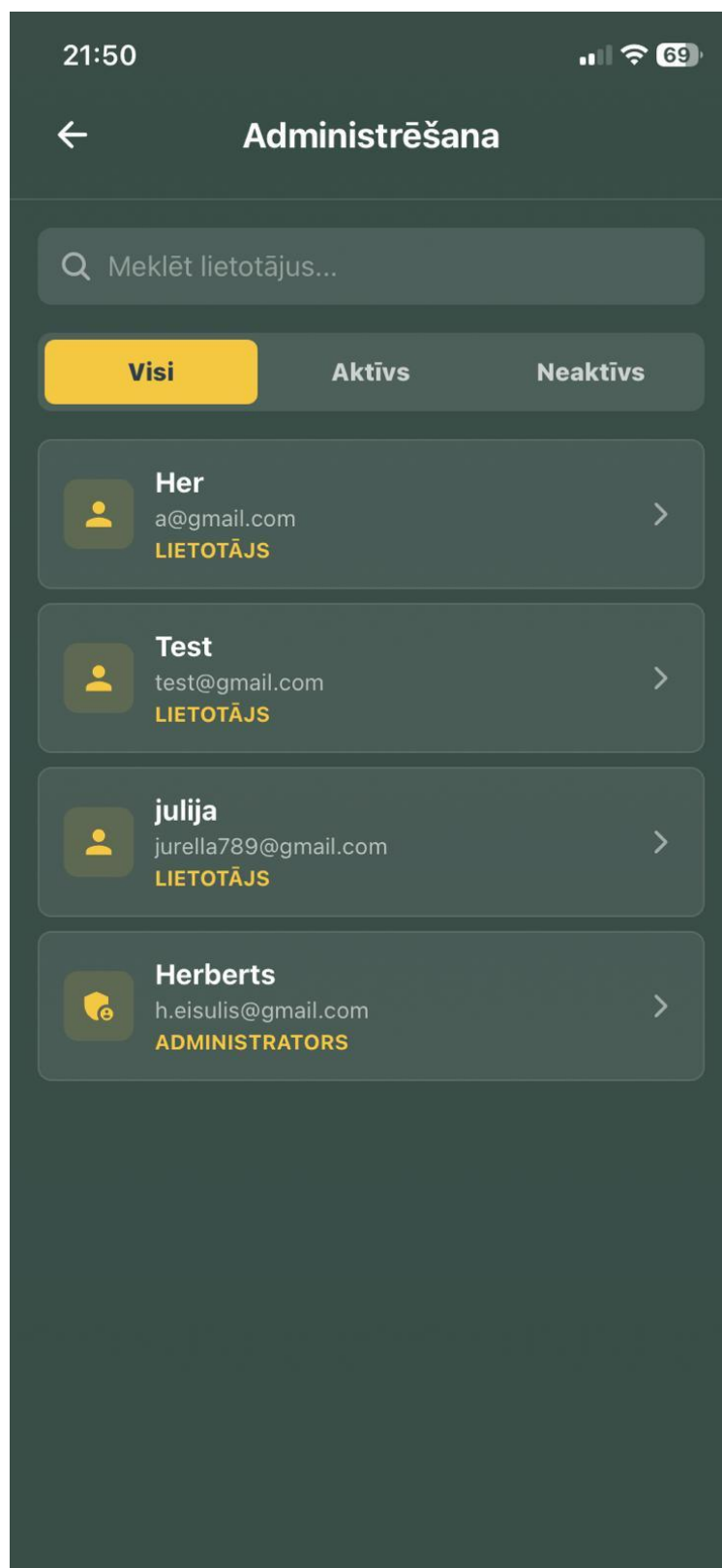
6.39. att. Vingrinājumu katalogs

Ēdienu katalogā lietotājs var apskatīt sistēmā pieejamos ēdienus. Sarakstā redzams ēdiena nosaukums un kaloriju daudzums uz 100 gramiem. Nospiežot uz ēdiena, lietotājs var apskatīt detalizētu uzturvērtības informāciju. (skat. 6.40. att.)



6.40. att. Ēdienu katalogs

Administrēšanas skatā administrators redz sistēmas lietotāju sarakstu. Augšpusē ir meklēšanas lauks un filtri “Visi”, “Aktīvs” un “Neaktīvs”. Katram lietotājam redzams vārds, e-pasts un loma. Nospiežot uz lietotāja ieraksta, tiek atvērts detalizēts pārvaldības skats. (skat. 6.41. att.)



6.41. att. Lietotāju administrēšana

Lietotāja administrēšanas skatā administrators var mainīt lietotāja vārdu, e-pasta adresi, lomu un konta statusu. Pieejamas cilnes “Informācija”, “Ēdieni” un “Vingrinājumi”, kas ļauj apskatīt lietotāja datus un izveidoto saturu. Poga “Saglabāt lietotāju” saglabā izmaiņas, bet poga “Deaktivizēt lietotāja kontu” liedz lietotājam turpmāku pieslēgšanos līdz konta atjaunošanai. (skat. 6.42. att.)

The screenshot shows a mobile application interface for user management. At the top, the status bar displays the time 21:50 and a 69% battery level. The app header includes a back arrow and the title 'Test'. Below the header is a tab bar with three options: 'Informācija' (highlighted in yellow), 'Ēdieni', and 'Vingrinājumi'. The main content area is divided into sections: 'Vārds' with a text field containing 'Test'; 'E-pasts' with a text field containing 'test@gmail.com'; 'Loma' with two buttons, 'Lietotājs' (highlighted in yellow) and 'Administrators'; and a status box showing 'Statuss:Aktīvs', 'Izveidots:3/5/2026', 'Pēdējā pieslēgšanās:--', and 'Dzimums:Male'. At the bottom, there are two large buttons: a yellow 'Saglabāt lietotāju' button with a checkmark icon, and a grey 'Deaktivizēt lietotāja kontu' button with a trash can icon.

6.42. att. Lietotāja administrācijas skats

7. PROGRAMMATŪRAS LIETOJAMĪBAS TESTĒŠANA

Šajā nodaļā tiek aprakstīti sistēmas lietojamības un funkcionalitātes testēšanas piemēri. Testēšanas laikā tika pārbaudītas galvenās sistēmas darbības ar korektiem un nekorektiem ievades datiem. Katra testa gadījumā tika salīdzināts sagaidāmais rezultāts ar faktisko sistēmas darbību.

6.1. tabula

Vingrinājuma izveides testēšanas pārskats

Lietotāja loma	Reģistrēts lietotājs	
Prasības identifikators :	PS 2.2.1	
Prasības apraksts		
Jānodrošina, ka reģistrēts lietotājs var izveidot jaunu vingrinājumu, norādot vingrinājuma nosaukumu, galveno muskuļu grupu, aprakstu un publiskuma statusu.		
<i>Ievaddati/situācijas apraksts</i>	<i>Sagaidāmais rezultāts</i>	<i>Statuss</i>
1. Jaunu vingrinājumu pievieno lietotājs, ievadot nosaukumu, aprakstu un izvēloties galveno muskuļu grupu.	vingrinājums tiek veiksmīgi pievienots	pareizi
2. Jaunu vingrinājumu mēģina pievienot lietotājs, neievadot vingrinājuma nosaukumu.	vingrinājums nav pievienots	pareizi
3. Jaunu vingrinājumu mēģina pievienot lietotājs, neizvēloties galveno muskuļu grupu.	vingrinājums nav pievienots	pareizi
4. Jaunu publisku vingrinājumu pievieno lietotājs, ieslēdzot publiskuma statusu.	vingrinājums tiek pievienots un ir pieejams katalogā	pareizi

6.2. tabula

Treniņa plānu izveides testēšanas pārskats

Lietotāja loma	Reģistrēts lietotājs	
Prasības identifikātors :	PS 2.2.2	
Prasības apraksts		
Jānodrošina, ka reģistrēts lietotājs var izveidot jaunu treniņa plānu, norādot nosaukumu, aprakstu un pievienojot vingrinājumus ar setu datiem.		
<i>Ievaddati/situācijas apraksts</i>	<i>Sagaidāmais rezultāts</i>	<i>Statuss</i>
1. Jaunu treniņa plānu izveido lietotājs, ievadot nosaukumu un pievienojot vingrinājumus.	treniņa plāns tiek veiksmīgi pievienots	pareizi
2. Jaunu treniņa plānu mēģina izveidot lietotājs, neievadot nosaukumu.	treniņa plāns nav pievienots	pareizi
3. Treniņa plānam pievieno vingrinājumu ar atkārtojumu skaitu un svaru.	vingrinājums tiek pievienots treniņa plānam	pareizi

<i>Ievaddati/situācijas apraksts</i>	<i>Sagaidāmais rezultāts</i>	<i>Statuss</i>
4. Esošu treniņa plānu rediģē lietotājs, mainot nosaukumu un setu datus.	treniņa plāna dati tiek atjaunināti	pareizi

6.3. tabula

Ēdiena izveides testēšanas pārskats

Lietotāja loma	Reģistrēts lietotājs	
Prasības identifikātors :	PS 2.2.3	
Prasības apraksts		
Jānodrošina, ka reģistrēts lietotājs var izveidot jaunu ēdienu, norādot ēdiena nosaukumu un uzturvērtības datus uz 100 gramiem.		
<i>Ievaddati/situācijas apraksts</i>	<i>Sagaidāmais rezultāts</i>	<i>Statuss</i>
1. Jaunu ēdienu pievieno lietotājs, ievadot nosaukumu, kalorijas, proteīnus, ogļhidrātus un taukus.	ēdiens tiek veiksmīgi pievienots	pareizi
2. Jaunu ēdienu mēģina pievienot lietotājs, neievadot ēdiena nosaukumu.	ēdiens nav pievienots	pareizi
3. Jaunu ēdienu mēģina pievienot lietotājs, neievadot kaloriju vērtību.	ēdiens nav pievienots	pareizi
4. Esoša ēdiena uzturvērtības datus rediģē lietotājs.	ēdiena dati tiek atjaunināti	pareizi

6.4. tabula

Ēdiena pievienošana maltītei testēšanas pārskats

Lietotāja loma	Reģistrēts lietotājs	
Prasības identifikātors :	PS 2.2.4	
Prasības apraksts		
Jānodrošina, ka reģistrēts lietotājs var pievienot ēdienus izvēlētai maltītei un sistēma aprēķina dienas kaloriju un makroelementu kopsavilkumu.		
<i>Ievaddati/situācijas apraksts</i>	<i>Sagaidāmais rezultāts</i>	<i>Statuss</i>
1. Lietotājs pievieno ēdienu brokastīm.	ēdiens tiek pievienots maltītei	pareizi
2. Lietotājs pievieno vairākus ēdienus pusdienām.	visi izvēlētie ēdieni tiek pievienoti maltītei	pareizi
3. Lietotājs dzēš ēdienu no maltītes.	ēdiens tiek noņemts no maltītes	pareizi
4. Pēc ēdiena pievienošanas sistēma pārrēķina kalorijas un makroelementus.	uztura kopsavilkums tiek atjaunināts	pareizi

NOBEIGUMS

Kvalifikācijas darba izstrādes laikā tika realizēta JustFitness mobilā lietotne, kas apvieno treniņu plānošanu, aktīvu treniņu sesiju reģistrēšanu, uztura un ūdens uzskaiti, svara progresu pārraudzību, profila iestatījumus un administrēšanas iespējas. Lietotnes saskarne izstrādāta ar Expo un React Native, servera daļa veidota ar Node.js un Express, bet dati glabājas MySQL datubāzē. Autentifikācijai izmantoti JWT piekļuves un atjaunošanas tokeni, paroles tiek šifrētas ar bcrypt.

Izstrādātais risinājums šobrīd izmantojams lokālā izstrādes vidē un mobilajā ierīcē vai emulatorā. Turpmāk sistēmu iespējams attīstīt, pievienojot publisku izvietošanu, treniņu un uztura ieteikumu algoritmus, pilnīgāku statistikas analīzi, soļu datu integrāciju ar mobilās ierīces sensoriem un plašāku administratora pārskatu sadaļu.

INFORMĀCIJAS AVOTI

1. JavaScript dokumentācija – <https://developer.mozilla.org/en-US/docs/Web/JavaScript> - (Resurss apskatīts 05.05.2026.)
2. Node.js dokumentācija – <https://nodejs.org/docs/latest/api/> - (Resurss apskatīts 05.05.2026.)
3. React dokumentācija – <https://react.dev/learn> - (Resurss apskatīts 05.05.2026.)
4. React Native dokumentācija – <https://reactnative.dev/docs/getting-started> - (Resurss apskatīts 05.05.2026.)
5. Expo dokumentācija – <https://docs.expo.dev/> - (Resurss apskatīts 05.05.2026.)
6. Expo Router dokumentācija – <https://docs.expo.dev/router/introduction/> - (Resurss apskatīts 05.05.2026.)
7. Express dokumentācija – <https://expressjs.com/en/guide/routing.html> - (Resurss apskatīts 05.05.2026.)
8. MySQL dokumentācija – <https://dev.mysql.com/doc/> - (Resurss apskatīts 05.05.2026.)
9. npm dokumentācija – <https://docs.npmjs.com/cli/> - (Resurss apskatīts 05.05.2026.)
10. mysql2 pakotnes dokumentācija – <https://www.npmjs.com/package/mysql2> - (Resurss apskatīts 05.05.2026.)
11. bcryptjs pakotnes dokumentācija – <https://www.npmjs.com/package/bcryptjs> - (Resurss apskatīts 05.05.2026.)
12. JSON Web Token dokumentācija – <https://jwt.io/introduction/> - (Resurss apskatīts 05.05.2026.)
13. Expo SecureStore dokumentācija – <https://docs.expo.dev/versions/latest/sdk/securestore/> - (Resurss apskatīts 05.05.2026.)
14. i18next dokumentācija – <https://www.i18next.com/> - (Resurss apskatīts 05.05.2026.)

PIELIKUMI

1. pielikums

Programmas pirmteksts

Routes/auth.js

```
const express = require('express');
const {
  AuthService,
  authenticateToken
} = require('../lib/auth');
const UserModel = require('../lib/DbModels/userModel');
const UserSettingsModel = require('../lib/DbModels/userSettingsModel');
const WeightHistoryModel = require('../lib/DbModels/weightHistoryModel');
const router = express.Router();

const toKilograms = (weight, unit = 'kg') => {
  const value = parseFloat(weight);
  if (!Number.isFinite(value)) {
    return null;
  }

  return unit === 'lb' || unit === 'lbs' ? value * 0.45359237 : value;
};

const toCentimeters = (height, unit = 'cm') => {
  const value = parseFloat(height);
  if (!Number.isFinite(value)) {
    return null;
  }

  return unit === 'in' ? value * 2.54 : value;
};

// Reģistrācijas maršruts.
router.post('/register', async (req, res) => {
  try {
    const {
```

```

    email,
    password,
    name,
    gender,
    birthDate,
    height,
    heightUnit,
    weight,
    weightUnit,
    goalWeight,
    calorieGoal,
    proteinGoal,
    fatGoal,
    carbGoal,
    stepGoal,
    language
  } = req.body;

  // Tiek validēti ievades dati.
  const validation = AuthService.validateRegistration(email, password, name);
  if (!validation.isValid) {
    return res.status(400).json({
      success: false,
      message: 'Validation failed',
      errors: validation.errors
    });
  }

  // Tiek pārbaudīts, vai lietotājs jau eksistē.
  const existingUser = await UserModel.findByEmail(email);
  if (existingUser) {
    return res.status(409).json({
      success: false,
      message: 'User with this email already exists'
    });
  }

  // Tiek šifrēta parole.
  const hashedPassword = await AuthService.hashPassword(password);

```

```

// Izveido jaunu lietotāju datubāzē
const newUser = await UserModel.create({
  email,
  password: hashedPassword,
  name
});

// Izveido lietotāja iestatījumus ar mērķiem
const goalWeightKg = goalWeight ? toKilograms(goalWeight, weightUnit) : null;

await UserSettingsModel.create(newUser.id, {
  language: language || 'en',
  gender: gender || null,
  birthDate: birthDate || null,
  height: height ? toCentimeters(height, heightUnit) : null,
  heightUnit: heightUnit || 'cm',
  weightUnit: weightUnit || 'kg',
  distanceUnit: 'km',
  goalWeight: goalWeightKg,
  calorieGoal: calorieGoal ? parseFloat(calorieGoal) : null,
  proteinGoal: proteinGoal ? parseFloat(proteinGoal) : null,
  fatGoal: fatGoal ? parseFloat(fatGoal) : null,
  carbGoal: carbGoal ? parseFloat(carbGoal) : null,
  stepGoal: stepGoal ? parseInt(stepGoal, 10) : 10000
});

if (weight) {
  try {
    const weightKg = toKilograms(weight, weightUnit);

    if (weightKg == null) {
      throw new Error('Invalid weight value');
    }

    await WeightHistoryModel.addEntry(
      newUser.id,
      weightKg
    );
  }
}

```

```

    } catch (weightError) {
        console.warn('Could not create initial weight history entry:',
weightError.message);
    }
}

// Iegūst izveidotos iestatījumus
const userSettings = await UserSettingsModel.findById(newUser.id);

// Tiek ģenerēti autentifikācijas tokeni.
const { accessToken, refreshToken } = await AuthService.generateTokens(newUser.id,
newUser.email);

// Tiek atgriezti lietotāja dati bez paroles.
const userWithoutPassword = AuthService.sanitizeUser(newUser);
userWithoutPassword.settings = userSettings;

res.status(201).json({
    success: true,
    message: 'User registered successfully',
    data: {
        user: userWithoutPassword,
        accessToken,
        refreshToken,
        accessTokenExpiresIn: '15m',
        refreshTokenExpiresIn: '30d'
    }
});

} catch (error) {
    console.error('Registration error:', error);
    res.status(500).json({
        success: false,
        message: 'Internal server error during registration'
    });
}
});

// Pieslēgšanās maršruts.
router.post('/login', async (req, res) => {

```

```

try {
  const { email, password } = req.body;

  // Tiek validēti ievades dati.
  const validation = AuthService.validateLogin(email, password);
  if (!validation.isValid) {
    return res.status(400).json({
      success: false,
      message: 'Validation failed',
      errors: validation.errors
    });
  }

  // Lietotājs tiek sameklēts datubāzē.
  const user = await UserModel.findByEmail(email);
  if (!user) {
    return res.status(401).json({
      success: false,
      message: 'Invalid email or password'
    });
  }

  // Tiek pārbaudīta parole.
  const isValidPassword = await AuthService.comparePassword(password, user.password);
  if (!isValidPassword) {
    return res.status(401).json({
      success: false,
      message: 'Invalid email or password'
    });
  }

  // Datubāzē tiek atjaunināts pēdējās pieslēgšanās laiks.
  await UserModel.updateLastLogin(user.id);

  // Tiek ģenerēti autentifikācijas tokeni.
  const { accessToken, refreshToken } = await AuthService.generateTokens(user.id,
    user.email);

  // Tiek atgriezti lietotāja dati bez paroles.

```

```

const userWithoutPassword = AuthService.sanitizeUser(user);
userWithoutPassword.settings = await UserSettingsModel.findById(user.id);

res.json({
  success: true,
  message: 'Login successful',
  data: {
    user: userWithoutPassword,
    accessToken,
    refreshToken,
    accessTokenExpiresIn: '15m',
    refreshTokenExpiresIn: '30d'
  }
});

} catch (error) {
  console.error('Login error:', error);
  res.status(500).json({
    success: false,
    message: 'Internal server error during login'
  });
}
});

// Tokena atjaunošanas maršruts, kuram nav nepieciešams piekļuves tokens.
router.post('/refresh', async (req, res) => {
  try {
    const { refreshToken } = req.body;

    if (!refreshToken) {
      return res.status(401).json({
        success: false,
        message: 'Refresh token required'
      });
    }

    // Tiek pārbaudīts atjaunošanas tokens.
    const decoded = await AuthService.verifyRefreshToken(refreshToken);

```



```

// Lietotājs tiek sameklēts datubāzē.
const user = await UserModel.findById(decoded.userId);
if (!user) {
  return res.status(404).json({
    success: false,
    message: 'User not found'
  });
}

// Atjaunošanas tokens tiek nomainīts, lai sesija varētu turpināties droši.
const newRefreshToken = await AuthService.rotateRefreshToken(refreshToken, user.id,
user.email);
const newAccessToken = AuthService.generateAccessToken(user.id, user.email);

res.json({
  success: true,
  message: 'Access token refreshed successfully',
  data: {
    accessToken: newAccessToken,
    refreshToken: newRefreshToken,
    user: {
      ...AuthService.sanitizeUser(user),
      settings: await UserSettingsModel.findById(user.id)
    },
    accessTokenExpiresIn: '15m',
    refreshTokenExpiresIn: '30d'
  }
});

} catch (error) {
  console.error('Token refresh error:', error);
  res.status(403).json({
    success: false,
    message: error.message || 'Invalid refresh token'
  });
}
});

// Izrakstīšanās maršruts dzēš konkrētās ierīces atjaunošanas tokenu.

```

```

router.post('/logout', async (req, res) => {
  try {
    const { refreshToken } = req.body;

    // Tiek dzēsts atjaunošanas tokens. if provided
    if (refreshToken) {
      const removed = await AuthService.removeRefreshToken(refreshToken);
      if (removed) {
        console.log('Refresh token successfully removed');
      }
    }

    res.json({
      success: true,
      message: 'Logged out successfully'
    });

  } catch (error) {
    console.error('Logout error:', error);
    res.status(500).json({
      success: false,
      message: 'Internal server error during logout'
    });
  }
});

// Izrakstīšanās no visām ierīcēm.
router.post('/logout-all', authenticateToken, async (req, res) => {
  try {
    // Tiek dzēsti visi šī lietotāja atjaunošanas tokeni.
    const removedCount = await AuthService.removeAllUserRefreshTokens(req.user.userId);

    res.json({
      success: true,
      message: `Logged out from ${removedCount} devices successfully`
    });

  } catch (error) {
    console.error('Logout all error:', error);
  }
});

```

```
    res.status(500).json({
      success: false,
      message: 'Internal server error during logout'
    });
  }
});

module.exports = router;
```

routes/user.js

```
const express = require('express');
const {
  AuthService,
  authenticateToken
} = require('../lib/auth');
const UserModel = require('../lib/DbModels/userModel');
const UserSettingsModel = require('../lib/DbModels/userSettingsModel');
const WeightHistoryModel = require('../lib/DbModels/weightHistoryModel');
const router = express.Router();

// Konvertē mārciņas uz kilogramiem
const toKilograms = (weight, unit = 'kg') => {
  const value = parseFloat(weight);
  if (!Number.isFinite(value)) {
    return null;
  }
  return unit === 'lb' || unit === 'lbs' ? value * 0.45359237 : value;
};

// Konvertē augumu uz cm
const toCentimeters = (height, unit = 'cm') => {
  const value = parseFloat(height);
  if (!Number.isFinite(value)) {
    return null;
  }
  // 1 colla (inch) = 2.54 cm
  return unit === 'in' ? Math.round(value * 2.54) : value;
};

// Konvertē augumu no cm uz lietotāja vienībām
const convertHeightFromCm = (heightCm, unit = 'cm') => {
  const value = parseFloat(heightCm);
  if (!Number.isFinite(value)) {
    return null;
  }
  // 1 colla (inch) = 2.54 cm
  return unit === 'in' ? value / 2.54 : value;
};
```

```

};

const convertWeightFromKg = (weightKg, unit = 'kg') => {
  const value = parseFloat(weightKg);
  if (!Number.isFinite(value)) {
    return null;
  }
  return unit === 'lb' || unit === 'lbs' ? value / 0.45359237 : value;
};

const hasOwn = (object, key) => Object.prototype.hasOwnProperty.call(object, key);

// Iegūst lietotāja datus - GET /api/user/
router.get("/", authenticateToken, async (req, res) => {
  try {
    const userId = req.user.userId;

    const user = await UserModel.findById(userId);
    if (!user) {
      return res.status(404).json({
        success: false,
        message: 'Lietotājs nav atrasts'
      });
    }

    res.json({
      success: true,
      data: user
    });
  } catch (error) {
    console.error('Kļūda iegūstot lietotāja datus:', error);
    res.status(500).json({
      success: false,
      message: 'Iekšējā servera kļūda'
    });
  }
});

// Atjaunina lietotāja datus (vārds, e-pasts) - PUT /api/user/

```

```

router.put("/", authenticateToken, async (req, res) => {
  try {
    const userId = req.user.userId;
    const { name, email } = req.body;

    // Tiek veikta validācija
    if (!name || !email) {
      return res.status(400).json({
        success: false,
        message: 'Vārds un e-pasts ir obligāti'
      });
    }

    // Pārbauda, vai e-pasts jau ir izmantots (ja e-pasts mainīts)
    const currentUser = await UserModel.findById(userId);
    if (email !== currentUser.email) {
      const existingUser = await UserModel.findByEmail(email);
      if (existingUser) {
        return res.status(409).json({
          success: false,
          message: 'Lietotājs ar šo e-pasta adresi jau eksistē'
        });
      }
    }
  }

  // Atjaunina lietotāju
  const updatedUser = await UserModel.updateProfile(userId, { name, email });

  res.json({
    success: true,
    message: 'Lietotāja dati atjaunināti',
    data: updatedUser
  });
} catch (error) {
  console.error('Kļūda atjauninot lietotāja datus:', error);
  res.status(500).json({
    success: false,
    message: 'Iekšējā servera kļūda'
  });
}

```

```

    }
  });

// Mainīt paroli - POST /api/user/change-password
router.post("/change-password", authenticateToken, async (req, res) => {
  try {
    const userId = req.user.userId;
    const { oldPassword, newPassword, confirmPassword } = req.body;

    // Tiek veikta validācija
    if (!oldPassword || !newPassword || !confirmPassword) {
      return res.status(400).json({
        success: false,
        message: 'Visas paroles ir obligātas'
      });
    }

    // Pārbauda, vai jaunā parole un apstiprinājums sakrīt
    if (newPassword !== confirmPassword) {
      return res.status(400).json({
        success: false,
        message: 'Jaunā parole un apstiprinājums nesakrīt'
      });
    }

    // Pārbauda, vai jaunā parole ir vismaz 8 rakstzīmes gara
    if (newPassword.length < 8) {
      return res.status(400).json({
        success: false,
        message: 'Jaunā parole ir jābūt vismaz 8 rakstzīmes garai'
      });
    }

    // Iegūst lietotāja datus ar paroli
    const user = await UserModel.findById(userId, true);
    if (!user) {
      return res.status(404).json({
        success: false,
        message: 'Lietotājs nav atrasts'
      });
    }
  }
});

```

```

    });
  }

  // Tiek pārbaudīta vecā parole
  const isValidPassword = await AuthService.comparePassword(oldPassword, user.password);
  if (!isValidPassword) {
    return res.status(401).json({
      success: false,
      message: 'Vecā parole ir nepareiza'
    });
  }

  // Jaunā parole nedrīkst būt tāda pati kā vecā
  const isSamePassword = await AuthService.comparePassword(newPassword, user.password);
  if (isSamePassword) {
    return res.status(400).json({
      success: false,
      message: 'Jaunā parole nedrīkst būt tāda pati kā vecā'
    });
  }

  // Tiek šifrēta jaunā parole
  const hashedPassword = await AuthService.hashPassword(newPassword);

  // Atjaunina paroli datubāzē
  const updatedUser = await UserModel.updatePassword(userId, hashedPassword);

  res.json({
    success: true,
    message: 'Parole veiksmīgi mainīta',
    data: updatedUser
  });
} catch (error) {
  console.error('Kļūda mainot paroli:', error);
  res.status(500).json({
    success: false,
    message: 'Iekšējā servera kļūda'
  });
}

```



```

});

// Iegūst lietotāja iestatījumus - GET /api/user/settings
router.get("/settings", authenticateToken, async (req, res) => {
  try {
    const userId = req.user.userId;

    const settings = await UserSettingsModel.findByUserId(userId);
    if (!settings) {
      return res.status(404).json({
        success: false,
        message: 'Lietotāja iestatījumi nav atrasti'
      });
    }

    // Konvertē augumu no cm uz lietotāja vienībām
    const displayHeight = settings.height ? convertHeightFromCm(settings.height,
settings.height_unit) : null;

    res.json({
      success: true,
      data: {
        ...settings,
        height: displayHeight
      }
    });
  } catch (error) {
    console.error('Kļūda iegūstot lietotāja iestatījumus:', error);
    res.status(500).json({
      success: false,
      message: 'Iekšējā servera kļūda'
    });
  }
});

// Atjaunina lietotāja iestatījumus - PUT /api/user/settings
router.put("/settings", authenticateToken, async (req, res) => {
  try {
    const userId = req.user.userId;

```

```

const {
  language,
  gender,
  birthDate,
  height,
  heightUnit,
  weightUnit,
  distanceUnit,
  goalWeight,
  calorieGoal,
  proteinGoal,
  fatGoal,
  carbGoal,
  stepGoal
} = req.body;

const settings = await UserSettingsModel.findByUserId(userId);
if (!settings) {
  return res.status(404).json({
    success: false,
    message: 'Lietotāja iestatījumi nav atrasti'
  });
}

const updates = {};

if (hasOwn(req.body, 'language')) updates.language = language;
if (hasOwn(req.body, 'gender')) updates.gender = gender;
if (hasOwn(req.body, 'birthDate')) updates.birthDate = birthDate;
if (hasOwn(req.body, 'heightUnit')) updates.heightUnit = heightUnit;
if (hasOwn(req.body, 'weightUnit')) updates.weightUnit = weightUnit;
if (hasOwn(req.body, 'distanceUnit')) updates.distanceUnit = distanceUnit;
if (hasOwn(req.body, 'calorieGoal')) updates.calorieGoal = calorieGoal;
if (hasOwn(req.body, 'proteinGoal')) updates.proteinGoal = proteinGoal;
if (hasOwn(req.body, 'fatGoal')) updates.fatGoal = fatGoal;
if (hasOwn(req.body, 'carbGoal')) updates.carbGoal = carbGoal;
if (hasOwn(req.body, 'stepGoal')) updates.stepGoal = stepGoal;

if (hasOwn(req.body, 'height')) {

```

```

    // Konvertē augumu uz cm pirms glabāšanas
    const sourceHeightUnit = heightUnit || settings.height_unit || 'cm';
    updates.height = height == null || height === '' ? null : toCentimeters(height,
sourceHeightUnit);
  }

  if (hasOwn(req.body, 'goalWeight')) {
    const sourceWeightUnit = weightUnit || settings.weight_unit || 'kg';
    updates.goalWeight = goalWeight == null || goalWeight === '' ? null :
toKilograms(goalWeight, sourceWeightUnit);
  }

  // Atjaunina iestatījumus - konvertē uz pareizajiem lauku nosaukumiem
  const updatedSettings = await UserSettingsModel.update(userId, updates);

  // Konvertē augumu atpakaļ uz lietotāja vienībām atbildē
  const displayHeight = updatedSettings.height ?
convertHeightFromCm(updatedSettings.height, updatedSettings.height_unit) : null;

  res.json({
    success: true,
    message: 'Lietotāja iestatījumi atjaunināti',
    data: {
      ...updatedSettings,
      height: displayHeight
    }
  });
} catch (error) {
  console.error('Kļūda atjauninot lietotāja iestatījumus:', error);
  res.status(500).json({
    success: false,
    message: 'Iekšējā servera kļūda'
  });
}
});

router.get("/weight", authenticateToken, async (req, res) => {
  try {
    const userId = req.user.userId;
    const settings = await UserSettingsModel.findByUserId(userId);

```

```

const latestWeight = await WeightHistoryModel.getLatestEntry(userId);
const weightUnit = settings?.weight_unit || 'kg';

res.json({
  success: true,
  data: {
    unit: weightUnit,
    latest: latestWeight ? {
      ...latestWeight,
      weight: convertWeightFromKg(latestWeight.weight, weightUnit)
    } : null
  }
});
} catch (error) {
  console.error('Error getting weight:', error);
  res.status(500).json({
    success: false,
    message: 'Internal server error'
  });
}
});

router.post("/weight", authenticateToken, async (req, res) => {
  try {
    const userId = req.user.userId;
    const settings = await UserSettingsModel.findByUserId(userId);
    const weightUnit = settings?.weight_unit || 'kg';
    const weightKg = toKilograms(req.body.weight, req.body.weightUnit || weightUnit);

    if (weightKg == null || weightKg <= 0) {
      return res.status(400).json({
        success: false,
        message: 'Please choose a valid weight'
      });
    }

    const createdWeight = await WeightHistoryModel.addEntry(userId, weightKg);

    res.status(201).json({

```

```

        success: true,
        message: 'Weight saved',
        data: {
            ...createdWeight,
            weight: convertWeightFromKg(createdWeight.weight, weightUnit),
            unit: weightUnit
        }
    });
} catch (error) {
    console.error('Error saving weight:', error);
    res.status(500).json({
        success: false,
        message: 'Internal server error'
    });
}
});

module.exports = router;

```

routes/foods.js

```
// Maršruti nodrošina ēdienu izveidi, izgūšanu, atjaunināšanu un dzēšanu.
// Tie apstrādā gan lietotāja privātos ēdienus, gan publiski pieejamos ēdienus ar
// uzturvērtības datiem.

const express = require('express');
const { authenticateToken } = require('../lib/auth');
const FoodModel = require('../lib/DbModels/foodModel');
const UserModel = require('../lib/DbModels/userModel');
const router = express.Router();

const isAdminRequest = async (req) => {
  const user = await UserModel.findById(req.user.userId);
  return user?.role === 'admin';
};

router.get('/', authenticateToken, async (req, res) => {
  try {
    const foods = await FoodModel.findVisibleToUser(req.user.userId);

    res.json({
      success: true,
      data: foods
    });
  } catch (error) {
    console.error('Error fetching foods:', error);
    res.status(500).json({
      success: false,
      message: 'Failed to fetch foods',
      error: error.message
    });
  }
});

router.get('/:id', authenticateToken, async (req, res) => {
  try {
    const { id } = req.params;
    const food = await FoodModel.findById(id);

    if (!food) {
```

```

    return res.status(404).json({
      success: false,
      message: 'Food not found'
    });
  }

  const isAdmin = await isAdminRequest(req);

  if (!isAdmin && Number(food.user_id) !== Number(req.user.userId) && !food.is_public) {
    return res.status(403).json({
      success: false,
      message: 'You can only view public foods or foods you created'
    });
  }

  res.json({
    success: true,
    data: food
  });
} catch (error) {
  console.error('Error fetching food:', error);
  res.status(500).json({
    success: false,
    message: 'Failed to fetch food',
    error: error.message
  });
}
});

router.post('/', authenticateToken, async (req, res) => {
  try {
    const { name, caloriesPer100g, proteinPer100g, carbsPer100g, fatPer100g, isPublic } =
      req.body;

    if (!name || name.trim().length === 0) {
      return res.status(400).json({
        success: false,
        message: 'Food name is required'
      });
    }
  }
});

```

```

    }

    if (
      caloriesPer100g === undefined ||
      proteinPer100g === undefined ||
      carbsPer100g === undefined ||
      fatPer100g === undefined
    ) {
      return res.status(400).json({
        success: false,
        message: 'All nutrition values are required'
      });
    }

    const food = await FoodModel.createFood(
      req.user.userId,
      name.trim(),
      parseFloat(caloriesPer100g),
      parseFloat(proteinPer100g),
      parseFloat(carbsPer100g),
      parseFloat(fatPer100g),
      isPublic === true
    );

    res.status(201).json({
      success: true,
      message: 'Food created successfully',
      data: food
    });
  } catch (error) {
    console.error('Error creating food:', error);
    if (error.code === 'ER_DUP_ENTRY') {
      return res.status(409).json({
        success: false,
        message: 'Food with this name already exists'
      });
    }
  }

  res.status(500).json({

```



```

        success: false,
        message: 'Failed to create food',
        error: error.message
    });
}
});

router.put('/:id', authenticateToken, async (req, res) => {
    try {
        const { id } = req.params;

        const { name, caloriesPer100g, proteinPer100g, carbsPer100g, fatPer100g, isPublic } =
            req.body;

        const food = await FoodModel.findById(id);
        if (!food) {
            return res.status(404).json({
                success: false,
                message: 'Food not found'
            });
        }

        const isAdmin = await isAdminRequest(req);

        if (!isAdmin && Number(food.user_id) !== Number(req.user.userId)) {
            return res.status(403).json({
                success: false,
                message: 'You can only edit foods you created'
            });
        }

        if (name !== undefined && !name.trim()) {
            return res.status(400).json({
                success: false,
                message: 'Food name is required'
            });
        }

        const nextValues = {
            name: name !== undefined ? name.trim() : food.name,

```

```

        caloriesPer100g: caloriesPer100g !== undefined ? parseFloat(caloriesPer100g) :
food.calories_per_100g,

        proteinPer100g: proteinPer100g !== undefined ? parseFloat(proteinPer100g) :
food.protein_per_100g,

        carbsPer100g: carbsPer100g !== undefined ? parseFloat(carbsPer100g) :
food.carbs_per_100g,

        fatPer100g: fatPer100g !== undefined ? parseFloat(fatPer100g) : food.fat_per_100g,

    };

    if (
        [nextValues.caloriesPer100g, nextValues.proteinPer100g, nextValues.carbsPer100g,
nextValues.fatPer100g]
        .some((value) => Number.isNaN(Number(value)))
    ) {
        return res.status(400).json({
            success: false,
            message: 'Nutrition values must be valid numbers'
        });
    }

    const updated = await FoodModel.update(
        id,
        nextValues.name,
        nextValues.caloriesPer100g,
        nextValues.proteinPer100g,
        nextValues.carbsPer100g,
        nextValues.fatPer100g,
        isPublic !== undefined ? isPublic === true : food.is_public
    );

    if (!updated) {
        return res.status(500).json({
            success: false,
            message: 'Failed to update food'
        });
    }

    const updatedFood = await FoodModel.findById(id);
    res.json({
        success: true,
        message: 'Food updated successfully',
    });

```

```

        data: updatedFood
    });
} catch (error) {
    console.error('Error updating food:', error);
    if (error.code === 'ER_DUP_ENTRY') {
        return res.status(409).json({
            success: false,
            message: 'Food with this name already exists'
        });
    }

    res.status(500).json({
        success: false,
        message: 'Failed to update food',
        error: error.message
    });
}
});

router.delete('/:id', authenticateToken, async (req, res) => {
    try {
        const { id } = req.params;
        const food = await FoodModel.findById(id);

        if (!food) {
            return res.status(404).json({
                success: false,
                message: 'Food not found'
            });
        }

        const isAdmin = await isAdminRequest(req);

        if (!isAdmin && Number(food.user_id) !== Number(req.user.userId)) {
            return res.status(403).json({
                success: false,
                message: 'You can only delete foods you created'
            });
        }
    }

```

```

const deleted = await FoodModel.delete(id);
if (!deleted) {
  return res.status(500).json({
    success: false,
    message: 'Failed to delete food'
  });
}

res.json({
  success: true,
  message: 'Food deleted successfully'
});
} catch (error) {
  console.error('Error deleting food:', error);
  res.status(500).json({
    success: false,
    message: 'Failed to delete food',
    error: error.message
  });
}
});

module.exports = router;

```

routes/meals.js

```
const express = require('express');
const { authenticateToken } = require('../lib/auth');
const MealModel = require('../lib/DbModels/mealModel');
const router = express.Router();

router.get('/', authenticateToken, async (req, res) => {
  try {
    const userId = req.user.userId;
    const meals = await MealModel.findByUserId(userId, req.query.date || null);

    res.json({
      success: true,
      data: meals
    });
  } catch (error) {
    console.error('Error fetching meals:', error);
    res.status(500).json({
      success: false,
      message: 'Failed to fetch meals',
      error: error.message
    });
  }
});

// POST /api/meals/:mealType/foods - ēdienu pievienošana maltītei.
router.post('/:mealType/foods', authenticateToken, async (req, res) => {
  try {
    const { mealType } = req.params;
    const { foods, date } = req.body;
    const userId = req.user.userId;

    if (!Array.isArray(foods) || foods.length === 0) {
      return res.status(400).json({
        success: false,
        message: 'foods must be a non-empty array'
      });
    }
  }
});
```

```

// Tiek iegūts šodienas datums.
const selectedDate = date || new Date().toISOString().split('T')[0];

// Tiek atrasta vai izveidota maltīte.
let mealId = null;
const existingMeals = await MealModel.findByUserId(userId, selectedDate);
const existingMeal = existingMeals.find(m => m.name === mealType);

if (existingMeal) {
    mealId = existingMeal.id;
} else {
    // Ja maltīte neeksistē, tiek izveidota jauna maltīte.
    const newMeal = await MealModel.createMeal(userId, mealType, selectedDate);
    mealId = newMeal.id;
}

// Katrs izvēlētais ēdiens tiek pievienots maltītei.
for (const food of foods) {
    await MealModel.addFood(mealId, food.id, food.quantity, 'g');
}

// Tiek atgriezta atjauninātā maltīte ar ēdieniem.
const updatedMeal = await MealModel.findById(mealId);

res.json({
    success: true,
    message: 'Foods added to meal',
    data: updatedMeal
});
} catch (error) {
    console.error('Error adding foods to meal:', error);
    res.status(500).json({
        success: false,
        message: 'Failed to add foods to meal',
        error: error.message
    });
}
});

```

```

router.delete('/:mealId/foods/:foodId', authenticateToken, async (req, res) => {
  try {
    const { mealId, foodId } = req.params;
    const userId = req.user.userId;

    await MealModel.removeFood(foodId);

    res.json({
      success: true,
      message: 'Food removed from meal'
    });
  } catch (error) {
    console.error('Error removing food from meal:', error);
    res.status(500).json({
      success: false,
      message: 'Failed to remove food from meal',
      error: error.message
    });
  }
});

module.exports = router;

```

routes/exercises.js

```
const express = require('express');
const { authenticateToken } = require('../lib/auth');
const ExerciseModel = require('../lib/DbModels/exerciseModel');
const UserModel = require('../lib/DbModels/userModel');
const router = express.Router();

const isAdminRequest = async (req) => {
  const user = await UserModel.findById(req.user.userId);
  return user?.role === 'admin';
};

/**
 * GET /api/exercises
 * Iegūst visus lietotājam pieejamos vingrinājumus.
 */
router.get('/', authenticateToken, async (req, res) => {
  try {
    const exercises = await ExerciseModel.findVisibleToUser(req.user.userId);

    res.json({
      success: true,
      data: exercises
    });
  } catch (error) {
    console.error('Error fetching exercises:', error);
    res.status(500).json({
      success: false,
      message: 'Failed to fetch exercises',
      error: error.message
    });
  }
});

/**
 * GET /api/exercises/:id
 * Iegūst konkrētu vingrinājumu pēc identifikatora.
 */
```



```

router.get('/:id', authenticateToken, async (req, res) => {
  try {
    const { id } = req.params;

    const exercise = await ExerciseModel.findById(id);
    if (!exercise) {
      return res.status(404).json({
        success: false,
        message: 'Exercise not found'
      });
    }

    const isAdmin = await isAdminRequest(req);

    if (!isAdmin && Number(exercise.user_id) !== Number(req.user.userId) &&
!exercise.is_public) {
      return res.status(403).json({
        success: false,
        message: 'You can only view public exercises or exercises you created'
      });
    }

    res.json({
      success: true,
      data: exercise
    });
  } catch (error) {
    console.error('Error fetching exercise:', error);
    res.status(500).json({
      success: false,
      message: 'Failed to fetch exercise',
      error: error.message
    });
  }
});

router.get('/:id/weight-history', authenticateToken, async (req, res) => {
  try {
    const { id } = req.params;

```

```

const userId = req.user.userId;
const limit = req.query.limit || 12;

const exercise = await ExerciseModel.findById(id);
if (!exercise) {
  return res.status(404).json({
    success: false,
    message: 'Exercise not found'
  });
}

const isAdmin = await isAdminRequest(req);

if (!isAdmin && Number(exercise.user_id) !== Number(userId) && !exercise.is_public) {
  return res.status(403).json({
    success: false,
    message: 'You can only view public exercises or exercises you created'
  });
}

const history = await ExerciseModel.getTopWeightHistory(id, userId, limit);
res.json({
  success: true,
  data: history.map((entry) => ({
    workoutLogId: entry.workout_log_id,
    workoutId: entry.workout_id,
    workoutName: entry.workout_name,
    date: entry.session_date,
    topWeight: entry.top_weight == null ? null : Number(entry.top_weight),
    setCount: entry.set_count,
  }))
});
} catch (error) {
  console.error('Error fetching exercise weight history:', error);
  res.status(500).json({
    success: false,
    message: 'Failed to fetch exercise weight history',
    error: error.message
  });
}

```

```

    }
  });

  /**
   * POST /api/exercises
   * Izveido jaunu vingrinājumu.
   * Obligātais lauks: name.
   * Neobligātie lauki: description, primaryMuscleGroupId, secondaryMuscleGroupIds.
   */
  router.post('/', authenticateToken, async (req, res) => {
    try {
      const { name, description, primaryMuscleGroupId, secondaryMuscleGroupIds, isPublic } =
        req.body;

      // Tiek pārbaudīti obligātie lauki.
      if (!name || name.trim().length === 0) {
        return res.status(400).json({
          success: false,
          message: 'Exercise name is required'
        });
      }

      // Izveido vingrinājumu ar lietotāja ID
      const exercise = await ExerciseModel.create(
        req.user.userId,
        name.trim(),
        description || null,
        isPublic === true
      );

      // Ja padota galvenā muskuļu grupa, tā tiek piesaistīta vingrinājumam.
      if (primaryMuscleGroupId) {
        try {
          await ExerciseModel.addMuscleGroup(exercise.id, primaryMuscleGroupId, true);
        } catch (linkError) {
          console.warn(`Could not link exercise to primary muscle group
            ${primaryMuscleGroupId}:`, linkError.message);
        }
      }
    }
  });

```

```

    // Ja padotas papildu muskuļu grupas, tās tiek piesaistītas vingrinājumam.
    if (secondaryMuscleGroupIds && Array.isArray(secondaryMuscleGroupIds) &&
        secondaryMuscleGroupIds.length > 0) {
        for (const muscleGroupId of secondaryMuscleGroupIds) {
            // Ieraksts tiek izlaists, ja tas sakrīt ar galveno muskuļu grupu.
            if (muscleGroupId === primaryMuscleGroupId) continue;
            try {
                await ExerciseModel.addMuscleGroup(exercise.id, muscleGroupId, false);
            } catch (linkError) {
                console.warn(`Could not link exercise to secondary muscle group
                    ${muscleGroupId}:`, linkError.message);
            }
        }
    }

    // Vingrinājuma dati tiek atkārtoti iegūti ar atjauninātajām muskuļu grupām.
    const updatedExercise = await ExerciseModel.findById(exercise.id);
    return res.status(201).json({
        success: true,
        message: 'Exercise created successfully',
        data: updatedExercise
    });
} catch (error) {
    console.error('Error creating exercise:', error);

    // Tiek apstrādāta nosaukuma dublēšanās kļūda.
    if (error.code === 'ER_DUP_ENTRY') {
        return res.status(409).json({
            success: false,
            message: 'Exercise with this name already exists'
        });
    }

    res.status(500).json({
        success: false,
        message: 'Failed to create exercise',
        error: error.message
    });
}
});

```

```

/**
 * PUT /api/exercises/:id
 * Atjaunina vingrinājuma datus.
 */
router.put('/:id', authenticateToken, async (req, res) => {
  try {
    const { id } = req.params;

    const { name, description, primaryMuscleGroupId, secondaryMuscleGroupIds, isPublic } =
req.body;

    // Tiek pārbaudīts, vai vingrinājums eksistē.
    const exercise = await ExerciseModel.findById(id);
    if (!exercise) {
      return res.status(404).json({
        success: false,
        message: 'Exercise not found'
      });
    }

    const isAdmin = await isAdminRequest(req);

    if (!isAdmin && Number(exercise.user_id) !== Number(req.user.userId)) {
      return res.status(403).json({
        success: false,
        message: 'You can only edit exercises you created'
      });
    }

    if (name !== undefined && !name.trim()) {
      return res.status(400).json({
        success: false,
        message: 'Exercise name is required'
      });
    }

    // Tiek atjaunināts vingrinājums.
    const updated = await ExerciseModel.update(id, {
      name: name !== undefined ? name.trim() : exercise.name,

```

```

        description: description !== undefined ? description : exercise.description,
        isPublic: isPublic !== undefined ? isPublic === true : exercise.is_public
    });

    if (updated) {
        const shouldUpdateMuscleGroups = primaryMuscleGroupId !== undefined ||
secondaryMuscleGroupIds !== undefined;

        const updatedExercise = shouldUpdateMuscleGroups
            ? await ExerciseModel.replaceMuscleGroups(
                id,
                primaryMuscleGroupId || null,
                Array.isArray(secondaryMuscleGroupIds) ? secondaryMuscleGroupIds : []
            )
            : await ExerciseModel.findById(id);

        res.json({
            success: true,
            message: 'Exercise updated successfully',
            data: updatedExercise
        });
    } else {
        res.status(500).json({
            success: false,
            message: 'Failed to update exercise'
        });
    }
} catch (error) {
    console.error('Error updating exercise:', error);
    res.status(500).json({
        success: false,
        message: 'Failed to update exercise',
        error: error.message
    });
}
});

/**
 * DELETE /api/exercises/:id
 * Dzēš vingrinājumu.
 */

```

```

router.delete('/:id', authenticateToken, async (req, res) => {
  try {
    const { id } = req.params;

    // Tiek pārbaudīts, vai vingrinājums eksistē.
    const exercise = await ExerciseModel.findById(id);
    if (!exercise) {
      return res.status(404).json({
        success: false,
        message: 'Exercise not found'
      });
    }

    const isAdmin = await isAdminRequest(req);

    if (!isAdmin && Number(exercise.user_id) !== Number(req.user.userId)) {
      return res.status(403).json({
        success: false,
        message: 'You can only delete exercises you created'
      });
    }

    // Tiek dzēsts vingrinājums.
    const deleted = await ExerciseModel.delete(id);
    if (deleted) {
      res.json({
        success: true,
        message: 'Exercise deleted successfully'
      });
    } else {
      res.status(500).json({
        success: false,
        message: 'Failed to delete exercise'
      });
    }
  } catch (error) {
    console.error('Error deleting exercise:', error);
    res.status(500).json({
      success: false,

```

```

        message: 'Failed to delete exercise',
        error: error.message
    });
}
});

/**
 * POST /api/exercises/:id/muscle-groups/:muscleGroupId
 * Pievieno muskuļu grupu vingrinājumam.
 */
router.post('/:id/muscle-groups/:muscleGroupId', authenticateToken, async (req, res) => {
    try {
        const { id, muscleGroupId } = req.params;

        // Tiek pārbaudīts, vai vingrinājums eksistē.
        const exercise = await ExerciseModel.findById(id);
        if (!exercise) {
            return res.status(404).json({
                success: false,
                message: 'Exercise not found'
            });
        }

        if (Number(exercise.user_id) !== Number(req.user.userId)) {
            return res.status(403).json({
                success: false,
                message: 'You can only edit exercises you created'
            });
        }

        // Tiek pievienota muskuļu grupa.
        await ExerciseModel.addMuscleGroup(id, muscleGroupId);

        const updatedExercise = await ExerciseModel.findById(id);
        res.json({
            success: true,
            message: 'Muscle group added to exercise',
            data: updatedExercise
        });
    }
});

```



```

    } catch (error) {
      console.error('Error adding muscle group to exercise:', error);

      if (error.code === 'ER_DUP_ENTRY') {
        return res.status(409).json({
          success: false,
          message: 'This exercise is already linked to this muscle group'
        });
      }

      res.status(500).json({
        success: false,
        message: 'Failed to add muscle group',
        error: error.message
      });
    }
  });

  /**
   * DELETE /api/exercises/:id/muscle-groups/:muscleGroupId
   * Noņem muskuļu grupu no vingrinājuma.
   */
  router.delete('/:id/muscle-groups/:muscleGroupId', authenticateToken, async (req, res) => {
    try {
      const { id, muscleGroupId } = req.params;

      // Tiek pārbaudīts, vai vingrinājums eksistē.
      const exercise = await ExerciseModel.findById(id);
      if (!exercise) {
        return res.status(404).json({
          success: false,
          message: 'Exercise not found'
        });
      }

      if (Number(exercise.user_id) !== Number(req.user.userId)) {
        return res.status(403).json({
          success: false,
          message: 'You can only edit exercises you created'
        });
      }
    }
  });

```

```

    });
  }

  // Tiek noņemta muskuļu grupa.
  const removed = await ExerciseModel.removeMuscleGroup(id, muscleGroupId);
  if (removed) {
    const updatedExercise = await ExerciseModel.findById(id);
    res.json({
      success: true,
      message: 'Muscle group removed from exercise',
      data: updatedExercise
    });
  } else {
    res.status(404).json({
      success: false,
      message: 'Link between exercise and muscle group not found'
    });
  }
} catch (error) {
  console.error('Error removing muscle group from exercise:', error);
  res.status(500).json({
    success: false,
    message: 'Failed to remove muscle group',
    error: error.message
  });
}
});

module.exports = router;

```

routes/dashboard.js

```
// Maršruts apkopo lietotāja paneļa datus par uzturu, treniņiem, soļiem un svaru.
// Tas sagatavo kopsavilkuma informāciju, kas mobilajā lietotnē tiek rādīta galvenajā
// ekrānā.

const express = require('express');
const { authenticateToken } = require('../lib/auth');
const MealModel = require('../lib/DbModels/mealModel');
const UserSettingsModel = require('../lib/DbModels/userSettingsModel');
const WeightHistoryModel = require('../lib/DbModels/weightHistoryModel');
const WorkoutModel = require('../lib/DbModels/workoutModel');
const WorkoutLogModel = require('../lib/DbModels/workoutLogModel');

const router = express.Router();

const DEFAULTS = {
  calorieGoal: 1600,
  proteinGoal: 30,
  fatGoal: 40,
  carbGoal: 30,
  stepGoal: 10000,
};

const toNumber = (value, fallback = 0) => {
  const number = Number(value);
  return Number.isFinite(number) ? number : fallback;
};

const round = (value, digits = 0) => {
  const factor = 10 ** digits;
  return Math.round(toNumber(value) * factor) / factor;
};

const convertWeightFromKg = (weight, unit = 'kg') => {
  if (weight == null) {
    return null;
  }
  const value = toNumber(weight);
  return unit === 'lb' || unit === 'lbs' ? value / 0.45359237 : value;
}
```

```

};

const getFoodCalories = (food) => (
  toNumber(food.calories_per_100g) * toNumber(food.quantity, 100) / 100
);

const getFoodMacro = (food, key) => (
  toNumber(food[key]) * toNumber(food.quantity, 100) / 100
);

const getDurationMinutes = (workoutLog) => {
  if (!workoutLog.started_at || !workoutLog.completed_at) {
    return 0;
  }

  const startedAt = new Date(workoutLog.started_at).getTime();
  const completedAt = new Date(workoutLog.completed_at).getTime();

  if (!Number.isFinite(startedAt) || !Number.isFinite(completedAt) || completedAt <=
    startedAt) {
    return 0;
  }

  return Math.round((completedAt - startedAt) / 60000);
};

const formatPeriodRange = (entries) => {
  const dates = entries
    .map((entry) => new Date(entry?.created_at))
    .filter((date) => !Number.isNaN(date.getTime()));

  if (!dates.length) {
    return '';
  }

  const sameYear = dates[0].getFullYear() === dates[dates.length - 1].getFullYear();
  const formatter = new Intl.DateTimeFormat('en', {
    month: 'short',
    day: 'numeric',
  });

```

```

    ...(sameYear ? {} : { year: 'numeric' }),
  });

  const first = formatter.format(dates[0]);
  const last = formatter.format(dates[dates.length - 1]);

  return first === last ? first : `${first} - ${last}`;
};

const uniqueById = (entries) => {
  const seen = new Set();
  return entries.filter((entry) => {
    if (!entry?.id || seen.has(entry.id)) {
      return false;
    }

    seen.add(entry.id);
    return true;
  });
};

const safely = async (fallback, task) => {
  try {
    return await task();
  } catch (error) {
    console.warn('Dashboard partial data unavailable:', error.message);
    return fallback;
  }
};

router.get('/', authenticateToken, async (req, res) => {
  try {
    const userId = req.user.userId;
    const weightPeriod = req.query.weightPeriod.toLowerCase() || 'month';

    const [settings, meals, workouts, workoutLogs, weightHistory] = await Promise.all([
      safely([], () => UserSettingsModel.findById(userId)),
      safely([], () => MealModel.findById(userId)),
      safely([], () => WorkoutModel.findById(userId, 500, 0)),
    ]);
  }
});

```

```

    safely([], () => WorkoutLogModel.findByUserId(userId, 500, 0)),
    safely([], () => WeightHistoryModel.findByPeriod(userId, weightPeriod)),
  ]);

  const foods = meals.flatMap((meal) => meal.foods || []);
  const caloriesFromMeals = foods.reduce((sum, food) => sum + getFoodCalories(food), 0);
  const protein = foods.reduce((sum, food) => sum + getFoodMacro(food, 'protein_per_100g'), 0);
  const fat = foods.reduce((sum, food) => sum + getFoodMacro(food, 'fat_per_100g'), 0);
  const carbs = foods.reduce((sum, food) => sum + getFoodMacro(food, 'carbs_per_100g'), 0);

  const macroTotal = protein + fat + carbs;
  const proteinGoalGrams = toNumber(settings?.protein_goal);
  const fatGoalGrams = toNumber(settings?.fat_goal);
  const carbGoalGrams = toNumber(settings?.carb_goal);
  const macroGoalTotal = proteinGoalGrams + fatGoalGrams + carbGoalGrams;

  const completedLogs = workoutLogs.filter((log) => log.completed_at);
  const now = new Date();
  const monthStart = new Date(now.getFullYear(), now.getMonth(), 1);
  const completedThisMonth = completedLogs.filter((log) => new Date(log.completed_at) >= monthStart);

  const workoutMinutes = completedThisMonth.reduce((sum, log) => sum + getDurationMinutes(log), 0);

  const calorieGoal = toNumber(settings?.calorie_goal, DEFAULTS.calorieGoal);
  const workoutCalories = 0;
  const displayWeightUnit = settings?.weight_unit || 'kg';

  const latestWeight = weightHistory[0] || null;
  const oldestWeight = weightHistory[weightHistory.length - 1] || null;

  res.json({
    success: true,
    data: {
      goals: {
        calories: calorieGoal,
        protein: macroGoalTotal ? round((proteinGoalGrams / macroGoalTotal) * 100) : DEFAULTS.proteinGoal,
        fat: macroGoalTotal ? round((fatGoalGrams / macroGoalTotal) * 100) : DEFAULTS.fatGoal,

```

```

        carbs: macroGoalTotal ? round((carbGoalGrams / macroGoalTotal) * 100) :
DEFAULTS.carbGoal,
        weight: settings?.goal_weight ? toNumber(settings.goal_weight) : null,
        steps: toNumber(settings?.step_goal, DEFAULTS.stepGoal),
    },
    nutrition: {
        meals,
        caloriesFromMeals: round(caloriesFromMeals),
        caloriesFromWorkouts: workoutCalories,
        caloriesRemaining: round(calorieGoal - caloriesFromMeals + workoutCalories),
        macros: {
            protein: {
                grams: round(protein, 1),
                percent: macroTotal ? round((protein / macroTotal) * 100) : 0,
            },
            fat: {
                grams: round(fat, 1),
                percent: macroTotal ? round((fat / macroTotal) * 100) : 0,
            },
            carbs: {
                grams: round(carbs, 1),
                percent: macroTotal ? round((carbs / macroTotal) * 100) : 0,
            },
        },
    },
    workouts: {
        total: workouts.length,
        completedThisMonth: completedThisMonth.length,
        caloriesBurned: workoutCalories,
        durationMinutes: workoutMinutes,
    },
    steps: {
        today: 0,
        goal: toNumber(settings?.step_goal, DEFAULTS.stepGoal),
    },
    weight: {
        period: weightPeriod,
        entries: weightHistory.map((entry) => ({
            id: entry.id,

```

```

        weight: round(convertWeightFromKg(entry.weight, displayWeightUnit), 1),
        created_at: entry.created_at,
    })),
    latest: latestWeight ? round(convertWeightFromKg(latestWeight.weight,
displayWeightUnit), 1) : null,
    comparison: oldestWeight ? round(convertWeightFromKg(oldestWeight.weight,
displayWeightUnit), 1) : null,
    change: latestWeight && oldestWeight
        ? round(convertWeightFromKg(toNumber(latestWeight.weight) -
toNumber(oldestWeight.weight), displayWeightUnit), 1)
        : null,
    goal: settings?.goal_weight ? round(convertWeightFromKg(settings.goal_weight,
displayWeightUnit), 1) : null,
    unit: displayWeightUnit,
},
},
});
} catch (error) {
    console.error('Error fetching dashboard:', error);
    res.status(500).json({
        success: false,
        message: 'Failed to fetch dashboard data',
        error: error.message,
    });
}
});

module.exports = router;

```


routes/muscleGroups.js

```
const express = require('express');
const { authenticateToken } = require('../lib/auth');
const MuscleGroupModel = require('../lib/DbModels/muscleGroupModel');
const router = express.Router();

/**
 * GET /api/muscle-groups
 * Iegūst visas muskuļu grupas.
 */
router.get('/', async (req, res) => {
  try {
    const muscleGroups = await MuscleGroupModel.findAll();

    res.json({
      success: true,
      data: muscleGroups
    });
  } catch (error) {
    console.error('Error fetching muscle groups:', error);
    res.status(500).json({
      success: false,
      message: 'Failed to fetch muscle groups',
      error: error.message
    });
  }
});

/**
 * GET /api/muscle-groups/:id
 * Iegūst konkrētu muskuļu grupu pēc identifikatora.
 */
router.get('/:id', async (req, res) => {
  try {
    const { id } = req.params;

    const muscleGroup = await MuscleGroupModel.findById(id);
    if (!muscleGroup) {
```

```

        return res.status(404).json({
            success: false,
            message: 'Muscle group not found'
        });
    }

    res.json({
        success: true,
        data: muscleGroup
    });
} catch (error) {
    console.error('Error fetching muscle group:', error);
    res.status(500).json({
        success: false,
        message: 'Failed to fetch muscle group',
        error: error.message
    });
}
});

/**
 * POST /api/muscle-groups
 * Izveido jaunu muskuļu grupu.
 * Obligātais lauks: name.
 * Neobligātais lauks: description.
 */
router.post('/', authenticateToken, async (req, res) => {
    try {
        const { name, description } = req.body;

        // Tiek pārbaudīti obligātie lauki.
        if (!name || name.trim().length === 0) {
            return res.status(400).json({
                success: false,
                message: 'Muscle group name is required'
            });
        }

        // Tiek izveidota muskuļu grupa.

```

```

const muscleGroup = await MuscleGroupModel.create(
  name.trim(),
  description || null
);

res.status(201).json({
  success: true,
  message: 'Muscle group created successfully',
  data: muscleGroup
});
} catch (error) {
  console.error('Error creating muscle group:', error);

  // Tiek apstrādāta nosaukuma dublēšanās kļūda.
  if (error.code === 'ER_DUP_ENTRY') {
    return res.status(409).json({
      success: false,
      message: 'Muscle group with this name already exists'
    });
  }

  res.status(500).json({
    success: false,
    message: 'Failed to create muscle group',
    error: error.message
  });
}
});

module.exports = router;

```

routes/admin.js

```
// Maršruti nodrošina administratora darbības ar lietotājiem un viņu izveidoto saturu.
// Šajā failā tiek apstrādāta lietotāju saraksta iegūšana, detalizēta apskate, rediģēšana
un bloķēšana.

const express = require('express');
const { AuthService, authenticateToken, requireAdmin } = require('../lib/auth');
const UserModel = require('../lib/DbModels/userModel');
const UserSettingsModel = require('../lib/DbModels/userSettingsModel');
const FoodModel = require('../lib/DbModels/foodModel');
const ExerciseModel = require('../lib/DbModels/exerciseModel');

const router = express.Router();

router.use(authenticateToken, requireAdmin);

router.get('/users', async (req, res) => {
  try {
    const users = await UserModel.findAll(500, 0);

    res.json({
      success: true,
      data: users
    });
  } catch (error) {
    console.error('Error fetching admin users:', error);
    res.status(500).json({
      success: false,
      message: 'Failed to fetch users',
      error: error.message
    });
  }
});

router.get('/users/:id', async (req, res) => {
  try {
    const user = await UserModel.findByIdForAdmin(req.params.id);

    if (!user) {
      return res.status(404).json({
```

```

        success: false,
        message: 'User not found'
    });
}

const settings = await UserSettingsModel.findById(user.id);

res.json({
    success: true,
    data: {
        user,
        settings
    }
});
} catch (error) {
    console.error('Error fetching admin user:', error);
    res.status(500).json({
        success: false,
        message: 'Failed to fetch user',
        error: error.message
    });
}
});

router.put('/users/:id', async (req, res) => {
    try {
        const { name, email, role } = req.body;
        const user = await UserModel.findByIdForAdmin(req.params.id);

        if (!user) {
            return res.status(404).json({
                success: false,
                message: 'User not found'
            });
        }

        if (email !== undefined && !email.trim().includes('@')) {
            return res.status(400).json({
                success: false,

```

```

        message: 'Please enter a valid email address'
    });
}

if (name !== undefined && !name.trim()) {
    return res.status(400).json({
        success: false,
        message: 'Name is required'
    });
}

if (role !== undefined && ![ 'user', 'admin' ].includes(role)) {
    return res.status(400).json({
        success: false,
        message: 'Role must be user or admin'
    });
}

if (email !== undefined && email.toLowerCase() !== user.email.toLowerCase()) {
    const existingUser = await UserModel.findByEmailIncludingInactive(email);
    if (existingUser && Number(existingUser.id) !== Number(user.id)) {
        return res.status(409).json({
            success: false,
            message: 'User with this email already exists'
        });
    }
}

const updatedUser = await UserModel.updateProfile(user.id, {
    name: name !== undefined ? name.trim() : undefined,
    email: email !== undefined ? email.trim() : undefined,
    role
});

res.json({
    success: true,
    message: 'User updated successfully',
    data: updatedUser
});

```

```

    } catch (error) {
      console.error('Error updating admin user:', error);
      res.status(500).json({
        success: false,
        message: 'Failed to update user',
        error: error.message
      });
    }
  });
});

router.delete('/users/:id', async (req, res) => {
  try {
    const user = await UserModel.findByIdForAdmin(req.params.id);

    if (!user) {
      return res.status(404).json({
        success: false,
        message: 'User not found'
      });
    }

    if (Number(user.id) === Number(req.user.userId)) {
      return res.status(400).json({
        success: false,
        message: 'You cannot delete your own admin account'
      });
    }

    const deleted = await UserModel.deactivate(user.id);
    if (!deleted) {
      return res.status(500).json({
        success: false,
        message: 'Failed to delete user'
      });
    }

    await AuthService.removeAllUserRefreshTokens(user.id);

    res.json({

```

```

        success: true,
        message: 'User deactivated successfully'
    });
} catch (error) {
    console.error('Error deleting admin user:', error);
    res.status(500).json({
        success: false,
        message: 'Failed to deactivate user',
        error: error.message
    });
}
});

router.post('/users/:id/reactivate', async (req, res) => {
    try {
        const user = await UserModel.findByIdForAdmin(req.params.id);

        if (!user) {
            return res.status(404).json({
                success: false,
                message: 'User not found'
            });
        }

        const reactivated = await UserModel.reactivate(user.id);
        if (!reactivated) {
            return res.status(500).json({
                success: false,
                message: 'Failed to reactivate user'
            });
        }

        const updatedUser = await UserModel.findByIdForAdmin(user.id);
        res.json({
            success: true,
            message: 'User reactivated successfully',
            data: updatedUser
        });
    } catch (error) {

```



```

    console.error('Error reactivating admin user:', error);
    res.status(500).json({
      success: false,
      message: 'Failed to reactivate user',
      error: error.message
    });
  }
});

router.get('/users/:id/foods', async (req, res) => {
  try {
    const user = await UserModel.findByIdForAdmin(req.params.id);
    if (!user) {
      return res.status(404).json({
        success: false,
        message: 'User not found'
      });
    }

    const foods = await FoodModel.findByIdByUserId(user.id);
    res.json({
      success: true,
      data: foods
    });
  } catch (error) {
    console.error('Error fetching admin user foods:', error);
    res.status(500).json({
      success: false,
      message: 'Failed to fetch user foods',
      error: error.message
    });
  }
});

router.get('/users/:id/exercises', async (req, res) => {
  try {
    const user = await UserModel.findByIdForAdmin(req.params.id);
    if (!user) {
      return res.status(404).json({

```

```

        success: false,
        message: 'User not found'
    });
}

const exercises = await ExerciseModel.findById(user.id);
res.json({
    success: true,
    data: exercises
});
} catch (error) {
    console.error('Error fetching admin user exercises:', error);
    res.status(500).json({
        success: false,
        message: 'Failed to fetch user exercises',
        error: error.message
    });
}
});

module.exports = router;

```